Pruebas de rendimiento

© JMA 2020. All rights reserved

CONCEPTOS BÁSICOS

© JMA 2020. All rights reserved

Introducción

- El rendimiento, o más bien la eficiencia del desempeño, se define como el desempeño relativo a la cantidad de recursos utilizados en las condiciones establecidas. [ISO25010].
- El rendimiento se relaciona con la duración de la ejecución de las tareas por parte de un sistema de TI y la carga que dicho sistema puede manejar. Dado que existen muchos usos diferentes de los sistemas de TI, las expectativas de rendimiento de los usuarios también serán diferentes. Ejemplos de rendimiento que suelen ser importantes son el tiempo de respuesta de un sistema en tiempo real, la duración de un proceso por lotes y la cantidad de usuarios que se pueden manejar simultáneamente.
- Al probar el rendimiento, intervienen varias capas de un sistema de TI. Cada capa puede tener un efecto específico en el rendimiento general experimentado por el usuario.
- La norma ISO25010 define tres subcaracterísticas de rendimiento:
  - Comportamiento: Grado en el que los tiempos de respuesta y procesamiento y las tasas de rendimiento de un producto o sistema, al realizar sus funciones, cumplen con los requisitos.
  - Utilización de recursos: Grado en el que las cantidades y tipos de recursos utilizados por un producto o sistema, al realizar sus funciones, cumplen con los requisitos.
  - Capacidad: Grado en que los límites máximos de un producto o parámetro del sistema cumplen con los requisitos.

© JMA 2020. All rights reserved

Pruebas de rendimiento

- Las pruebas de rendimiento establecen si un sistema de TI cumple con los requisitos de rendimiento pertinentes. Tratar correctamente los requisitos de rendimiento se vuelve aún más importante cuando la tecnología se integra en todos los aspectos de nuestra vida profesional y personal (es decir, computación basada en la nube, soluciones móviles e "Internet de las cosas").
- En la vida real, los requisitos de desempeño a menudo no están especificados. Para los actores empresariales es evidente que el desempeño debe ser bueno, pero muchas veces no son capaces de especificar lo que para ellos significa "bueno". Durante el refinamiento de las historias de usuario (u otros documentos de requisitos), se deben prestar especial atención a los requisitos no funcionales, como la eficiencia del desempeño. E incluso cuando falten por completo los requisitos de rendimiento, será útil que se organicen y realicen pruebas de rendimiento.
- Los aspectos técnicos de las pruebas de rendimiento deben responder preguntas sobre el comportamiento del tiempo, la utilización de recursos y la capacidad. La implementación técnica específica depende de las herramientas e infraestructura disponibles.

© JMA 2020. All rights reserved

Importancia de las pruebas de rendimiento

- **Experiencia de usuario**: Un rendimiento deficiente (tiempos de respuesta lentos, errores) frustra a los usuarios y puede llevar al abandono de la aplicación o sitio web.
- **Reputación de la marca**: Un software lento o inestable daña la imagen de una empresa y la confianza del cliente.
- **Pérdidas económicas**: En aplicaciones de comercio electrónico o servicios críticos, un rendimiento deficiente puede traducirse directamente en pérdidas de ventas o interrupciones del negocio.
- **Identificación de cuellos de botella**: Revelan dónde se encuentran las limitaciones del sistema (base de datos, servidor de aplicaciones, red, código, etc.) antes de que se conviertan en problemas en producción.
- **Planificación de la capacidad**: Ayudan a prever la infraestructura necesaria para soportar un número creciente de usuarios o transacciones.
- **Validación de SLAs**: Permiten verificar si la aplicación cumple con los Acuerdos de Nivel de Servicio (SLAs) establecidos.

© JMA 2020. All rights reserved

Principios

- Las pruebas deben estar alineadas con las expectativas definidas de los diferentes grupos de implicados, en particular los usuarios, los diseñadores del sistema y el personal de operaciones.
- Las pruebas deben ser reproducibles. Deberán obtenerse resultados estadísticamente idénticos (dentro de una tolerancia especificada) repitiendo las pruebas en un sistema que no haya sufrido cambios.
- Las pruebas deben aportar resultados que sean comprensibles y que puedan compararse fácilmente con las expectativas de los implicados.
- Se pueden realizar las pruebas, cuando los recursos lo permitan, tanto en sistemas completos como parciales o en entornos de prueba representativos del sistema de producción.
- Las pruebas deben ser asequibles, desde el punto de vista práctico, y ejecutables dentro del plazo establecido por el proyecto.

© JMA 2020. All rights reserved

Tipos de prueba

- Con el paso del tiempo:
  - Las pruebas de rendimiento se han ido especializando en múltiples tipos de pruebas que ponen el foco en aspectos concretos del rendimiento.
  - Aparecen nuevas tecnologías, metodologías, arquitecturas, … por lo que surgen nuevos aspectos que requieren nuevos tipos de pruebas mas específicos.
  - Se han extendido del nivel de sistema original al resto de niveles.

© JMA 2020. All rights reserved

Pruebas de carga, estrés y picos

- **Pruebas de carga** (load test): pruebas para determinar y validar la respuesta de la aplicación cuando es sometida a una carga de usuarios y/o transacciones que se espera en el ambiente de producción. Ejemplo: verificar la correcta respuesta de la aplicación ante el alta de 100 usuarios en forma simultanea. Se compara con el volumen esperado.
- **Pruebas de estrés** (stress test): pruebas para encontrar el volumen de datos o de tiempo en que la aplicación comienza a fallar o es incapaz de responder a las peticiones. Son pruebas de carga o rendimiento, pero superando los límites esperados en el ambiente de producción y/o determinados en las pruebas. Ejemplo: encontrar la cantidad de usuarios simultáneos, en que la aplicación deja de responder (cuelgue o time out) en forma correcta a todas las peticiones.
- **Pruebas de picos** (spike testing): es un sub tipo de prueba de estrés que mide el rendimiento del software bajo un «pico» significativo y repentino o una carga de trabajo creciente como la de los usuarios simulados. Indica si el software puede manejar ese aumento abrupto de la carga de trabajo de forma repetida y rápida.

© JMA 2020. All rights reserved

Pruebas de resistencia, volumen y escalabilidad

- **Pruebas de resistencia** (soak testing, endurance testing): evalúa el rendimiento del software durante un periodo prolongado bajo una carga de trabajo regular y fija, determina cuánto tiempo puede soportar el software una carga de trabajo constante para proporcionar sostenibilidad a largo plazo. Durante estas pruebas, los equipos de pruebas supervisan los KPI como las fugas de memoria, de proceso, etc. Las pruebas de resistencia también analizan los tiempos de respuesta y el rendimiento tras un uso prolongado para mostrar si estas métricas son consistentes.
- **Pruebas de volumen** (volume testing): comprueban la eficacia del software cuando se somete a grandes volúmenes de datos. Comprueba la pérdida de datos, el tiempo de respuesta del sistema, la fiabilidad del almacenamiento de datos, etc.
- **Pruebas de escalabilidad** (scalability testing): miden la eficacia del software a la hora de manejar una cantidad creciente de carga de trabajo, añadiendo volumen de datos o usuarios de forma gradual mientras se supervisa el rendimiento del software. La prueba informará sobre el comportamiento cuando aumenten o disminuyan los atributos de rendimiento del software.

© JMA 2020. All rights reserved

Pruebas de concurrencia y capacidad

- **Prueba de Concurrencia** (concurrency testing): La prueba de concurrencia se centra en el impacto de situaciones en las que se realizan acciones específicas simultáneamente (por ejemplo, cuando un gran número de usuarios inicia sesión al mismo tiempo). Los problemas de concurrencia son notoriamente difíciles de encontrar y reproducir, particularmente cuando el problema ocurre en un entorno donde la prueba tiene poco o ningún control, como en el caso de la producción.

- **Prueba de Capacidad** (capacity testing): La prueba de capacidad determina el número de usuarios y/o transacciones que un sistema dado soportará sin dejar de cumplir los objetivos de rendimiento establecidos. Estos objetivos también pueden expresarse en relación con los volúmenes de datos resultantes de las transacciones.

© JMA 2020. All rights reserved

Variedad de pruebas

- Las pruebas cubren una gran cantidad de escenarios por lo que tienen una gran variedad de clasificaciones en función de la metodología, enfoque, objetivos, criterios, …
  - Tipo: **Estáticas y Dinámicas**
  - Información: Pruebas de defectos y **Pruebas estadísticas**
  - Nivel: **Unitarias, Integración, *Sistema*, Aceptación**
  - Modo: Pruebas manuales o **automatizadas**
  - Características de calidad: Pruebas funcionales y **no funcionales** (rendimiento, estructurales, de mantenimiento, seguridad, …)
  - Enfoque: Basadas en la especificación, en la estructura o en la experiencia.
  - Estrategias: Pruebas de humo, progresión, regresión, aprendizaje, exploratorias, …

© JMA 2020. All rights reserved

Pruebas Estáticas

- Las actividades de prueba estática son, a menudo, más importantes para las pruebas de rendimiento que para las pruebas de adecuación funcional. Esto se debe a que se introducen muchos defectos críticos de rendimiento en la arquitectura y el diseño del sistema. Estos defectos pueden ser introducidos por malentendidos o por la falta de conocimiento de los diseñadores y arquitectos. Estos defectos también pueden introducirse porque los requisitos no reflejaban adecuadamente el tiempo de respuesta, el rendimiento o los objetivos de utilización de recursos, la carga y el uso previstos del sistema, o las restricciones.
- Las actividades de prueba estática para determinar el rendimiento pueden incluir:
  - Revisiones de los requisitos, con especial atención a los aspectos de rendimiento y los riesgos.
  - Revisiones de esquemas de bases de datos, diagramas de entidad-relación, metadatos, procedimientos almacenados y consultas.
  - Revisiones de la arquitectura del sistema y de la red.
  - Revisiones de los modelos de despligue.
  - Revisiones de fragmentos críticos de código del sistema (por ejemplo, algoritmos complejos).

© JMA 2020. All rights reserved

Pruebas Dinámicas

- A medida que se construye el sistema, la prueba dinámica de rendimiento debe comenzar tan pronto como sea posible. Las oportunidades para la prueba dinámica de rendimiento incluyen:
  - Durante la *prueba unitaria*, incluyendo el uso de información de perfilado para determinar cuellos de botella potenciales y el análisis dinámico para evaluar la utilización de los recursos.
  - Durante la *prueba de integración* de componentes, a través de casos de uso clave y flujos de trabajo, especialmente cuando se integran diferentes prestaciones de casos de uso o cuando se integran con la estructura "troncal" de un flujo de trabajo.
  - Durante la *prueba de sistema* de los comportamientos globales de extremo a extremo bajo diversas condiciones de carga.
  - Durante la *prueba de integración de sistemas*, especialmente para flujos de datos y flujos de trabajo a través de interfaces clave entre sistemas. En la prueba de integración de sistemas no es infrecuente que el "usuario" sea otro sistema o máquina (por ejemplo, entradas de entradas de sensores y otros sistemas).
  - Durante la *prueba de aceptación*, para generar confianza en el usuario, el cliente y el operador con respecto al rendimiento adecuado del sistema y para ajustar el sistema en condiciones reales (pero generalmente no para encontrar defectos de rendimiento en el sistema).

© JMA 2020. All rights reserved

Concepto de carga

- Para llevar a cabo los diversos tipos de pruebas de rendimiento, se deben modelar, generar y suministrar cargas representativas del sistema al sistema sujeto a prueba. Las cargas son comparables a las entradas de datos utilizadas para los casos de pruebas funcionales, pero difieren en los siguientes aspectos:
  - Una carga de prueba de rendimiento debe representar muchas entradas de usuario, no sólo una.
  - Una carga de prueba de rendimiento puede requerir hardware y herramientas dedicadas para su generación.
  - La generación de una carga de prueba de rendimiento depende de la ausencia de defectos funcionales en el sistema sujeto a prueba que puedan afectar a la ejecución de la prueba.
- La generación eficiente y fiable de una carga específica es un factor de éxito clave en la ejecución de las pruebas de rendimiento. Existen diferentes opciones para la generación de cargas.

© JMA 2020. All rights reserved

Requisitos para las pruebas

- Herramientas de captura y reproducción de tráfico de red (proxy o record & replay):
  - diseño, construcción y mantenimiento del entorno de prueba (tipo producción) y uso de las herramientas disponibles para crear scripts de prueba de rendimiento que coincidan con los perfiles de usuario designados.
- Configuración del generador de carga múltiple:
  - diseño e implementación del escenario de rendimiento (combinación de scripts de prueba) para simular la carga en el sistema como se define en el modelo de carga a niveles similares a los de producción.
- Configuración del laboratorio de pruebas:
  - diseño y mantenimiento de un laboratorio con suficiente capacidad o flexibilidad para ejecutar múltiples aplicaciones en un escenario de prueba (se necesitan habilidades adicionales en componentes de red, administración de máquinas virtuales, experiencia en redes, etc.)
- Monitoreo del entorno :
  - ejecutar y mantener herramientas similares a las utilizadas en el entorno de producción con herramientas de prueba de rendimiento capaces de vincularse con esos resultados de monitoreo.

© JMA 2020. All rights reserved

Perfiles

- Un **perfil operativo** describe en términos cuantitativos cómo un tipo particular de usuario utiliza el sistema. El perfil describe qué transacciones realiza un usuario y con qué frecuencia. Un perfil operativo describe el uso realista respondiendo a la pregunta: "Cuando el sistema está en esta condición, ¿qué posibilidades hay de que el usuario lleve a cabo esta acción?" Este perfil proporciona un promedio estadístico de cómo 'el usuario' maneja el sistema y, si se distinguen tipos de usuarios con comportamientos significativamente diferente, es aconsejable crear un perfil operativo separado para cada tipo. La prueba tiene como objetivo examinar si: "El sistema sigue funcionando correctamente cuando las transacciones se han realizado con frecuencia y durante un largo tiempo".
- Los **perfiles de carga** muestran el grado en que los recursos del sistema (CPU, memoria, capacidad de red) están cargados en realidad. La carga suele mostrarse en términos de la cantidad de usuarios o transacciones en un período determinado. Por lo general, la carga de un sistema no es uniforme de forma continua, sino que varía a lo largo del tiempo: hay picos y valles en un tramo de 24 horas. A menudo, los fines de semana mostrarán una carga diferente a la de los días laborables. Y durante los períodos de vacaciones y días festivos, la carga de un sistema puede volver a verse diferente.

© JMA 2020. All rights reserved

Diseño de pruebas

- Un **modelo de carga** es una descripción de los distintos niveles de carga esperados en un sistema de TI, que es la base para especificar y ejecutar pruebas de rendimiento.
- El modelo de carga consta de las siguientes partes:
  - Requisitos de desempeño: Los requisitos están inventariados y, si es necesario, detallados. Un requisito en cuanto al número de usuarios simultáneos, por ejemplo, puede ser inexacto debido al ritmo de trabajo variable, también porque no se sabe con qué frecuencia un usuario realiza una transacción. Por lo tanto, se requiere información adicional, preferiblemente el número de transacciones/accesos por unidad de tiempo.
  - Objeto de prueba: En base a los requisitos y la estimación de riesgos, se decide sobre el objeto de prueba (p. ej., la aplicación (cadena) completa o tal vez solo una base de datos individual o un servidor de aplicaciones). Esto tiene un impacto directo en los entornos de prueba requeridos.
  - Carga a generar: El objetivo final de un modelo de carga es generar una carga representativa en el sistema. Esta carga consta de un número específico de usuarios, cada uno de los cuales realiza determinadas tareas/transacciones (lo mas realista posible, usar un record & replay). Posteriormente se determina la frecuencia con la que se están realizando las transacciones y el número de usuarios requeridos para un período de tiempo específico.

© JMA 2020. All rights reserved

Diseño de pruebas

- El **modelo de iteración** describe a nivel técnico cómo se cargará el objeto de prueba y cómo se medirá el rendimiento. Un modelo de iteración describe la forma (técnica) en la que se distribuye la carga, de modo que se pueda lograr rápidamente una combinación dinámica de acciones, que luego se puede mantener durante un cierto período de tiempo. El modelo de iteración depende del tipo de prueba de rendimiento que se esté diseñando.
- El **plan de métricas de desempeño** indica qué partes del entorno de prueba se monitorean como parte de la ejecución de la prueba. Se indica por parte qué métricas se recopilan, en qué intervalos y cómo se protegerá y presentará la información una vez finalizada. Los perfiles de carga muestran el grado en que los recursos del sistema (CPU, memoria, red) están cargados en realidad.
- El análisis deberá tener en cuenta el posible impacto del seguimiento del rendimiento y deberán restablecerse los ajustes para la continuación de las pruebas periódicas.

© JMA 2020. All rights reserved

Inicio temprano

- Con la integración de las pruebas de rendimiento en el ciclo de vida de entrega de TI, es posible realizar una serie de mejoras que permitirán un nivel más exhaustivo de pruebas de rendimiento:
  - Haga que el rendimiento sea una parte importante de la definición de hecho. Esto no sólo requiere una atención especial al diseño y desarrollo para el rendimiento; también requiere una infraestructura de pruebas y una configuración de herramientas que proporcionen los beneficios de todas las variedades relevantes de pruebas de rendimiento durante los sprints.
  - Programe pruebas de rendimiento de un extremo a otro de manera oportuna con respecto a la versión de los sistemas de TI para permitir las últimas correcciones.
  - Facilite el ciclo de retroalimentación desde las pruebas de rendimiento de un extremo a otro y el monitoreo del rendimiento de la producción hasta cualquier parte del flujo de información de DevOps.
  - Realice pruebas de aceptación de rendimiento dedicadas (unidad/componente) integradas en un proceso de CI/CD.

© JMA 2020. All rights reserved

Fallo de rendimiento comunes

- **Respuesta lenta independientemente de los niveles de carga**: En algunos casos, la respuesta es inaceptable independientemente de la carga. Esto puede ser causado por problemas de rendimiento subyacentes, incluyendo, pero no limitado a, mal diseño o implementación de la base de datos, latencia de la red y otras cargas en segundo plano. Estos problemas se pueden identificar durante la prueba funcional y de usabilidad, no sólo durante la prueba de rendimiento, por lo que los analistas de pruebas deben estar atentos a ellos e informarlos.
- **Respuesta lenta bajo niveles de carga moderada a elevada**: En algunos casos, la respuesta se degrada de forma inaceptable con cargas de moderadas a elevadas, incluso cuando dichas cargas se encuentran totalmente dentro de los rangos normales, esperados y permitidos. Los defectos subyacentes incluyen la saturación de uno o más recursos y cargas en segundo plano variables.
- **Respuesta degradada con el transcurso del tiempo**: En algunos casos, la respuesta se degrada gradual o drásticamente con el tiempo. Las causas subyacentes incluyen fugas de memoria, fragmentación de disco, aumento de la carga de la red con el tiempo, crecimiento del repositorio de archivos y crecimiento inesperado de la base de datos.
- **Manipulación de errores inadecuada o descuidada bajo cargas elevadas o excesivas**: En algunos casos, el tiempo de respuesta es aceptable, pero el manejo de errores se degrada con niveles de carga elevados y por encima del límite. Los defectos subyacentes incluyen la insuficiencia de recursos, colas y pilas de tamaño insuficiente, y configuraciones de tiempo de espera demasiado rápidas.

© JMA 2020. All rights reserved

Causas de fallo comunes

- Una aplicación basada en la web que proporciona información sobre los servicios de una empresa no responde a las solicitudes de los usuarios en un plazo de siete segundos (una regla general del sector). La eficiencia de desempeño del sistema no se puede lograr bajo condiciones de carga específicas.
- Un sistema presenta un fallo total o es incapaz de responder a las entradas de los usuarios cuando son objeto de un gran número de solicitudes de los usuarios (por ejemplo, la venta de entradas para un gran evento deportivo). La capacidad del sistema para tratar este número de usuarios es insuficiente.
- La respuesta del sistema se degrada significativamente cuando los usuarios envían solicitudes de grandes cantidades de datos (por ejemplo, se publica en un sitio web para ser descargado un informe de gran tamaño e importancia). La capacidad del sistema para tratar los volúmenes de datos generados es insuficiente.
- El procesamiento por lotes no puede completarse antes de que sea necesario el procesamiento en línea. El tiempo de ejecución de los procesos por lotes es insuficiente para el período de tiempo disponible.
- Un sistema en tiempo real se queda sin RAM cuando los procesos paralelos generan grandes demandas de memoria dinámica que no pueden ser liberadas a tiempo. La memoria RAM no está dimensionada adecuadamente o no se ha establecido un orden de prioridad adecuado para las solicitudes de memoria RAM.
- Un componente de un sistema en tiempo real, el componente A, que suministra entradas al componente B del sistema en tiempo real, no puede calcular las actualizaciones con la tasa requerida. El sistema general es incapaz de responder a tiempo y puede fallar. Los módulos de código en el componente A deben ser evaluados y modificados ("perfilado del rendimiento") para asegurar que se puedan alcanzar las tasas de actualización requeridas.

© JMA 2020. All rights reserved

FUNDAMENTOS DE LA MEDICIÓN DEL RENDIMIENTO

© JMA 2020. All rights reserved

Introducción

- Las mediciones precisas y las métricas que se derivan de esas mediciones son esenciales para definir los objetivos de la prueba de rendimiento y para evaluar los resultados de dichas prueba. La prueba de rendimiento no debe llevarse a cabo sin antes comprender qué mediciones y métricas son necesarias.
- Los siguientes riesgos del proyecto se aplican si se ignora este consejo:
  - Se desconoce si los niveles de rendimiento son aceptables para cumplir los objetivos operativos.
  - Los requisitos de rendimiento no están definidos en términos mensurables.
  - Es posible que no sea posible identificar tendencias que puedan predecir niveles más bajos de rendimiento.
  - Los resultados reales de una prueba de rendimiento no pueden evaluarse comparándolos con un conjunto de medidas de rendimiento de referencia que definan un rendimiento aceptable y/o inaceptable.
  - Los resultados de la prueba de rendimiento se evalúan en base a la opinión subjetiva de una o más personas.
  - No se entienden los resultados proporcionados por una herramienta de prueba de rendimiento.

© JMA 2020. All rights reserved

Pruebas estadísticas

- La prueba estadística se puede utilizar para probar el rendimiento del programa y su confiabilidad.
- Las pruebas se diseñan para reflejar la frecuencia de entradas reales de usuario.
- El objetivo de las pruebas estadísticas es obtener medidas sobre las diferentes métricas y no fallos (pruebas de defectos), centrándose en las operaciones y no en la funcionalidad.
- Después de ejecutar las pruebas, se puede hacer una estimación de la confiabilidad operacional del sistema. La prueba falla si los valores obtenidos están fuera de rango o se ha producido una ruptura.
- El rendimiento se puede juzgar midiendo la ejecución de las pruebas estadísticas.

© JMA 2020. All rights reserved

Métricas y Medidas

- Podemos definir una **métrica** como una asignación de un valor a un atributo (tiempo, complejidad, etc.) de una entidad software, ya sea un producto (código, diseño, etc.) o un proceso (pruebas, diseño, etc.).
- Estas métricas podrían medir atributos internos (aquellos que pueden medirse directamente) o externos (los que necesitan ser medidos de forma indirecta).
- Una **medida de producto** es aquella que representa una característica del producto en un instante de tiempo claramente determinado. La medición puede realizarse tanto en la fase de diseño como durante el mantenimiento, pero no contiene sino una descripción instantánea del atributo medido.
- Una **medida de proceso** sirve para evaluar diferentes aspectos del proceso de mantenimiento, permitiendo a sus responsables efectuar revisiones de costes y esfuerzo, detectar anomalías en el proceso de mantenimiento o mejorar actividades.

© JMA 2020. All rights reserved

Métricas

- Como con cualquier forma de medición, es posible obtener y expresar métricas de forma precisa. Por lo tanto, cualquiera de las métricas y mediciones utilizadas pueden y deben definirse para que sean significativas en un contexto particular. Esto se refiere a realizar pruebas iniciales y aprender qué métricas necesitan ser refinadas y cuáles necesitan ser incorporadas.
- Las métricas recogidas en una prueba de rendimiento específica variarán en función de los siguientes factores:
  - contexto de negocio (procesos de negocio, comportamiento de clientes y usuarios, y expectativas de los implicados).
  - contexto operativo (tecnología y cómo se utiliza).
  - objetivos de prueba.
- Una forma común de clasificar las mediciones y métricas de rendimiento es considerar el entorno técnico, el entorno de negocio o el entorno operativo en el que se necesita la evaluación del rendimiento.

© JMA 2020. All rights reserved

Entorno Técnico

Tipo de entorno técnico

- Basado en la Web.
- Móvil.
- Internet de las Cosas (IoT).
- Dispositivos cliente de escritorio.
- Procesamiento en el servidor.
- Mainframe.
- Bases de datos.
- Redes.
- La naturaleza del software que se ejecuta en el entorno (por ejemplo, integrado).

Métricas

- Tiempo de respuesta (por ejemplo, por transacción, por usuario concurrente, tiempos de carga de página).
- Utilización de recursos (por ejemplo, CPU, memoria, ancho de banda de red, latencia de red, espacio disponible en el disco, velocidad de E/S, hilos).
- Velocidad de procesamiento de una transacción clave (es decir, el número de transacciones que se pueden procesar en un período de tiempo determinado).
- Tiempo de procesamiento de lotes (por ejemplo, tiempos de espera, de procesamiento, de respuesta de la base de datos, de compleción).
- Número de errores que impactan al rendimiento.
- Tiempo de compleción (por ejemplo, para crear, leer, actualizar y borrar datos).
- Carga en segundo plano sobre recursos compartidos (especialmente en entornos virtualizados).
- Métricas de software (por ejemplo, complejidad del código).

© JMA 2020. All rights reserved

Entornos de Negocio y Operativo

Negocio

- Eficiencia de los procesos de negocio (por ejemplo, la velocidad con la que se realiza un proceso de negocio global que incluye flujos de casos de uso normal, alternativos y excepcionales).
- Procesamiento de datos, transacciones y otras unidades de trabajo realizadas (por ejemplo, pedidos procesados por hora, filas de datos añadidas por minuto).
- Índices de cumplimiento o incumplimiento del Acuerdo de Nivel de Servicio (SLA por sus siglas en inglés) (por ejemplo, incumplimiento del ANS por unidad de tiempo).
- Alcance del uso (por ejemplo, porcentaje de usuarios globales o nacionales que realizan tareas en un momento dado).
- Concurrencia de uso (por ejemplo, el número de usuarios que realizan una tarea de forma simultánea).
- Cronología de uso (por ejemplo, el número de pedidos procesados durante las horas de carga máxima).

Operativo

- Procesos operativos (por ejemplo, el tiempo necesario para la puesta en marcha del entorno, copias de seguridad, tiempos de apagado y reanudación).
- Restauración de sistema (por ejemplo, el tiempo necesario para restaurar los datos de una copia de seguridad).
- Alertas y avisos (por ejemplo, el tiempo necesario para que el sistema emita una alerta o aviso).

© JMA 2020. All rights reserved

Componentes a supervisar

- Las áreas siguientes afectan al rendimiento:
  - Recursos del sistema (hardware)
  - Sistema operativo
  - Arquitectura de red
  - Configuración de servidores e infraestructuras
  - Aplicaciones de bases de datos
  - Aplicaciones cliente
- Dentro de los Recursos del sistema, los componentes principales en los que se debe concentrar inicialmente son:
  - Actividad del disco
  - Uso de la memoria
  - Uso del procesador
  - Uso de la red

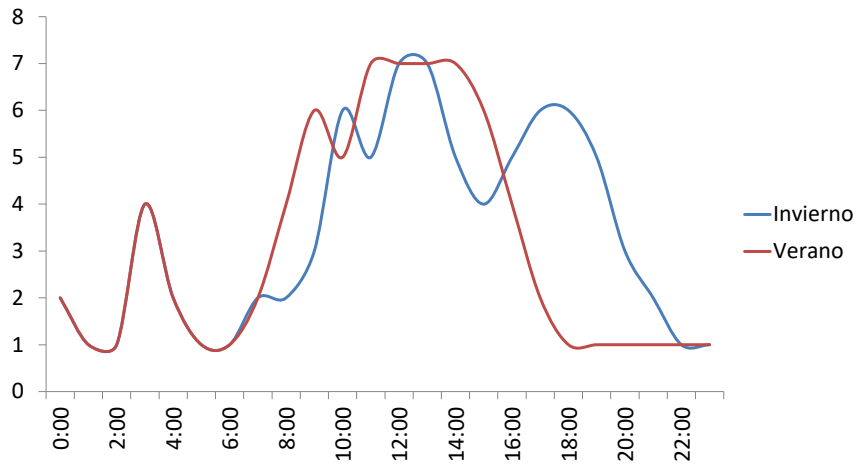
© JMA 2020. All rights reserved

Toma de medidas

- Se denomina toma de medidas al proceso de captura de valores cuantitativos sobre los elementos representativos en diferentes momentos. Teniendo en cuenta las diferentes idiosincrasias de los sistemas y de los recursos humanos, es necesario tomar medidas:
  - A diferentes horas del día.
  - Los diferentes días de la semana.
  - A lo largo de todo el mes
  - En diferentes meses
- La toma de medidas es un proceso continuo acumulativo que deben representar las cargas de trabajo habituales que se ejecutan en las bases de datos o sistemas.
- La calidad de las medidas tomadas determina la calidad del análisis.

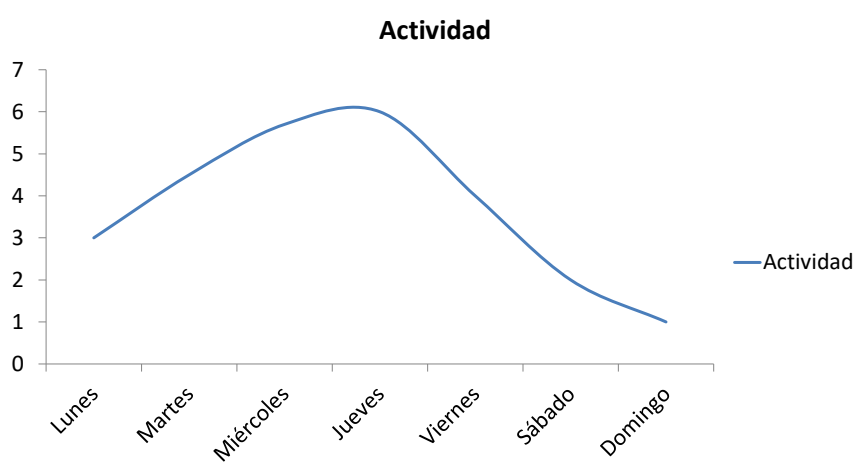
© JMA 2020. All rights reserved

Gráficas diarias



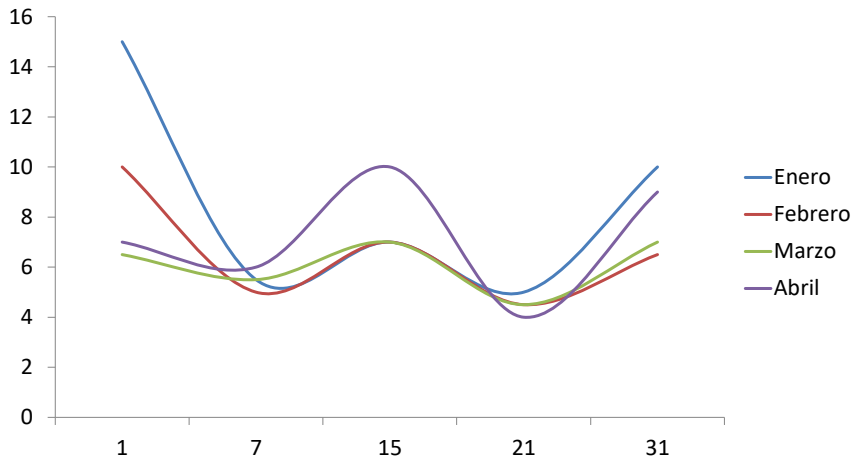
© JMA 2020. All rights reserved

Gráfica semanal



© JMA 2020. All rights reserved

Graficas mensuales



© JMA 2020. All rights reserved

Cuellos de botella

- Se denomina cuello de botella al fenómeno en donde el rendimiento o capacidad de un sistema completo es severamente limitado por componentes individuales.
- Entre las causas de estos cuellos de botella se incluyen:
  - Recursos que funcionan incorrectamente.
  - Recursos mal configurados.
  - Recursos del mismo tipo que no distribuyen de forma equilibrada las cargas de trabajo; por ejemplo, cuando un recurso monopoliza un disco.
  - Recursos insuficientes que requieren componentes adicionales o actualizados.
- Para detectar los cuellos de botella es necesario realizar un proceso de toma de medidas y su análisis ulterior.

© JMA 2020. All rights reserved

Fuentes clave de las métricas de rendimiento

- Para evitar la paradoja del observador, que hace referencia a una situación en que el fenómeno observado se ve influido por la presencia del propio observador tergiversando el resultado, el rendimiento del sistema solo debería verse mínimamente afectado por el esfuerzo de recopilación de métricas (conocido como el "efecto sonda"). Además, el volumen, la precisión y la velocidad con la que deben recopilarse las métricas de rendimiento hacen que el uso de diferentes tipos herramientas sea un requisito imprescindible. Aunque el uso combinado de herramientas es frecuente, el uso de herramientas de prueba puede introducir redundancia y otros problemas.
- Hay tres tipos de herramientas para obtener las medidas de métricas de rendimiento:
  - Herramientas de Prueba de Rendimiento: Todas las herramientas de pruebas de rendimiento proporcionan mediciones y métricas como resultado de una prueba. Estas herramientas pueden influir en el observado, por lo que deben ejecutar remotamente y restringe las medidas obtenidas a su extremo (E2E).
  - Herramientas de Monitorización del Rendimiento: A menudo, los sistemas cuentan con mecanismos de emisión de medidas para su monitorización y supervisión. Se pueden utilizar dichas herramientas para monitorizar el rendimiento del sistema de forma continua o durante el proceso de prueba.
  - Herramientas de Análisis de Registros: Así mismo, los sistemas cuentan con mecanismos que registran los sucesos que ocurren en sistema. Existen herramientas que analizan los registros del servidor y elaboran métricas a partir de ellos, algunas pueden crear gráficos para proporcionar una vista gráfica de los datos.
- Las dos últimas, si ya estaban activadas, no requieren recursos adicionales por lo que no influyen.

© JMA 2020. All rights reserved

Micrometer, Prometheus, Grafana

- La combinación de Micrometer, Prometheus y Grafana es un stack de monitoreo moderno y de código abierto muy popular en entornos de microservicios y cloud-native.
- Es un stack muy flexible, escalable, ideal para monitoreo dinámico y métricas en tiempo real.
- Micrometer permite exportar a cualquiera de los más populares sistemas de monitorización los datos de las métricas. Usando Micrometer la aplicación se abstrae del sistema de métricas empleado pudiendo cambiar en un futuro si se desea.
- Uno de los sistemas más populares de monitorización es Prometheus, que se encarga de recoger y almacenar los datos de las métricas expuestas por las aplicaciones y ofrece un lenguaje de consulta de los datos con el que otras aplicaciones pueden visualizarlos en gráficas y paneles de control.
- Grafana es una herramienta especializada en crear cuadros de mando y gráficos a partir de múltiples fuentes, lo que permite visualizar los datos proporcionados por Prometheus.
- Estos sistemas de monitorización ofrecen un sistema de alertas que se integran entre otros con Slack.

© JMA 2020. All rights reserved

Otras herramientas de monitorización

- Zabbix:
  - Función: Un sistema de monitoreo de código abierto muy popular y completo que puede monitorizar casi cualquier cosa (servidores, redes, aplicaciones, bases de datos). Utiliza agentes para recopilar datos y ofrece una potente interfaz web para visualización, alertas y gestión.
  - Ventajas: Escalable, flexible, con plantillas predefinidas para muchos sistemas y aplicaciones, y un sistema de alertas muy configurable.
- Nagios:
  - Función: Otro sistema de monitoreo de código abierto veterano. Se centra en la monitorización del estado (UP/DOWN) y la disponibilidad de servicios y hosts, aunque también puede monitorizar métricas de rendimiento.
  - Ventajas: Muy estable, gran comunidad, muchas plugins disponibles.
- Datadog, New Relic, Dynatrace (Soluciones Comerciales):
  - Función: Plataformas de monitoreo de rendimiento de aplicaciones (APM) y de infraestructura basadas en la nube. Ofrecen monitoreo integral, análisis de causa raíz, tracing distribuido y funcionalidades avanzadas.
  - Ventajas: Menos configuración manual, soporte completo, dashboards preconstruidos y análisis automatizado. Son ideales para entornos complejos y empresas que buscan una solución todo en uno.

© JMA 2020. All rights reserved

ELK: Elasticsearch, Logstash y Kibana

- ELK es un stack de código abierto que permite la ingesta de grandes volúmenes de textos (logs o registros de servidor) para su análisis y presentación (en cuadros y gráficas), muy popular en entornos de microservicios y cloud-native.
- ELK son las siglas para tres proyectos open source: Elasticsearch, Logstash y Kibana.
- Logstash es un ETL obtiene, transforma y envía de forma dinámica los datos independientemente de su formato o complejidad. Admite una variedad de entradas de una manera de transmisión continua que extraen eventos de una multitud de fuentes comunes, todo al mismo tiempo: logs, métricas, aplicaciones web, almacenes de datos, varios servicios de AWS, ... los filtros transforman cada evento, para que converjan en un formato común para el análisis y un valor comercial más poderosos. Los resultados se pueden enviar a Elasticsearch para búsquedas y análisis o a una variedad de salidas.
- Elasticsearch es un motor de búsqueda y analítica RESTful distribuido capaz de abordar multitud de casos de uso. Como núcleo del Elastic Stack, almacena e indexa de forma centralizada los datos para una búsqueda (estructuradas, no estructuradas, geográficas, métricas, …) a gran velocidad, con relevancia refinada y analíticas que escalan con facilidad.
- Kibana es una interfaz de usuario gratuita y abierta que permite visualizar los datos de Elasticsearch y navegar en el Elastic Stack, desde rastrear la carga de búsqueda hasta comprender la forma en que las solicitudes fluyen por las apps. Como una imagen vale más que mil líneas de log, Kibana envía los datos en forma de histogramas, grafos de líneas, gráficos circulares, proyecciones solares y más, para análisis de logs, monitoreo de infraestructura, APM (Application Performance Monitoring), operaciones de seguridad, analítica de negocios, …

© JMA 2020. All rights reserved

Evaluar la prueba: Resultados esperados

- En la prueba funcional, los resultados esperados generalmente se pueden definir claramente y los resultados de la prueba se pueden interpretar para determinar si pasó o falló la prueba. Las pruebas funcionales a menudo se benefician de oráculos de prueba bien definidos, mientras que las pruebas de rendimiento, a menudo, carecen de esta fuente de información.
- No sólo los implicados son notoriamente poco hábiles a la hora de articular los requisitos de rendimiento, sino que los encargados de especificar tampoco lo son a la hora de obtener tales requisitos. Los probadores, a menudo, reciben una orientación limitada, ambigua o inexistente para definir los resultados esperados de la prueba.
- Cuando se evalúan los resultados de la prueba de rendimiento, es importante observar los resultados de cerca. Los resultados iniciales en bruto pueden ser engañosos, ya que los fallos de rendimiento se pueden ocultar bajo unos resultados generales aparentemente buenos. Por ejemplo, la utilización de los recursos puede estar muy por debajo del 75% en el caso de todos los recursos clave con potencial de cuellos de botella, pero la capacidad de procesamiento o el tiempo de respuesta de transacciones clave son demasiado lentos.
- Los resultados específicos a evaluar varían dependiendo de las pruebas que se realizan.

© JMA 2020. All rights reserved

Oráculos de pruebas de rendimiento

- En la antigüedad, un oráculo era la respuesta que una deidad, a través de un intermediario y en un lugar sagrado, daba a una pregunta personal concerniente generalmente al futuro, usados como método de adivinación.
- En el contexto de las pruebas de software, un oráculo de pruebas es un mecanismo o fuente que determina si el resultado de una prueba es correcto o incorrecto y supera o falla la prueba. Para las pruebas de rendimiento, los oráculos son cruciales para establecer las expectativas y comparar los resultados medidos con lo que se considera un rendimiento aceptable.
- No se trata de un "oráculo" en el sentido de una herramienta específica o un componente de software, sino más bien de un conjunto de criterios, valores previstos o reglas de negocio que permiten juzgar si el sistema cumple con sus objetivos de rendimiento.
- La mayoría de los valores de los oráculos quedan obsoletos y deben revisarse periódicamente, lo que ayer era aceptable hoy ya no lo es.

© JMA 2020. All rights reserved

Oráculos basados en: Requisitos de Rendimiento (NFRs, SLAs)

- Esta es la fuente más fiable, fundamental y formal de oráculos para pruebas de rendimiento. Los requisitos de rendimiento (a menudo parte de las Especificaciones de Requisitos No Funcionales - NFRs, o Acuerdos de Nivel de Servicio - SLAs) establecen umbrales y expectativas claras para el sistema.
- Ejemplos de oráculos basados en requisitos:
  - Tiempos de Respuesta (Response Times):
    - "El 95% de las solicitudes de inicio de sesión deben responder en menos de 2 segundos."
    - "El tiempo promedio para la carga de una página de producto debe ser inferior a 1.5 segundos."
    - "La búsqueda de clientes debe completarse en menos de 500 milisegundos para 100 usuarios concurrentes."
  - Rendimiento (Throughput):
    - "El sistema debe procesar al menos 100 transacciones por segundo."
    - "La API de pedidos debe soportar 500 solicitudes por minuto."
  - Capacidad de Usuarios Concurrentes:
    - "La aplicación debe soportar 10,000 usuarios concurrentes sin degradación del rendimiento."
    - "El punto de quiebre del sistema debe ser al menos 15,000 usuarios concurrentes."
  - Utilización de Recursos:
    - "El uso de CPU en los servidores de aplicación no debe exceder el 80% bajo carga máxima esperada."
    - "El uso de memoria del servidor de base de datos no debe superar el 90%."
    - "La latencia de E/S del disco no debe exceder los 10 ms."
  - Tasas de Error:
    - "La tasa de errores HTTP 5xx no debe superar el 0.1% bajo ninguna carga."
    - "No deben producirse errores de base de datos durante las pruebas de carga."

© JMA 2020. All rights reserved

Oráculos basados en: Especificaciones técnicas de dispositivos

- Los suministradores de dispositivos, en las especificaciones técnicas, definen los valores de los parámetros de capacidad y márgenes operacionales del dispositivo, que permiten evaluar el rendimiento de los mismos, como:
  - Tiempo de respuesta (Response Time): Tiempo que tarda el sistema en responder a una solicitud. Unidad: Milisegundos (ms). Ejemplo: "Tiempo de carga de una página web: 320 ms"
  - Rendimiento o rendimiento sostenido (Throughput): Cantidad de trabajo que realiza un sistema en un periodo de tiempo. Unidad: Solicitudes por segundo (req/s), transacciones por segundo (TPS), MB/s, operaciones por segundo (IOPS). Ejemplo: "Procesa 5,000 solicitudes HTTP/s"
  - Latencia: Tiempo entre una solicitud y la primera respuesta (clave en redes y APIs). Unidad: ms. Ejemplo: "Latencia promedio: 45 ms"
  - IOPS (Input/Output Operations Per Second): Cantidad de operaciones de lectura/escritura que puede hacer un disco. Unidad: Operaciones/s. Ejemplo: "SSD NVMe con 500,000 IOPS"
  - Tiempo de arranque / ejecución: Tiempo que tarda el sistema o una aplicación en estar lista. Unidad: Segundos. Ejemplo: "Sistema operativo arranca en 8.5 s"
  - FPS (Frames Per Second): Rendimiento gráfico en visualización (gaming o UI pesada). Unidad: fps. Ejemplo: "Juego corre a 120 fps estables en 1080p Ultra"
  - Consumo energético: Cuánta energía consume el sistema bajo carga. Unidad: Watts (W). Ejemplo: "Consumo promedio durante la prueba: 65W"

© JMA 2020. All rights reserved

Oráculos basados en: Estándares de la Industria y Mejores Prácticas

- Ciertas industrias o tipos de aplicaciones tienen expectativas de rendimiento implícitas o explícitas para juzgar el rendimiento.
- Ejemplos:
  - Aplicaciones Web: Se espera que las páginas se carguen en pocos segundos (a menudo menos de 3 segundos es un objetivo común).
    - Core Web Vitals de Google: Son métricas estandarizadas para medir la experiencia del usuario (usadas por su algoritmo de posicionamiento).
  - APIs REST: Tiempos de respuesta en el rango de milisegundos.

© JMA 2020. All rights reserved

Oráculos basados en: Comportamiento Anterior (Históricos)

- Si el sistema ya existe (una versión anterior, un sistema de referencia o un competidor), su comportamiento actual o histórico puede servir como oráculo. También conocidos como Líneas Base (Baselines) o Comportamiento del Sistema Preexistente. Así mismo, pueden estar disponible benchmark de sistemas similares propios o ajenos.
- Los benchmark (comparativas de rendimiento) son una técnica utilizada para medir el rendimiento de un sistema o uno de sus componentes y poder compararlos con sistemas similares. Más formalmente, puede entenderse como una prueba de rendimiento de uno o varios programas predefinidos que hacen un uso intensivo de recursos en diferentes maquinas. El objetivo es obtener un conjunto de medidas que poder comparar con los resultados obtenidos en máquinas similares con las mismas condiciones.
- Ejemplos:
  - Rendimiento Histórico: Si una versión anterior de la aplicación tenía un tiempo de respuesta de X segundos para una función específica, salvo justificación, la nueva versión no debería ser significativamente más lenta.
  - Rendimiento de la Competencia: Si un competidor directo carga una página en 1 segundo, tu aplicación no debería tardar 5 segundos.
  - Líneas Base de Pruebas de Desarrollo: Durante las primeras etapas de desarrollo, se pueden establecer "líneas base" de rendimiento en un entorno controlado y con una carga mínima. Estas líneas base se utilizan para comparar el rendimiento a medida que el sistema evoluciona o se somete a cargas mayores.

© JMA 2020. All rights reserved

Oráculos basados en: Experiencia (Observación y sentido común)

- Los arquitectos, desarrolladores senior, administradores de sistemas y expertos en el dominio a menudo tienen un conocimiento implícito sobre cómo debería comportarse el sistema.
- Ejemplos:
  - Expectativas Intuitivas: Un experto podría decir: "Es lógico que esta transacción compleja tarde un poco más, pero si excede los 5 segundos, algo anda mal."
  - Comportamiento Anómalo: Detectar picos inusuales en el uso de CPU o memoria que no se correlacionan con la actividad de carga.
  - Patrones de Transacción: El experto sabe qué transacciones deberían ser rápidas y cuáles naturalmente tomarán más tiempo.

© JMA 2020. All rights reserved

Oráculos basados en: Perfil de Uso del Usuario / Modelado de Carga

- Comprender cómo los usuarios interactúan con la aplicación es un oráculo vital. Este perfil se traduce en el "modelado de carga" para las pruebas.
- Ejemplos:
  - Distribución de Carga: Si el 60% de los usuarios realizan búsquedas, el 30% añaden al carrito y el 10% realizan compras, las pruebas de rendimiento deben simular esta proporción.
  - "Think Time" (Tiempo de Reflexión): Los usuarios reales no hacen clic instantáneamente; hay un tiempo de espera entre acciones. Incluir este tiempo en los scripts de prueba es un oráculo de comportamiento realista.
  - Volumen de Datos: Si se espera que los usuarios carguen archivos de 10 MB, las pruebas deben usar archivos de ese tamaño para evaluar el impacto en la red y el servidor.

© JMA 2020. All rights reserved

Problema del oráculo

- A diferencia de las pruebas funcionales, donde el oráculo puede ser un resultado esperado muy específico, en las pruebas de rendimiento, el "problema del oráculo" es más flexible pero también más complejo. No hay una "respuesta correcta" única, sino relativa, expresada como un valor máximo o mínimo o un rango de valores aceptables. Los oráculos requieren:
  - Identificación temprana: Los oráculos de rendimiento deben definirse lo más pronto posible en el ciclo de desarrollo, idealmente como parte de los requisitos no funcionales.
  - Claridad y Mensurable: Los oráculos deben ser claros, cuantificables y medibles. Evita frases vagas como "la aplicación debe ser rápida", "la aplicación debe consumir pocos recursos", …
  - Colaboración: La definición de oráculos requiere la colaboración entre las partes interesadas (negocio, desarrollo, operaciones) para establecer expectativas realistas y alcanzables.
  - Iteración: Los oráculos pueden necesitar ajustarse a medida que se aprende más sobre el comportamiento del sistema y se descubren limitaciones.

© JMA 2020. All rights reserved

Métricas Clave

- Tiempos de Respuesta (Response Times): El tiempo que tarda el sistema en responder a una solicitud.
  - Promedio: Tiempo medio de respuesta.
  - Percentiles (90th, 95th, 99th): Indican que un cierto porcentaje de las respuestas estuvieron por debajo de ese umbral (ej. el 95% de las respuestas fueron más rápidas que X segundos).
- Throughput (Rendimiento): Número de transacciones o solicitudes procesadas por unidad de tiempo (ej. transacciones por segundo, peticiones por minuto).
- Usuarios Concurrentes (Concurrent Users): El número de usuarios activos que utilizan el sistema al mismo tiempo.
- Errores por Segundo (Errors per Second): La tasa de errores generados por el sistema bajo carga.
- Utilización de Recursos:
  - CPU: Porcentaje de uso del procesador.
  - Memoria: Uso de RAM.
  - Disco: Lectura/escritura en disco.
  - Red: Tráfico de red.
- Latencia (Latency): El tiempo de retardo entre el envío de una solicitud y la recepción del primer byte de la respuesta.
- Conexiones de Base de Datos: Número de conexiones abiertas a la base de datos.
- Pools de Hilos: Utilización y saturación de los pools de hilos en servidores de aplicaciones.

© JMA 2020. All rights reserved

Agregación de métricas

- El propósito de la agregación de las métricas de rendimiento es agrupar y resumir las medidas para lograr comprenderlas y expresarlas de una manera que transmita con precisión la imagen completa del rendimiento del sistema.
- Cuando las métricas de rendimiento se consideran sólo en el nivel detallado, puede ser difícil llegar a la conclusión correcta, especialmente para los implicados del negocio.
- Para muchos implicados, la principal preocupación es general, como que el tiempo de respuesta de un sistema, sitio web u otro objeto de prueba esté dentro de unos límites aceptables.
- Una vez que se ha logrado una comprensión más profunda de las métricas de rendimiento, las métricas se pueden agregar para que:
  - Los implicados del negocio y del proyecto pueden ver el estado general del rendimiento del sistema.
  - Se puedan identificar las tendencias de rendimiento.
  - Las métricas de rendimiento pueden ser informadas de forma comprensible.

© JMA 2020. All rights reserved

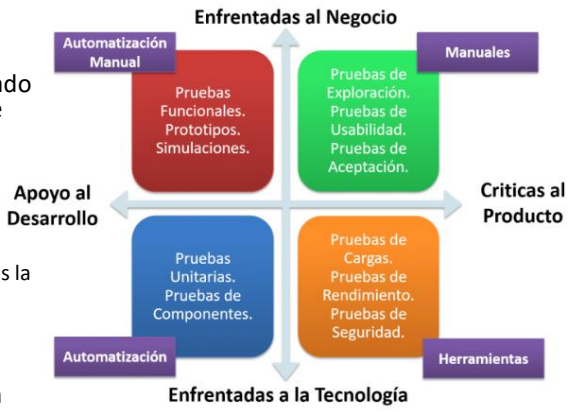
Cantidad correcta de pruebas

- Hay un feroz debate que gira en torno a la cantidad correcta de pruebas. Muy pocas pruebas son un problema: las funciones no se especifican correctamente, los errores pasan desapercibidos, ocurren regresiones. Pero demasiadas pruebas consumen tiempo de desarrollo y recursos, no generan ganancias adicionales y ralentizan el desarrollo a largo plazo.
- No es cuestión de hacer muchas pruebas, de hecho, hay que hacer las imprescindibles, pero seleccionando buenas pruebas: las que mayor probabilidad tengan de detectar un error y cubran el mayor numero de escenarios.
- Las pruebas difieren en su valor y calidad. Algunas pruebas son más significativas que otras. Los recursos son limitados.
- Esto significa que la calidad de las pruebas es más importante que su cantidad.

© JMA 2020. All rights reserved

Cuadrante de Pruebas

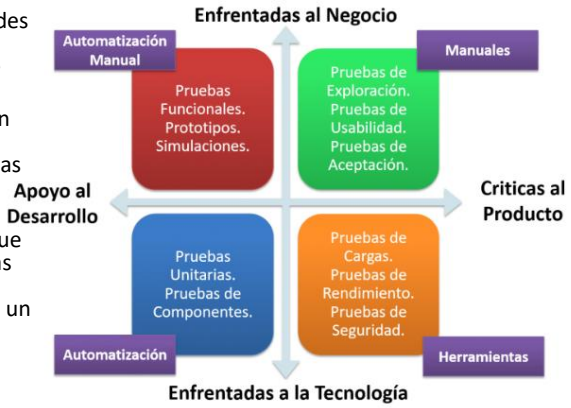
- El cuadrante de prueba divide todo el proceso de prueba en 4 partes. Esto hace que el proceso de prueba sea fácil de entender.
- El cuadrante de pruebas fue originalmente publicado por Brian Marick en 2003 como parte de una serie de artículos sobre Testeo Ágil y originalmente fue conocido como “Matriz de pruebas de Marick” (Marick Test Matrix”).
- Hay cuatro cuadrantes de prueba que son el resultado de dos ejes.
  - El eje vertical muestra la perspectiva tecnológica versus la perspectiva empresarial.
  - El eje horizontal trata sobre la orientación durante la creación de un producto versus la evaluación del producto una vez que está listo.
- Hay que asegurarse de que se organicen y realicen pruebas para los cuatro cuadrantes.



© JMA 2020. All rights reserved

Cuadrante de Pruebas

- El primer cuadrante (**abajo a la izquierda**) es donde la atención se centra en la tecnología y donde las actividades de prueba guían al equipo en la creación del producto. Suele implicar TDD para las unidades y la integración de las unidades. Las pruebas deberían automatizarse.
- El segundo cuadrante (**arriba a la izquierda**) se centra en el negocio y también se centra en guiar al equipo. Esto implica, por ejemplo, el desarrollo impulsado por pruebas de aceptación (ATDD), donde se prueba un proceso de negocio, preferiblemente automatizadas.
- El tercer cuadrante (**arriba a la derecha**) tiene un enfoque empresarial en la evaluación del producto. Estas pruebas se ejecutan principalmente de forma manual, ya sea siguiendo escenarios de prueba preparados o mediante un enfoque exploratorio.
- El cuarto cuadrante (**abajo a la derecha**) se centra en la tecnología y evalúa los aspectos no funcionales de un producto que sólo pueden validarse una vez que el producto esté listo. Las pruebas de rendimiento dinámico son un buen ejemplo de ello.



© JMA 2020. All rights reserved

Caso practico: Windows

SUPERVISIÓN DEL SISTEMA OPERATIVO

© JMA 2020. All rights reserved

Administrador de Tareas (Task Manager)

- Acceso: Ctrl + Shift + Esc, o Ctrl + Alt + Supr y selecciona "Administrador de tareas", o clic derecho en la barra de tareas y selecciona "Administrador de tareas".
- Pestañas clave:
  - Procesos: Muestra una lista de todos los procesos en ejecución, aplicaciones, procesos en segundo plano y procesos de Windows. Puedes ver el uso de CPU, Memoria, Disco y Red por cada proceso. Si un proceso consume muchos recursos, lo verás aquí. Puedes ordenar por cada columna haciendo clic en el encabezado.
  - Rendimiento: Ofrece una vista gráfica en tiempo real del uso general de CPU, Memoria, Disco, Ethernet/Wi-Fi (Red) y GPU. Es excelente para ver tendencias y detectar picos. Al hacer clic en cada gráfico, obtendrás más detalles.
  - Historial de aplicaciones: Muestra el uso de recursos de las aplicaciones a lo largo del tiempo.
  - Usuarios: Muestra el uso de recursos por usuario.
  - Detalles: Una vista más granular de los procesos, similar a la pestaña "Procesos" pero con más columnas de información (PID, estado, etc.).
  - Servicios: Muestra los servicios de Windows en ejecución.
- Uso: Ideal para una verificación rápida de los consumos de recursos en un momento dado.

© JMA 2020. All rights reserved

Monitor de Recursos (Resource Monitor - resmon.exe)

- Acceso: Desde el Administrador de Tareas (pestaña "Rendimiento", haz clic en "…" y "Monitor de recursos" en la parte superior izquierda), o busca "Monitor de recursos" en el menú Inicio.
- Función: Ofrece una vista mucho más detallada que el Administrador de Tareas. Proporciona gráficos y tablas en tiempo real para CPU, Disco, Red y Memoria, desglosando el uso por procesos, servicios, actividad, …
- Pestañas clave:
  - Overview (Información general): Resumen de todos los recursos.
  - CPU: Muestra el uso de CPU por proceso y los hilos (threads) activos.
  - Disk: Muestra las operaciones de lectura/escritura en disco por proceso y los archivos que están siendo accedidos. Muy útil para diagnosticar cuellos de botella de E/S.
  - Network: Detalla la actividad de red por proceso, conexiones TCP y puertos de escucha.
  - Memory: Proporciona un desglose del uso de memoria (Memoria en uso, Memoria disponible, En caché, Libre, Paginada, No paginada).
- Uso: Esencial para una investigación más profunda de problemas de rendimiento específicos, como qué proceso está saturando el disco o la red.

© JMA 2020. All rights reserved

Monitor de Rendimiento (Performance Monitor - perfmon.msc)

- Acceso: Busca "Monitor de rendimiento" en el menú Inicio, o ejecuta perfmon.msc.
- Función: La herramienta de monitoreo más potente y flexible de Windows. Permite recopilar y ver datos de rendimiento en tiempo real o registrar datos para análisis posterior. Utiliza "contadores de rendimiento" para rastrear virtualmente cualquier aspecto del sistema operativo y las aplicaciones.
- Características:
  - Contadores: Puedes seleccionar entre cientos de contadores para CPU, memoria, disco, red, procesos, servicios, .NET, IIS, SQL Server, etc. Permite un monitoreo muy granular.
  - Registros de datos: Puedes configurar conjuntos de recopiladores de datos para registrar métricas a lo largo del tiempo en un archivo, lo que es invaluable para analizar problemas intermitentes o tendencias de rendimiento.
  - Alertas: Puedes configurar alertas para que se disparen cuando un contador de rendimiento supere un umbral.
  - Vistas: Muestra los datos en gráficos, histogramas o informes.
- Uso: Para análisis profundo de rendimiento, creación de líneas base, solución de problemas complejos y monitoreo a largo plazo. Requiere un poco más de conocimiento para configurar los contadores adecuados.

© JMA 2020. All rights reserved

netstat (Command Prompt / PowerShell)

- Acceso: Abre el Símbolo del sistema o PowerShell como administrador.
- Función: Muestra las conexiones de red activas, puertos de escucha, estadísticas de protocolo y tablas de enrutamiento.
- Comandos útiles:
  - netstat -ano: Muestra todas las conexiones y puertos de escucha, incluyendo el ID del proceso (PID) asociado a cada conexión.
  - netstat -bn: Muestra las conexiones y los ejecutables que las están utilizando.
  - netstat -s: Muestra estadísticas detalladas por protocolo (TCP, UDP, IP).
- Uso: Para diagnosticar problemas de red, identificar procesos que están usando la red y verificar puertos abiertos.

© JMA 2020. All rights reserved

wmic (Windows Management Instrumentation Command-line)

- Obsoleto en versiones recientes
- Acceso: Símbolo del sistema o PowerShell.
- Función: Permite acceder a la información del sistema a través de WMI. Útil para obtener información de hardware y rendimiento de manera programática.
- Ejemplos:
  - wmic cpu get loadpercentage: Uso actual de CPU.
  - wmic OS get FreePhysicalMemory,TotalPhysicalMemory: Memoria RAM libre y total.
  - wmic logicaldisk get Caption,Freespace,Size: Espacio libre en disco.
- Uso: Scripting y automatización para recopilar métricas.

© JMA 2020. All rights reserved

Contadores de rendimiento clave

- En general los contadores clave son los % de uso o consumo y las longitudes de la cola. La longitud de la cola mide el número de procesos que están esperando para utilizar el recurso, si la media o durante periodos continuos significativos, la longitud es mayor que 2, se producen cuellos de botella al estar permanentemente procesos esperando a ser atendidos.
- Contadores de Memoria: Si "Memoria en uso" se acerca al total de tu RAM, y el "Uso de memoria paginada" es alto, tu sistema está usando mucho el archivo de paginación (swap), lo que ralentiza todo.
  - Memory\Available Bytes: Cantidad de memoria física disponible. Un valor muy bajo es crítico.
  - Memory\Committed Bytes: Cantidad de memoria virtual que se ha reservado para el uso de procesos.
  - Memory\Page Faults/sec: Número de errores de página que se producen por segundo. No todos son graves, pero un aumento súbito puede indicar problemas.
  - Memory\Pages Input/sec / Memory\Pages Output/sec: Velocidad a la que las páginas de memoria son leídas/escritas desde/hacia el disco. Valores altos indican paginación excesiva.
  - Paging File\% Usage: Porcentaje de uso del archivo de paginación. Un uso sostenido y alto indica que la RAM es insuficiente.

© JMA 2020. All rights reserved

Contadores de rendimiento clave

- Contadores de CPU: Si el uso de CPU está consistentemente cerca del 100%, tu sistema está sobrecargado. Investiga qué procesos están consumiendo la mayor parte.
  - Procesador
    - Processor\% Processor Time: Porcentaje total de tiempo que el procesador no está inactivo.
    - Processor\% User Time: Porcentaje de tiempo que el procesador gasta en código de usuario.
    - Processor\% Privileged Time: Porcentaje de tiempo que el procesador gasta en modo kernel.
    - Processor\% DPC Time / Processor\% Interrupt Time: Tiempo que la CPU gasta manejando llamadas a procedimientos diferidos (DPC) e interrupciones. Valores altos pueden indicar problemas de driver.
    - Processor Queue Length: Número de hilos esperando tiempo de procesador. Si es consistentemente mayor que el número de núcleos de CPU, es un cuello de botella de CPU.
  - Procesos/Aplicaciones
    - Process\% Processor Time: Porcentaje de tiempo de CPU utilizado por un proceso específico.
    - Process\Private Bytes: Memoria asignada exclusivamente a un proceso.
    - Process\Working Set: Memoria física que un proceso tiene en uso actualmente.
    - Process\Handle Count: Número de "handles" (punteros a recursos del sistema como archivos, claves de registro, hilos, etc.) abiertos por un proceso. Un aumento constante puede indicar una fuga.
    - Process\Thread Count: Número de hilos en un proceso.

© JMA 2020. All rights reserved

Contadores de rendimiento clave

- Contadores de Disco (E/S): Un "Tiempo de actividad promedio" o "% de uso" de disco consistentemente alto (cercano al 100%) en el Monitor de Recursos indica un cuello de botella de E/S. Verifica qué procesos están realizando la mayor cantidad de lecturas/escrituras.
  - LogicalDisk\% Disk Time: Porcentaje de tiempo que el disco está ocupado manejando solicitudes de lectura o escritura.
  - LogicalDisk\Current Disk Queue Length: Número de solicitudes pendientes en la cola del disco. Si es consistentemente mayor que el número de spindles del disco (1 para un HDD simple, más para RAID/SSD), el disco es un cuello de botella.
  - LogicalDisk\Avg. Disk sec/Read / Avg. Disk sec/Write: Tiempo promedio (en segundos) para que una operación de lectura/escritura se complete.
  - LogicalDisk\Disk Reads/sec / Disk Writes/sec: Número de operaciones de lectura/escritura por segundo.
  - LogicalDisk\Disk Read Bytes/sec / Disk Write Bytes/sec: Cantidad de datos leídos/escritos por segundo.
- Contadores de Red: Una "Utilización" de red cercana al 100% o una baja "Velocidad de vínculo" pueden indicar que tu conexión de red es un cuello de botella. El Monitor de Recursos te mostrará qué procesos están enviando y recibiendo más datos.
  - Network Interface\Bytes Total/sec: Tasa total de bytes enviados y recibidos.
  - Network Interface\Current Bandwidth: Ancho de banda actual de la interfaz de red.
  - Network Interface\% Bandwidth Utilization: Porcentaje del ancho de banda de la interfaz que se está utilizando.
  - Network Interface\Packets Received Discarded / Packets Outbound Discarded: Paquetes descartados debido a errores o falta de capacidad.
  - TCPv4\Connections Established: Número de conexiones TCP establecidas.

© JMA 2020. All rights reserved

Pasos para la supervisión

1. Definir los objetivos concretos de supervisión.
2. Seleccionar la herramienta apropiada.
3. Identificar los componentes que se desea supervisar.
4. Seleccionar métricas para dichos componentes.
5. Tomar medidas.
6. Analizar las medidas obtenidas.
7. Solucionar los problemas detectados o implementar las posibles mejoras.

© JMA 2020. All rights reserved

Caso practico: Linux

SUPERVISIÓN DEL SISTEMA OPERATIVO

© JMA 2020. All rights reserved

Comandos Básicos de Terminal (Vista en Tiempo Real)

- top:
  - Función: Muestra un resumen dinámico y en tiempo real de los procesos en ejecución, el uso de CPU, memoria, swap y la carga promedio del sistema.
  - Uso: Simplemente escribe top en la terminal.
  - Interpretación clave:
    - uptime: Tiempo que el sistema lleva encendido y carga promedio (load average) en los últimos 1, 5 y 15 minutos (cuántos procesos están esperando ser ejecutados).
    - Tasks: Número total de procesos, y cuántos están en ejecución, durmiendo, detenidos o zombies.
    - %Cpu(s): Desglose del uso de CPU:
      - us: CPU utilizada por procesos de usuario.
      - sy: CPU utilizada por el kernel.
      - ni: CPU utilizada por procesos con prioridad nice modificada.
      - id: CPU inactiva (idle).
      - wa: Tiempo de espera de E/S (I/O wait): CPU esperando operaciones de disco/red.
      - hi, si: Interrupciones de hardware y software.
      - st: Tiempo "robado" en máquinas virtuales.
    - Mem / Swap: Uso de memoria RAM y SWAP (total, libre, usada, caché).
  - Lista de procesos: PID, usuario, prioridad (PR), nice (NI), uso de CPU (%CPU), uso de memoria (%MEM), tiempo de ejecución (TIME+), comando (COMMAND).
  - Interactividad: Dentro de top, puedes presionar:
    - M: Ordenar por uso de memoria. P: Ordenar por uso de CPU (por defecto). k: Terminar un proceso (te pedirá el PID). q: Salir.

© JMA 2020. All rights reserved

Comandos Básicos de Terminal (Vista en Tiempo Real)

- htop:
  - Función: Una alternativa interactiva y más amigable a top, con una interfaz de usuario a color y más intuitiva. Permite desplazar, ordenar y manipular procesos fácilmente con el teclado o el ratón.
  - Uso: htop (necesitas instalarlo si no lo tienes: sudo apt install htop en Debian/Ubuntu, sudo yum install htop en RHEL/CentOS).
  - Ventajas: Más fácil de usar, muestra el uso de CPU por núcleo, tiene barras de progreso, y permite hacer scroll lateral y vertical.

- free:
  - Función: Muestra la cantidad de memoria libre y usada en el sistema (RAM y swap).
  - Uso: free -h (para formato legible por humanos).
  - Interpretación:
    - total: Memoria total.
    - used: Memoria realmente utilizada por los procesos.
    - free: Memoria totalmente libre (sin nada).
    - shared: Memoria compartida por procesos.
    - buff/cache: Memoria utilizada para buffers del kernel y caché de archivos (esto es memoria "libre" que puede ser reclamada rápidamente si un proceso la necesita).
    - available: Memoria disponible para nuevas aplicaciones (incluye free y parte de buff/cache). Este es el valor más relevante para saber cuánta memoria tienes realmente disponible.

© JMA 2020. All rights reserved

Comandos Básicos de Terminal (Vista en Tiempo Real)

- df:
  - Función: Muestra el espacio en disco utilizado y disponible para los sistemas de archivos montados.
  - Uso: df -h (para formato legible por humanos).
  - Interpretación: Muestra el tamaño total, usado, disponible y el porcentaje de uso para cada partición. Busca particiones con alto porcentaje de uso.
- du:
  - Función: Estima el espacio en disco utilizado por archivos o directorios.
  - Uso: du -sh /path/to/directory (para tamaño total de un directorio de forma legible). Útil para encontrar qué directorios están ocupando mucho espacio.
- iostat:
  - Función: Reporta estadísticas de uso de CPU y actividad de E/S (Input/Output) de dispositivos de almacenamiento. Es parte del paquete sysstat.
  - Uso: iostat -xz 1 (muestra estadísticas detalladas, extendidas y sobre E/S por segundo, actualizando cada 1 segundo).
  - Interpretación clave:
    - %util: Porcentaje de tiempo durante el cual el dispositivo está ocupado procesando solicitudes. Valores altos (>70%) indican un posible cuello de botella de E/S.
    - await: Tiempo promedio (en milisegundos) que las solicitudes de E/S pasan en la cola de espera del dispositivo.
    - svctm: Tiempo promedio (en milisegundos) para que el dispositivo de E/S procese una solicitud.
    - r/s, w/s: Lecturas/escrituras por segundo.
    - rkB/s, wkB/s: Kilobytes leídos/escritos por segundo.

© JMA 2020. All rights reserved

Comandos Básicos de Terminal (Vista en Tiempo Real)

- vmstat:
  - Función: Informa sobre procesos, memoria, paginación, E/S de bloques, traps y actividad de CPU.
  - Uso: vmstat 1 (actualiza cada segundo).
  - Interpretación clave:
    - r: Procesos en ejecución o esperando CPU (runnable). Un valor alto indica una CPU sobrecargada.
    - b: Procesos en sueño no interrumpible (esperando E/S). Un valor alto indica un cuello de botella de E/S.
    - si / so: Cantidad de memoria intercambiada desde/hacia el disco (swap in/swap out). Valores altos indican falta de RAM.
    - wa (CPU): Tiempo de espera de E/S.
- netstat / ss:
  - Función: Muestran conexiones de red, tablas de enrutamiento, estadísticas de interfaz, etc. ss es la herramienta moderna y más rápida que netstat.
  - Uso:
    - netstat -tulnp (TCP, UDP, listening, numerical, processes).
    - ss -tulnp (equivalente moderno).
    - netstat -s (estadísticas de red).
  - Interpretación: Busca puertos en escucha, conexiones activas y estados de conexión (ESTABLISHED, WAIT_TIME, etc.).
- sar (System Activity Reporter):
  - Función: Parte del paquete sysstat, es una herramienta muy potente para recopilar, reportar y guardar información sobre la actividad del sistema (CPU, memoria, E/S, red, etc.) a lo largo del tiempo.
  - Uso:
    - sar -u 1 5: 5 informes de uso de CPU con 1 segundo de intervalo.
    - sar -r 1 5: 5 informes de uso de memoria con 1 segundo de intervalo.
    - sar -b 1 5: 5 informes de actividad de E/S de bloques con 1 segundo de intervalo.
    - sar -n DEV 1 5: 5 informes de actividad de red por interfaz.
    - sar -q 1 5: 5 informes de longitud de la cola de ejecución y carga promedio.
  - Ventaja: Puede leer datos históricos (sar -f /var/log/sysstat/saXX) lo que es invaluable para analizar problemas pasados.

© JMA 2020. All rights reserved

Herramientas de Monitorización Interactivas y Gráficas

- gnome-system-monitor / ksysguard (para entornos de escritorio):
  - Función: Interfaces gráficas que muestran el uso de CPU, memoria, red y procesos en tiempo real, a menudo con gráficos históricos. Equivalentes al "Administrador de Tareas" de Windows.
  - Uso: Busca "Monitor del sistema" en tu menú de aplicaciones.
- nmon:
  - Función: Herramienta interactiva de línea de comandos que ofrece una vista consolidada y muy detallada de CPU, memoria, discos, red, NFS y mucho más.
  - Uso: nmon (instala si no lo tienes: sudo apt install nmon). Una vez dentro, presiona letras para ver diferentes estadísticas (ej., c para CPU, m para memoria, d para discos, n para red).
- Glances:
  - Función: Un monitor de sistema de línea de comandos multiplataforma que utiliza la biblioteca psutil para obtener información. Proporciona una gran cantidad de datos en una sola pantalla con colores y alertas.
  - Uso: glances (instala si no lo tienes: sudo pip install glances o vía tu gestor de paquetes).

© JMA 2020. All rights reserved

Interpretación de los Datos de Rendimiento

- Carga de CPU: Si %us (usuario) y %sy (sistema) son consistentemente altos (ej., >80-90%), la CPU es un cuello de botella. Si %wa (espera de E/S) es alto, la E/S es el problema.
- Uso de Memoria: Si free es bajo y swap in/out (si/so en vmstat) es alto, el sistema está usando mucha memoria de intercambio, lo que es mucho más lento que la RAM y degrada gravemente el rendimiento. Busca procesos que consuman mucha memoria (%MEM en top/htop).
- Actividad de Disco (E/S): %util alto en iostat o b alto en vmstat indican que el disco está trabajando al máximo.
- Actividad de Red: Un alto número de paquetes o bytes por segundo en sar -n DEV o un uso de ancho de banda cercano al límite de tu interfaz de red pueden indicar un cuello de botella de red.
- Carga Promedio (Load Average): En top, si el load average es consistentemente mayor que el número de núcleos de CPU de tu sistema, es un indicio de sobrecarga.
- Procesos: Identifica los procesos que consumen más CPU o memoria. A menudo, el problema de rendimiento se reduce a uno o dos procesos problemáticos.

© JMA 2020. All rights reserved

METODOLOGÍA

© JMA 2020. All rights reserved

Introducción

- Las pruebas de rendimiento no deben ser una actividad de "último minuto", deben integrarse a lo largo de todo el ciclo de vida del desarrollo del software. Esto asegura que los problemas de rendimiento se detecten y resuelvan tempranamente, cuando son más fáciles y económicos de corregir. No es una actividad aislada al final del ciclo de vida, sino un proceso continuo que influye y se beneficia de todas las fases del desarrollo del software para asegurar la calidad y eficiencia del sistema.
- La prueba de rendimiento es de naturaleza iterativa. Cada prueba proporciona información valiosa sobre el rendimiento de la aplicación y del sistema. La información obtenida de una prueba se utiliza para corregir u optimizar la aplicación y los parámetros del sistema. La siguiente iteración de la prueba mostrará los resultados de las modificaciones, y así sucesivamente hasta que se alcancen los objetivos de prueba.

© JMA 2020. All rights reserved

Integración en el ciclo de vida

- **Requisitos y Diseño**: Las bases del rendimiento se sientan en las fases iniciales.
  - Identificación de requisitos no funcionales: Es crucial definir claramente los requisitos de rendimiento (tiempos de respuesta, rendimiento, uso de recursos, etc.) desde el principio. Estos requisitos deben ser medibles y cuantificables.
  - Diseño de la arquitectura: Las decisiones de diseño de la arquitectura del sistema, incluyendo la elección de tecnologías y la infraestructura, tienen un impacto significativo en el rendimiento. Es vital que estas decisiones consideren los requisitos de rendimiento establecidos.
- **Implementación**: Durante la codificación y desarrollo, se deben tomar medidas proactivas para asegurar el rendimiento.
  - Optimización del código: Los desarrolladores deben escribir código eficiente y optimizado desde el principio.
  - Pruebas unitarias de rendimiento: Aunque no son pruebas de carga, las pruebas unitarias pueden incluir verificaciones básicas de rendimiento para componentes individuales, como el tiempo de ejecución de algoritmos críticos.
  - Métricas y monitoreo: Se deben integrar mecanismos de monitoreo y recolección de métricas en el código para facilitar el seguimiento del rendimiento en etapas posteriores.

© JMA 2020. All rights reserved

Integración en el ciclo de vida

- **Pruebas y Evaluación**: Esta es la fase donde las pruebas de rendimiento formal se ejecutan, pero no es la única.
  - Pruebas de rendimiento formales: Se diseñan y ejecutan diferentes tipos de pruebas de rendimiento (carga, estrés, resistencia, escalabilidad, etc.) para validar los requisitos de rendimiento definidos.
  - Entorno de pruebas: El entorno utilizado para estas pruebas debe ser lo más similar posible al entorno de producción para obtener resultados realistas.
  - Análisis de resultados: Se analizan los datos recopilados (tiempos de respuesta, throughput, uso de recursos) y se comparan con los requisitos y las líneas base establecidas.
  - Identificación de cuellos de botella: Se utilizan herramientas de monitoreo y análisis para identificar las causas de los problemas de rendimiento (código ineficiente, base de datos lenta, recursos insuficientes, …).
- **Despliegue y Mantenimiento**: Incluso después del despliegue, el rendimiento sigue siendo una preocupación.
  - Monitoreo en producción: Después del despliegue, el sistema debe ser monitoreado continuamente en el entorno de producción para detectar cualquier degradación del rendimiento en tiempo real.
  - Pruebas post-despliegue: Se pueden realizar pequeñas pruebas de humo o pruebas de carga ligeras para validar que el rendimiento se mantiene después de actualizaciones o cambios.
  - Retroalimentación: La información del rendimiento en producción alimenta el ciclo de vida, ayudando a ajustar requisitos futuros o a planificar mejoras.

© JMA 2020. All rights reserved

Actividades

- Las pruebas de rendimiento son un proceso estructurado que involucra varias actividades clave para garantizar que una aplicación o sistema cumpla con sus expectativas de velocidad, escalabilidad y estabilidad.
- Las principales actividades son:
  - Planificación de la Prueba de Rendimiento
  - Diseño y Desarrollo de Scripts de Prueba
  - Ejecución de la Prueba de Rendimiento
  - Análisis de Resultados e Informes
  - Sintonización y Re-pruebas (Retest)

© JMA 2020. All rights reserved

Planificación

- Esta es la fase inicial y crucial donde se sientan las bases de toda la estrategia de prueba.
- Las principales tareas son:
  - Definir Objetivos y Alcance: Se establecen los motivos por los que se realizan las pruebas (por ejemplo, validar un requisito, identificar cuellos de botella) y qué partes del sistema se probarán.
  - Identificar Requisitos No Funcionales (NFRs): Se especifican las métricas de rendimiento esperadas, como los tiempos de respuesta (ej., "el 90% de las transacciones de inicio de sesión deben responder en menos de 2 segundos"), el rendimiento (throughput) (ej., "el sistema debe procesar 100 transacciones por segundo") y los límites de concurrencia (ej., "soportar 500 usuarios concurrentes").
  - Definir el Entorno de Prueba: Se planifica el entorno donde se ejecutarán las pruebas, buscando que sea lo más similar posible al entorno de producción en cuanto a hardware, software, configuración de red y datos.
  - Seleccionar Herramientas: Se eligen las herramientas de prueba de rendimiento adecuadas (JMeter, Gatling, LoadRunner, …) y las herramientas de monitoreo (Dynatrace, Prometheus, …).
  - Crear el Plan de Pruebas: Se documenta detalladamente la estrategia, los objetivos, los recursos, el cronograma y las responsabilidades.

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

- El Plan de Prueba de Rendimiento (Performance Test Plan) es un documento esencial en el ciclo de vida de las pruebas de software, especialmente cuando el rendimiento es un factor crítico para el éxito de una aplicación.
- El plan no es simplemente una lista de lo que se va a probar, sino un documento estratégico que guía todo el proceso de pruebas de rendimiento, desde la definición inicial hasta el análisis final.
- El plan debe ser exhaustivo, claro y medible, y debe alinearse con los requisitos no funcionales (NFRs) del proyecto.
- El plan es una guía exhaustiva que cubre desde la justificación de las pruebas hasta los detalles técnicos de su ejecución y análisis. Su objetivo principal es asegurar que el rendimiento del sistema sea una consideración central y medible a lo largo de todo el ciclo de vida del desarrollo.

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

1. Introducción
   - Propósito del Plan: Explica el objetivo del documento y por qué se están llevando a cabo las pruebas de rendimiento. (Ej., "Este plan describe las actividades para asegurar que el sistema X cumpla con los requisitos de rendimiento establecidos antes del lanzamiento").
   - Alcance de las Pruebas: Define claramente qué partes del sistema (módulos, funcionalidades, APIs) se incluirán y cuáles se excluirán de las pruebas de rendimiento.
   - Audiencia: Identifica a quién va dirigido el plan (equipo de pruebas, desarrolladores, gerentes de proyecto, stakeholders).

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

2. Requisitos de Rendimiento (Objetivos y Criterios): Esta es la sección más crítica, ya que establece lo que se espera del sistema.
   - Objetivos de Rendimiento: Describe las metas generales de las pruebas (Ej., "Verificar la escalabilidad del sistema hasta 1000 usuarios concurrentes", "Identificar cuellos de botella en la base de datos", "Validar los tiempos de respuesta de las transacciones críticas").
   - Requisitos No Funcionales (NFRs) Específicos: Detalla los criterios de éxito medibles y cuantificables. Estos son los oráculos para tus pruebas. Incluye:
     - Tiempos de Respuesta: Promedio, máximo y percentiles (ej., 90%, 95%) para transacciones clave.
     - Throughput (Rendimiento): Número de transacciones por segundo/minuto o hits por segundo que el sistema debe manejar.
     - Capacidad de Usuarios Concurrentes: Cuántos usuarios activos puede soportar el sistema.
     - Utilización de Recursos: Umbrales aceptables para CPU, memoria, disco y red en servidores de aplicación, bases de datos y otros componentes.
     - Tasas de Error: Porcentaje máximo de errores tolerado bajo carga.
     - Resistencia/Estabilidad: Comportamiento esperado durante periodos prolongados de carga.
     - Escalabilidad: Cómo debe responder el sistema al aumentar los recursos.

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

3. Estrategia de Pruebas de Rendimiento: Esta sección detalla cómo se llevarán a cabo las pruebas.
   - Tipos de Pruebas de Rendimiento a Realizar: Describe cuáles de los siguientes tipos se ejecutarán y por qué:
     - Pruebas de Carga (Load Testing): Para verificar el rendimiento bajo carga esperada.
     - Pruebas de Estrés (Stress Testing): Para encontrar el punto de quiebre del sistema.
     - Pruebas de Resistencia/Estabilidad (Endurance/Soak Testing): Para detectar fugas de memoria o degradación a lo largo del tiempo.
     - Pruebas de Escalabilidad (Scalability Testing): Para entender cómo el sistema se comporta al añadir recursos.
     - Pruebas de Pico (Spike Testing): Para simular aumentos repentinos de carga.
   - Metodología de Prueba: Describe el enfoque general (basado en objetivos, en riesgos).
   - Fases de las Pruebas: Si las pruebas se dividirán en fases (ej., pruebas de unidad de rendimiento, pruebas de integración de rendimiento, pruebas de sistema de rendimiento).

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

5. Escenarios de Prueba y Modelos de Carga: Describe el "qué" y el "cómo" de la simulación de usuarios.
   - Identificación de Escenarios de Negocio Críticos: Lista las transacciones y funcionalidades clave que se simularán, basadas en su frecuencia, importancia o impacto en el rendimiento.
   - Modelo de Carga de Trabajo: Detalla el comportamiento esperado de los usuarios reales. Esto incluye:
     - Distribución de Usuarios: Cuántos usuarios realizarán cada tipo de transacción.
     - "Think Time" (Tiempo de Reflexión): El tiempo que los usuarios "esperarán" entre acciones para simular un comportamiento realista.
     - Volumen de Transacciones: Número esperado de transacciones por unidad de tiempo para cada escenario.
   - Scripts de Prueba: Se mencionará que se crearán scripts para cada escenario, y se harán consideraciones sobre la parametrización y correlación de datos.

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

6. Recopilación y Análisis de Datos: Define cómo se medirán y evaluarán los resultados.
   - Métricas a Recopilar: Lista específica de los contadores de rendimiento clave que se monitorearán durante las pruebas (CPU, memoria, E/S de disco, E/S de red, tiempo de respuesta de BD, etc.).
   - Procedimientos de Monitoreo: Cómo se recopilarán los datos (herramientas de monitoreo, logs).
   - Proceso de Análisis: Cómo se analizarán los datos para identificar cuellos de botella y evaluar el cumplimiento de los NFRs.
   - Criterios de Aceptación/Criterios de Fallo: Reitera los umbrales específicos que determinarán si una prueba es exitosa o si se debe considerar un fallo.
7. Roles y Responsabilidades: Aclara quién hace qué.
   - Equipo de Pruebas de Rendimiento: Describe los roles y responsabilidades de los miembros del equipo (ingeniero de pruebas de rendimiento, arquitecto, administrador de sistema, etc.).
   - Stakeholders: Identifica a otros equipos o individuos que participarán o serán informados (desarrolladores, administradores de base de datos, gerencia).

© JMA 2020. All rights reserved

Plan de Prueba de Rendimiento

8. Calendario y Entregables: Establece el cronograma y los resultados esperados.
   - Cronograma: Un cronograma estimado para las fases de planificación, diseño, ejecución y análisis.
   - Entregables: Lista los documentos y artefactos que se producirán (ej., scripts de prueba, informes de ejecución de pruebas, informe final de prueba de rendimiento).
9. Gestión de Riesgos: Identifica y planifica la mitigación de posibles problemas.
   - Riesgos Potenciales: Los riesgos asociados a las pruebas de rendimiento (ej., entorno no representativo, datos de prueba insuficientes, herramientas inadecuadas, problemas de tiempo).
   - Planes de Mitigación: Estrategias para abordar cada riesgo identificado.
10. Glosario y Abreviaturas:
    - Proporciona definiciones de términos técnicos y acrónimos utilizados en el plan.

© JMA 2020. All rights reserved

Análisis

- Análisis de la Base de Pruebas de Rendimiento:
  - Evaluar los requisitos no funcionales (NFRs) de rendimiento: Comprender y verificar que los NFRs estén claros, sean medibles, consistentes y realistas (ej., tiempos de respuesta, throughput, capacidad, utilización de recursos, objetivos de escalabilidad).
  - Revisar la arquitectura del sistema y el diseño: Entender cómo está construido el sistema, identificar los componentes clave, las dependencias y los posibles cuellos de botella (bases de datos, servicios externos, balanceadores de carga, etc.). Esto ayuda a definir qué partes del sistema necesitan ser monitoreadas y probadas con mayor detalle.
  - Analizar el perfil de usuario y el modelado de carga: Estudiar cómo los usuarios interactúan con el sistema en condiciones reales o esperadas. Esto incluye:
    - Identificar los escenarios de negocio críticos: Las transacciones o flujos de trabajo más importantes o más frecuentes.
    - Definir la distribución de la carga: Determinar el porcentaje de usuarios que realizarán cada escenario.
    - Establecer los tiempos de reflexión (think times): Simular el tiempo que un usuario real tarda en "pensar" entre acciones.
    - Considerar el volumen de datos: Definir la cantidad y el tipo de datos que se procesarán durante las pruebas.
  - Identificar los riesgos de rendimiento: Detectar posibles problemas que podrían afectar el rendimiento del sistema (ej., picos de tráfico inesperados, lentitud de la base de datos, capacidad insuficiente del servidor).
- Priorizar las condiciones de prueba de rendimiento: Con base en el análisis de riesgos y la importancia del negocio, decidir qué escenarios y tipos de pruebas de rendimiento (carga, estrés, resistencia, etc.) son los más críticos para ejecutar.

© JMA 2020. All rights reserved

Diseño y Desarrollo de Scripts de Prueba

- Aquí se preparan las "acciones" que simularán a los usuarios.
- Las principales tareas son:
  - Identificar Escenarios de Uso Críticos: Se seleccionan las funcionalidades clave o las transacciones más utilizadas por los usuarios (ej., inicio de sesión, búsqueda de productos, añadir al carrito, realizar un pedido).
  - Modelado de Carga y Perfiles de Usuario: Se simula el comportamiento de los usuarios reales. Esto incluye cuántos usuarios realizarán cada tipo de transacción, con qué frecuencia y con qué "tiempo de reflexión" (el tiempo que un usuario real espera entre acciones).
  - Crear Scripts de Prueba: Se desarrollan los scripts automatizados que imitarán el comportamiento del usuario, enviando solicitudes al sistema. Estos scripts deben ser parametrizados para usar diferentes datos de prueba (nombres de usuario, IDs de producto, etc.) y deben manejar las correlaciones (valores dinámicos que el sistema genera y que deben ser capturados y reutilizados en solicitudes posteriores).
  - Preparar Datos de Prueba: Se generan o se preparan datos realistas y suficientes para que los scripts de prueba puedan ejecutar las transacciones con variabilidad.

© JMA 2020. All rights reserved

Diseño y Desarrollo de Scripts de Prueba

- Diseñar los escenarios de prueba de rendimiento:
  - Convertir las condiciones de prueba en escenarios de prueba: Traducir los flujos de trabajo identificados en el análisis en secuencias de acciones concretas que las herramientas de prueba puedan simular.
  - Definir los patrones de carga: Establecer cómo se aumentará el número de usuarios virtuales (rampa de carga), la duración de la prueba, el número máximo de usuarios, etc., para cada tipo de prueba (carga, estrés, resistencia).
  - Diseñar el modelo de datos de prueba: Planificar cómo se obtendrán, crearán y gestionarán los datos necesarios para la ejecución de las pruebas. Estos datos deben ser realistas y variados para evitar efectos de caché falsos o repetición de datos que no reflejen el uso real.

© JMA 2020. All rights reserved

Diseño y Desarrollo de Scripts de Prueba

- Diseñar la estrategia de monitoreo:
  - Identificar las métricas clave a recolectar: Seleccionar los contadores de rendimiento específicos que se monitorearán en cada componente del sistema (CPU, memoria, disco, red, bases de datos, servidores de aplicaciones, etc.).
  - Planificar el uso de herramientas de monitoreo: Determinar qué herramientas se usarán para la recopilación de métricas (ej., Monitor de Rendimiento en Windows, sar en Linux, APM, etc.).
- Definir la estrategia de análisis de resultados: Planificar cómo se compararán los resultados obtenidos con los NFRs y cómo se identificarán los cuellos de botella.
- Especificar los procedimientos de prueba: Detallar los pasos necesarios para configurar, ejecutar y limpiar cada prueba de rendimiento.

© JMA 2020. All rights reserved

Diseño y Desarrollo de Scripts de Prueba

- Desarrollar y optimizar scripts de prueba:
  - Crear los scripts de generación de carga: Utilizar la herramienta de prueba de rendimiento seleccionada (como JMeter o Gatling) para programar las secuencias de solicitudes HTTP/otras que simulan el comportamiento de los usuarios.
  - Parametrizar los scripts: Reemplazar los valores fijos en los scripts con variables que se extraen de los datos de prueba, para simular usuarios diferentes (ej. login con distintos usuarios, búsqueda de diferentes productos).
  - Correlacionar los scripts: Manejar los valores dinámicos que el sistema genera en tiempo de ejecución (ej. ID de sesión, tokens de seguridad, IDs de transacción) para asegurar que los scripts se comporten como un usuario real.
  - Depurar y validar los scripts: Asegurarse de que los scripts funcionan correctamente a nivel de un solo usuario antes de aplicar carga.

© JMA 2020. All rights reserved

Diseño y Desarrollo de Scripts de Prueba

- Configurar el entorno de prueba:
  - Aprovisionar la infraestructura: Asegurarse de que los servidores, la red, las bases de datos y los generadores de carga estén configurados según lo planificado.
  - Desplegar el sistema bajo prueba: Asegurarse de que la versión correcta de la aplicación esté desplegada en el entorno de prueba.
  - Configurar herramientas de monitoreo: Instalar y configurar los agentes y las herramientas de monitoreo en los servidores y componentes relevantes.
- Preparar los datos de prueba:
  - Cargar o generar los datos de prueba: Asegurarse de que la base de datos o el sistema contenga los volúmenes de datos necesarios para los escenarios de prueba.
  - Limpiar los datos de prueba: Establecer procedimientos para restablecer el estado de los datos entre ejecuciones si es necesario.
  - Establecer el plan de ejecución: Organizar el orden y la secuencia de las pruebas de rendimiento a ejecutar.

© JMA 2020. All rights reserved

Ejecución

- Esta es la fase donde se aplica la carga al sistema.
- Las principales tareas son:
  - Verificar el Entorno: Se aseguran de que el entorno de prueba esté listo, las herramientas instaladas y el sistema bajo prueba esté en un estado conocido.
  - Ejecutar Escenarios de Carga: Se lanzan los scripts de prueba, aplicando gradualmente la carga definida (número de usuarios virtuales, tasa de transacciones) al sistema.
  - Monitoreo en Tiempo Real: Mientras las pruebas están en curso, se monitorean activamente los recursos del sistema (CPU, memoria, disco, red) y las métricas de la aplicación (tiempos de respuesta, tasas de error) para detectar problemas a medida que ocurren.

© JMA 2020. All rights reserved

Análisis de Resultados e Informes

- Una vez que las pruebas terminan, se evalúa el rendimiento.
- Las principales tareas son:
  - Recopilación de Datos: Se recolectan todas las métricas de rendimiento (tiempos de respuesta, rendimiento, uso de recursos, errores) de las herramientas de prueba y de monitoreo.
  - Análisis de Cuellos de Botella: Se analizan los datos para identificar dónde está fallando el sistema o dónde se encuentra el límite de su capacidad. Esto puede involucrar el análisis de logs, trazas de base de datos o perfiles de aplicación.
  - Generación de Informes: Se crea un informe detallado que resume los hallazgos. Este informe debe incluir:
    - Los objetivos de la prueba y el escenario.
    - Las métricas clave y si se cumplieron o no los requisitos de rendimiento.
    - Los cuellos de botella identificados y sus posibles causas.
    - Recomendaciones para mejorar el rendimiento.
  - Comunicación de Resultados: Los hallazgos se comunican a las partes interesadas (desarrollo, operaciones, negocio) para informar las decisiones sobre optimización o escalabilidad.

© JMA 2020. All rights reserved

Análisis de Resultados e Informes

- Recopilación y Consolidación de Datos:
  - Consolidar métricas: Reunir los datos de rendimiento de todas las fuentes: la herramienta de generación de carga (ej., JMeter, Gatling, LoadRunner), las herramientas de monitoreo del sistema (ej., Monitor de Rendimiento en Windows, sar/iostat en Linux, APM, Zabbix, Prometheus/Grafana) y los logs de la aplicación/servidor.
  - Sincronizar datos: Asegurarse de que los datos de diferentes fuentes estén alineados temporalmente para una correlación precisa.
- Evaluación del Cumplimiento de los Requisitos:
  - Comparar resultados con los NFRs: Cotejar las métricas obtenidas (tiempos de respuesta, rendimiento, tasas de error) con los Requisitos No Funcionales (NFRs) de rendimiento y los criterios de éxito/fallo definidos en el plan de pruebas.
  - Identificar desviaciones: Resaltar cualquier punto donde el rendimiento medido no cumpla con las expectativas.

© JMA 2020. All rights reserved

Análisis de Resultados e Informes

- Identificación de Cuellos de Botella:
  - Correlacionar métricas: Buscar relaciones entre el rendimiento de la aplicación y la utilización de los recursos del sistema. Por ejemplo, si los tiempos de respuesta se disparan cuando el uso de CPU alcanza el 90%, la CPU es un posible cuello de botella.
  - Analizar el uso de recursos: Examinar el uso de CPU, memoria, disco, red, base de datos y otros componentes clave para identificar qué recurso está saturado o es el factor limitante.
  - Examinar los logs de la aplicación y del servidor: Buscar errores, advertencias, transacciones lentas, o patrones de comportamiento inusuales que puedan indicar problemas de rendimiento.
  - Profundizar en la aplicación (si es posible): Utilizar herramientas de perfilado de código o APM para identificar métodos o consultas de base de datos específicos que son ineficientes o que consumen muchos recursos.

© JMA 2020. All rights reserved

Análisis de Resultados e Informes

- Análisis de Tendencias:
  - Observar el comportamiento a lo largo del tiempo: En pruebas de resistencia (soak tests), buscar degradación del rendimiento con el tiempo, lo que podría indicar fugas de memoria o acumulación de recursos.
  - Analizar la estabilidad bajo carga: Evaluar si el sistema es estable o si experimenta fallos o errores bajo carga sostenida.
- Recomendaciones para la Optimización:
  - Con base en el análisis, proponer soluciones o áreas de mejora para los cuellos de botella identificados (ej., optimizar consultas SQL, refactorizar código, añadir índices, escalar hardware, ajustar configuraciones de software).

© JMA 2020. All rights reserved

Informes de las Pruebas de Rendimiento

- El informe es la comunicación formal de los resultados del análisis. Debe ser claro, conciso y proporcionar la información necesaria para que los stakeholders tomen decisiones informadas. La estructura un buen informe de rendimiento suele incluir:
  - Resumen Ejecutivo: Una visión general de alto nivel de los resultados clave, si se cumplieron los objetivos y las principales conclusiones y recomendaciones. Orientado a la gerencia.
  - Introducción: Propósito de la prueba, alcance, entorno de prueba y NFRs objetivo de la prueba.
  - Metodología de Prueba: Cómo se realizó la prueba (tipos de prueba, herramientas, escenarios, modelo de carga).
  - Resultados Detallados: Presentación de las métricas clave (tiempos de respuesta, throughput, tasas de error) para cada escenario y tipo de prueba, a menudo con gráficos y tablas.
  - Análisis de Cuellos de Botella: Descripción de los problemas de rendimiento identificados, la evidencia que los respalda y su impacto.
  - Recomendaciones: Acciones específicas y priorizadas para resolver los problemas de rendimiento.
  - Conclusiones y Próximos Pasos: Resumen final y sugerencias para futuras pruebas o mejoras.
  - Apéndices: Datos brutos, configuraciones, logs detallados, etc.

© JMA 2020. All rights reserved

Informes de las Pruebas de Rendimiento

- Un buen informe de rendimiento debe contar con:
  - Claridad y Visualización:
    - Utilizar gráficos y tablas para presentar los datos de forma clara y comprensible. Las visualizaciones ayudan a identificar tendencias y anomalías rápidamente.
    - Asegurarse de que el lenguaje sea claro, conciso y libre de jerga innecesaria para la audiencia a la que va dirigido.
  - Objetividad y Basado en Evidencia:
    - Presentar los resultados de forma objetiva, respaldando todas las afirmaciones con datos y evidencias (ej., capturas de pantalla de monitoreo, fragmentos de logs).
    - Evitar la especulación; si la causa raíz no está clara, indicarlo y sugerir una investigación adicional.
  - Comunicación:
    - Presentar el informe: Compartir el informe con los stakeholders relevantes (equipo de desarrollo, arquitectos, administradores de sistemas, gerencia, dueños de producto).
    - Sesión de preguntas y respuestas: Estar preparado para discutir los hallazgos, explicar la metodología y defender las recomendaciones.

© JMA 2020. All rights reserved

Sintonización y Re-pruebas (Retest)

- Las pruebas de rendimiento suelen ser un proceso iterativo.
- Las principales tareas son:
  - Optimización del Sistema: Basado en el análisis, los equipos de desarrollo y operaciones implementan cambios para mejorar el rendimiento (ej., optimizar código, ajustar configuraciones de base de datos, añadir hardware).
  - Ejecución de Re-pruebas: Después de implementar las mejoras, se vuelven a ejecutar las pruebas de rendimiento (o al menos las pruebas de regresión de rendimiento) para verificar que los cambios han tenido un efecto positivo y no han introducido nuevos problemas.

© JMA 2020. All rights reserved

HERRAMIENTAS

© JMA 2020. All rights reserved

Automatización de la Prueba

- La automatización de la prueba permite ejecutar muchos casos de prueba de forma consistente y repetida en las diferentes versiones del sistema sujeto a prueba (SSP) y/o entornos. Pero la automatización de pruebas es más que un mecanismo para ejecutar un juego de pruebas sin interacción humana. Implica un proceso de diseño de productos de prueba, entre los que se incluyen:
  - Software.
  - Documentación.
  - Casos de prueba.
  - Entornos de prueba
  - Datos de prueba
- Una Solución de Automatización Pruebas (SAP) debe permitir:
  - Implementar casos de prueba automatizados.
  - Monitorizar y controlar la ejecución de pruebas automatizadas.
  - Interpretar, informar y registrar los resultados de pruebas automatizadas.

© JMA 2020. All rights reserved

Objetivos de la automatización de pruebas

- Mejorar la eficiencia de la prueba.
- Aportar una cobertura de funciones más amplia.
- Reducir el coste total de la prueba.
- Realizar pruebas que los probadores manuales no pueden.
- Acortar el período de ejecución de la prueba.
- Aumentar la frecuencia de la prueba y reducir el tiempo necesario para los ciclos de prueba.

© JMA 2020. All rights reserved

Ventajas y Desventajas de la automatización

Ventajas

- Se pueden realizar más pruebas por compilación.
- La posibilidad de crear pruebas que no se pueden realizar manualmente (pruebas en tiempo real, remotas, en paralelo).
- Las pruebas pueden ser más complejas.
- Las pruebas se ejecutan más rápido.
- Las pruebas están menos sujetas a errores del operador.
- Uso más eficaz y eficiente de los recursos de pruebas
- Información de retorno más rápida sobre la calidad del software.
- Mejora de la fiabilidad del sistema (por ejemplo, repetibilidad y consistencia).
- Mejora de la consistencia de las pruebas.

Desventajas

- Requiere tecnologías adicionales.
- Existencia de costes adicionales.
- Inversión inicial para el establecimiento de la SAP.
- Requiere un mantenimiento continuo de la SAP.
- El equipo necesita tener competencia en desarrollo y automatización.
- Puede distraer la atención respecto a los objetivos de la prueba, por ejemplo, centrándose en la automatización de casos de prueba a expensas del objetivo de las pruebas.
- Las pruebas pueden volverse más complejas.
- La automatización puede introducir errores adicionales.

© JMA 2020. All rights reserved

Limitaciones de la automatización de pruebas

- No todas las pruebas manuales se pueden automatizar.
- La automatización sólo puede comprobar resultados predecibles e interpretables por la máquina.
- La automatización sólo puede comprobar los resultados reales que pueden ser verificados por un oráculo de prueba automatizado.
- No es un sustituto de las pruebas exploratorias.

© JMA 2020. All rights reserved

Técnicas para la automatización de pruebas

- Las herramientas para la automatización de pruebas funcionales aplican técnicas diferentes para la creación y ejecución de pruebas:
  - Técnica de **registro/reproducción** (Record & Playback): Esta técnica consiste en grabar una ejecución de la prueba realizada en la interfaz de la aplicación y su reproducción posterior.
  - Técnica de **Programación de Scripts Estructurados**: Difiere de la técnica de registro/reproducción en la introducción de una librería de scripts reutilizables, que realizan secuencias de instrucciones que se requieren comúnmente en una serie de pruebas. Los scripts de pruebas se crean en un lenguaje de scripting común.
  - Técnica de **Programación con Frameworks**: Los marcos de pruebas unitarias (unitarias, integración, sistema) simplifican la creación y ejecución de los procedimientos de pruebas en el mismo lenguaje que el objeto de la prueba. La ejecución y cobertura de las pruebas se pueden integrar en los entornos de desarrollo.
    - Unit Test Frameworks: JUnit, MSTest, NUnit, xUnit, Jest, PHPUnit, …
    - Mocking Frameworks: Mockito, Microsoft Fakes, Moq, NSubstitute, Rhino Mocks, …
    - Code Coverage: Jacoco, Clover, NCover, RCov, …
  - Técnica de **Data-Driven**: Esta es una técnica que se enfoca en la separación de los datos de prueba de los scripts, y los almacena en archivos separados. Por lo tanto, los scripts sólo contendrán los procedimientos de prueba y las acciones para la aplicación.
  - Técnica de **Keyword-Driven**: Se basa en la recuperación de los procedimientos de prueba desde los scripts, quedando sólo los datos de prueba y acciones específicas de prueba, que se identifican por palabras clave.

© JMA 2020. All rights reserved

Entornos de pruebas

- Un enfoque de varios entornos permite compilar, probar y liberar código con mayor velocidad y frecuencia para que la implementación sea lo más sencilla posible. Permite quitar la sobrecarga manual y el riesgo de una versión manual y, en su lugar, automatizar el desarrollo con un proceso de varias fases destinado a diferentes entornos.
  - Desarrollo: es donde se desarrollan los cambios en el software.
  - Prueba: permite que los evaluadores humanos o las pruebas automatizadas prueben el código nuevo y actualizado. Los desarrolladores deben aceptar código y configuraciones nuevos mediante la realización de pruebas unitarias en el entorno de desarrollo antes de permitir que esos elementos entren en uno o varios entornos de prueba.
  - Ensayo/preproducción: donde se realizan pruebas finales inmediatamente antes de la implementación en producción, debe reflejar un entorno de producción real con la mayor precisión posible.
  - UAT: Las pruebas de aceptación de usuario (UAT) permiten a los usuarios finales o a los clientes realizar pruebas para comprobar o aceptar el sistema de software antes de que una aplicación de software pueda pasar a su entorno de producción.
  - Producción: es el entorno con el que interactúan directamente los usuarios.

© JMA 2020. All rights reserved

Herramientas de automatización de pruebas

Selenium
APACHE JMeter™
Telerik Test Studio
TestingWhiz Code Less, Test More
LAMBDATEST
Kobiton
appium
SoapUI Supported by SMARTBEAR
FitNesse
POSTMAN
cucumber
Katalon
HP UFT
TestArchitect

© JMA 2020. All rights reserved

Tipos de herramientas

- Generadores de Carga
  - El generador, a través de un IDE, un editor de guiones o un paquete de herramientas, es capaz de crear y ejecutar múltiples instancias de cliente que simulan el comportamiento del usuario de acuerdo con un perfil operativo definido. La creación de múltiples instancias en períodos cortos de tiempo causará carga en un sistema bajo prueba. El generador crea la carga y también recoge las métricas para su posterior comunicación.
  - Al ejecutar las pruebas de rendimiento, el objetivo del generador de carga es imitar el mundo real tanto como sea práctico. Esto a menudo significa que se necesitan solicitudes de usuarios procedentes de varias ubicaciones, no sólo de la ubicación de la prueba. Los entornos que se configuran con múltiples puntos de presencia se distribuirán donde se origina la carga, de modo que no todo provenga de una sola red. Esto proporciona realismo a la prueba, aunque a veces puede sesgar los resultados si los saltos en las redes intermedias crean retrasos.
  - El generador de carga se debe ejecutar servidor que dedica todos sus recursos de forma exclusiva para realizar las tareas de simulación de los usuarios virtuales, es recomendable que solo contenga el sistema operativo. No debe compartir recursos con otros procesos, porque se realizan tareas extremadamente precisas de medición de tiempo y cualquier proceso ajeno a la prueba podría afectar negativamente las métricas, y potencialmente arruinar los cronometrajes.

© JMA 2020. All rights reserved

Tipos de herramientas

- Consola de Gestión de la Carga
  - La consola de gestión de carga proporciona el control para iniciar y detener los generadores de carga. La consola también agrega métricas de las diversas transacciones que se definen dentro de las instancias de carga utilizadas por el generador. La consola permite visualizar informes y gráficos de las ejecuciones de las pruebas y soporta el análisis de resultados.
- Herramienta de Monitorización
  - Las herramientas de monitorización se ejecutan simultáneamente con el componente o sistema sujeto a prueba y supervisan, registran y/o analizan el comportamiento del componente o sistema. Los componentes típicos que son monitorizados incluyen colas de servidores web, memoria del sistema y espacio en disco. Las herramientas de monitorización pueden apoyar eficazmente el análisis de la causa raíz del rendimiento en un sistema sujeto a prueba y también se puede utilizar para controlar un entorno de producción cuando se entrega el producto. Durante la prueba de rendimiento también se pueden utilizar monitores de ejecución en el propio generador de carga.

© JMA 2020. All rights reserved

Adecuación de las Herramientas

- Compatibilidad: En general, se selecciona una herramienta para la organización y no sólo para un proyecto. Esto significa que se deben tener en cuenta los siguientes factores en la organización:
  - Protocolos: Los protocolos son un aspecto muy importante para la selección de herramientas de rendimiento. Entender qué protocolos utiliza un sistema y cuáles de ellos serán probados proporcionará la información necesaria para evaluar la herramienta de prueba adecuada.
  - Interfaces a componentes externos: Puede ser necesario considerar las interfaces con componentes de software u otras herramientas como parte de los requisitos de integración completa para cumplir con los requisitos de proceso u otros requisitos de interoperabilidad (por ejemplo, la integración en el proceso de IC).
  - Plataformas: La compatibilidad con las plataformas (y sus versiones) dentro de una organización es esencial. Esto se aplica a las plataformas utilizadas para alojar las herramientas y a las plataformas con las que las herramientas interactúan para la monitorización y/o generación de cargas.
- Escalabilidad: Otro factor a considerar es el número total de simulaciones de usuarios concurrentes que la herramienta puede tratar. Esto incluirá varios factores:
  - Número máximo de licencias requeridas.
  - Requisitos de configuración de la estación de trabajo/servidor de generación de cargas.
  - Capacidad para generar carga desde múltiples puntos de presencia (por ejemplo, servidores distribuidos).

© JMA 2020. All rights reserved

Adecuación de las Herramientas

- Capacidad de ser Entendido: Un factor importante a considerar es el nivel de conocimiento técnico necesario para utilizar la herramienta. Esto a menudo se pasa por alto y puede llevar a que probadores inexpertos configuren incorrectamente las pruebas, lo que a su vez proporciona resultados inexactos. Para las pruebas que requieren escenarios complejos y un alto nivel de programabilidad y personalización, los equipos deben asegurarse de que el probador cuente con las competencias, los antecedentes y la capacitación necesarios.
- Monitorización:
  - ¿Es suficiente el control que proporciona la herramienta?
  - ¿Existen otras herramientas de monitorización disponibles en el entorno que puedan utilizarse para complementar la monitorización de la herramienta?
  - ¿Se puede correlacionar la monitorización con las transacciones definidas?
  - Todas estas preguntas deben ser contestadas para determinar si la herramienta proporcionará la monitorización requerida por el proyecto.
  - Cuando la monitorización es un programa/herramientas/pila completa, se puede utilizar para monitorizar el entorno de producción cuando se entrega el producto.

© JMA 2020. All rights reserved

Herramientas para Pruebas de Rendimiento

- **Apache JMeter**: Una herramienta de código abierto muy popular y versátil para pruebas de carga y rendimiento. Permite probar varios protocolos (HTTP/HTTPS, FTP, JDBC, SOAP/REST, JMS, etc.).
- **Gatling**: Un framework de pruebas de carga de código abierto basado en Scala y Akka, conocido por su alto rendimiento y escalabilidad, ideal para aplicaciones web modernas.
- **K6**: Una herramienta de código abierto escrita en Go, diseñada para pruebas de rendimiento, escalabilidad y estrés, con scripting en JavaScript.
- **LoadRunner** (Micro Focus): Una herramienta comercial muy potente y completa, utilizada en entornos empresariales grandes.
- **NeoLoad** (Tricentis): Herramienta comercial que ofrece pruebas de carga y estrés para aplicaciones web y móviles, con funcionalidades avanzadas.
- **BlazeMeter**: Una plataforma de pruebas de rendimiento basada en la nube que soporta JMeter, Gatling, Selenium y más.
- **Locust**: Una herramienta de pruebas de carga de código abierto escrita en Python, que permite escribir escenarios de prueba en código Python.

© JMA 2020. All rights reserved

https://jmeter.apache.org/

JMETER

© JMA 2020. All rights reserved

Introducción

- Apache JMeter es un software de código abierto, una aplicación Java 100% pura, diseñada para cargar el comportamiento funcional de las pruebas y medir el rendimiento.
- Originalmente fue diseñado para probar aplicaciones web, pero desde entonces se ha expandido a otras funciones de prueba.
- Apache JMeter puede usarse para probar el rendimiento tanto en recursos estáticos como dinámicos: aplicaciones web dinámicas.
- Se puede usar para simular una carga pesada en un servidor, grupo de servidores, red u objeto para probar su resistencia o para analizar el rendimiento general bajo diferentes tipos de carga.
- JMeter no es apropiado para pruebas E2E dado que no es un navegador, funciona a nivel de protocolo. En lo que respecta a servicios web y servicios remotos, JMeter se parece a un navegador (o más bien, a múltiples navegadores); sin embargo, JMeter no realiza todas las acciones admitidas por los navegadores. En particular, JMeter no ejecuta el Javascript que se encuentra en las páginas HTML. Tampoco representa las páginas HTML como lo hace un navegador (es posible ver la respuesta como HTML, etc., pero los tiempos no se incluyen en ninguna muestra, y solo una muestra en una secuencia se muestra a la vez).

© JMA 2020. All rights reserved

Características

- IDE de prueba con todas las funciones que permite la grabación rápida del plan de prueba (desde navegadores o aplicaciones nativas), construcción y depuración .
- Modo en línea de comandos (consola de gestión de la carga) para cargar la prueba desde cualquier SO compatible con Java (Linux, Windows, Mac OSX, ...)
- Un informe HTML completo y listo para presentar.
- Correlación sencilla mediante la capacidad de extraer datos de los formatos de respuesta más populares: HTML, JSON, XML o cualquier formato de texto
- Portabilidad completa y 100% Java puro.
- El marco completo de subprocesos múltiples permite el muestreo simultáneo mediante varios subprocesos y el muestreo simultáneo de diferentes funciones mediante grupos de subprocesos separados.
- Caché y análisis/reproducción fuera de línea de los resultados de las pruebas.

© JMA 2020. All rights reserved

Características

- Núcleo altamente extensible:
  - Los muestreadores enchufables permiten capacidades de prueba ilimitadas.
  - Muestreadores de secuencias de comandos (lenguajes compatibles con JSR223 como Groovy y BeanShell)
  - Se pueden elegir varias estadísticas de carga con temporizadores conectables.
  - Los complementos de análisis y visualización de datos permiten una gran extensibilidad y personalización.
  - Las funciones se pueden utilizar para proporcionar una entrada dinámica a una prueba o para proporcionar manipulación de datos.
  - Integración Continua simplificada a través de bibliotecas de código abierto de terceras partes para Maven, Gradle y Jenkins.

© JMA 2020. All rights reserved

Tipos de aplicaciones/servidores/protocolo

- Web: HTTP, HTTPS (Java, NodeJS, PHP, ASP.NET, Servicios Web de SOAP / REST, ...)
- TCP
- FTP
- Correo: SMTP (S), POP3 (S) e IMAP (S)
- Middleware orientado a mensajes (MOM) a través de JMS
- Base de datos a través de JDBC, Mongo, Bolt (Neo4j)
- LDAP
- Comandos nativos o scripts de shell
- Objetos Java

© JMA 2020. All rights reserved

Debilidades

- Aunque permite también el diseño y automatización de pruebas funcionales, existen herramientas más adecuadas para ello.
- JMeter NO se comporta como un navegador. Esto tiene varias implicaciones importantes: por defecto no guarda ni envía cookies, no interpreta código JavaScript, … Cualquiera de estas funcionalidades debe ser implementada específicamente en el testplan
- Con JMeter, el probador trabaja a nivel de protocolos: el desarrollador de un testplan debe descender a este nivel, por lo que normalmente el probador tiene que apoyarse en herramientas adicionales durante el desarrollo de un testplan, como Firebug, HttpFox, SoapUI, Badboy, ...
- Los tipos de aplicaciones que se pueden testear con JMeter dependen de los protocolos que implementen las interfaces de acceso a la aplicación. Con JMeter se pueden testear los siguientes tipos de interfaces: HTTP, HTTPS, SOAP (sobre HTTP), XML-RPC (sobre HTTP), FTP, LDAP, POP3, IMAP, SMTP, JMS, JDBC y TCP. El testeo de otro tipo de interfaces no es inmediato. En concreto, con JMeter no es posible (con un esfuerzo razonable), implementar un testplan para una interfaz RMI o WebDav

© JMA 2020. All rights reserved

Instalación

- Descargar JMeter
  - https://jmeter.apache.org/
- Descomprimir
- Descargar JMeter Plugins
  - https://jmeter-plugins.org/install
- Copiar plugins-manager.jar en jmeter/lib/ext
- Descargar drivers de JDBC y copiar los jar en ./lib
- Configurar, si es necesario, jmeter/bin/jmeter.bat
- Abrir GUI: jmeter/bin/jmeterw.cmd

© JMA 2020. All rights reserved

Modos

- Modo GUI
- Modo sin GUI (modo de línea de comandos)
  - Para las pruebas de carga, se debe ejecutar JMeter en modo sin GUI para obtener los resultados óptimos.

    jmeter -n -t my_test.jmx -l log.jtl
- Modo de servidor
  - Para las pruebas distribuidas, se ejecuta JMeter en modo servidor en los nodos remotos y se controla los servidores desde el GUI. También puede utilizar el modo no GUI para ejecutar pruebas remotas. Para iniciar cada servidor en cada host:

    jmeter-server

© JMA 2020. All rights reserved

Plan de Pruebas (Test Plan)

- El Plan de Pruebas (Test Plan) es donde se especifican las configuraciones generales para una prueba. Un plan de pruebas describe una serie de pasos que JMeter ejecutará. Un plan de pruebas completo constará de uno o más grupos de subprocesos, controladores lógicos, controladores generadores de muestras, escuchas, temporizadores, aserciones y elementos de configuración. JMeter dispone de plantillas con planes de ejemplo.
- El plan de pruebas es el elemento raíz de cualquier script de JMeter y contiene todos los demás elementos que definen tu prueba de rendimiento. Se pueden definir variables estáticas para valores que se repiten a lo largo de una prueba, como los nombres de los servidores.
- Piensa en el Plan de Pruebas como un árbol, donde cada elemento que añades es una rama o una hoja que contribuye a la simulación de la carga. Sus características son:
  - Propósito: Define la prueba completa. Aquí se pueden establecer variables globales, propiedades del usuario, e incluso añadir un plan de ejecución distribuido.
  - Ubicación: Es el nodo superior en el árbol de JMeter.
  - Persistencia: Se guardan en ficheros con formato XML y .jmx como extensión.

© JMA 2020. All rights reserved

Elementos de un plan de prueba

- Thread Group (Grupo de Hilos)
  - Propósito: Define cómo y cuántos usuarios virtuales (hilos) simularán la carga. Es la población de tus usuarios de prueba.
  - Ubicación: Directamente bajo el Test Plan. Puedes tener múltiples Thread Groups para simular diferentes tipos de usuarios o escenarios.
- Config Elements (Elementos de Configuración)
  - Propósito: Proporcionan configuración y valores predeterminados para los Samplers que se encuentran dentro de su ámbito. No envían peticiones por sí mismos.
  - Ubicación: Pueden estar en el Test Plan (afectando a todo el plan), en un Thread Group, o dentro de Samplers específicos.

© JMA 2020. All rights reserved

Elementos de un plan de prueba

- Controladores
  - Samplers (Muestreadores)
    - Propósito: Son los "solicitantes". Representan la acción que un usuario virtual realiza, enviando una petición específica a un servidor y esperando una respuesta. Es donde JMeter mide el tiempo de respuesta.
    - Ubicación: Dentro de un Thread Group, o dentro de un Logic Controller.
  - Logic Controllers (Controladores Lógicos)
    - Propósito: Controlan el orden en que se ejecutan los Samplers y otros elementos dentro de su ámbito. Permiten definir la lógica de negocio y el flujo de los escenarios.
    - Ubicación: Dentro de un Thread Group, conteniendo Samplers y otros Logic Controllers.

© JMA 2020. All rights reserved

Elementos de un plan de prueba

- Procesadores
  - Pre-Processors (Pre-Procesadores)
    - Propósito: Se ejecutan antes de que su Sampler padre (o los Samplers en su ámbito) se ejecute. Son útiles para preparar datos o realizar acciones previas a la petición.
    - Ubicación: Antes del Sampler al que afectan.
  - Post-Processors (Post-Procesadores)
    - Propósito: Se ejecutan después de que su Sampler padre (o los Samplers en su ámbito) se haya ejecutado. Son esenciales para extraer datos de la respuesta de una petición.
    - Ubicación: Después del Sampler al que afectan.

© JMA 2020. All rights reserved

Elementos de un plan de prueba

- Assertions (Aserciones)
  - Propósito: Verifican que la respuesta del servidor a un Sampler sea la esperada. Si una aserción falla, el Sampler se marca como "fallido".
  - Ubicación: Bajo un Sampler (afectando solo a ese Sampler) o en un Logic Controller/Thread Group (afectando a todos los Samplers dentro de su ámbito).
- Listeners (Monitores/Escuchadores)
  - Propósito: Muestran los resultados de la prueba en diferentes formatos (tablas, gráficos, archivos).
  - Ubicación: Bajo el Test Plan (muestran los resultados de toda la prueba) o bajo un Thread Group/Logic Controller (muestran solo los resultados de su ámbito).

© JMA 2020. All rights reserved

Grabador de scripts de prueba HTTP(S)

- El grabador de scripts de prueba HTTP(S) permite a JMeter interceptar y registrar sus acciones mientras navega por su aplicación web con su navegador habitual. JMeter creará objetos de muestra de prueba y los almacenará directamente en su plan de pruebas sobre la marcha (para que pueda ver ejemplos de forma interactiva mientras los crea). El grabador se implementa como un servidor proxy HTTP(S). Debe configurar su navegador para que use el proxy para todas las solicitudes HTTP y HTTPS.
- Desde JMeter 2.10, se ha mejorado la grabación para gestionar mejor los recursos incrustados y la creación de certificados sobre la marcha. Para habilitar estas funciones, JMeter utiliza la utilidad keytool (disponible en JRE/JDK), por lo que debe asegurarse de que su configuración sea correcta.
- Seleccionando la plantilla de grabación en la lista plantillas, se genera un plan de pruebas completo. Una vez configurados los Valores predeterminados de solicitud HTTP se hace clic en el botón Iniciar en la parte superior.
- Esto iniciará el servidor proxy de JMeter, que se utiliza para interceptar las solicitudes del navegador. Se generará un archivo llamado ApacheJMeterTemporaryRootCA.crt en la carpeta JMETER_HOME/bin. En este punto, el proxy de JMeter se está ejecutando.

© JMA 2020. All rights reserved

Grupo de hilos

- Los elementos del grupo de hilos son los puntos de inicio de cualquier plan de prueba. Todos los controladores y muestreadores deben estar bajo un grupo de subprocesos. El grupo de hilos controla el número de hilos que JMeter usará para ejecutar la prueba. Los controles para un grupo de hilos le permiten:
  - Establecer el número de hilos
  - Establecer el período de aceleración
  - Establecer el número de veces para ejecutar la prueba.
- Cada hilo ejecutará el plan de prueba en su totalidad y de forma completamente independiente de otros hilos de prueba. Se utilizan varios subprocesos para simular conexiones simultáneas a su aplicación de servidor.
- El período de aceleración le dice a JMeter cuánto tiempo se tarda en "aumentar" la cantidad total de hilos elegidos.
- La expansión debe ser lo suficientemente larga para evitar una carga de trabajo demasiado grande al inicio de una prueba, y lo suficientemente corta para que los últimos hilos comiencen a ejecutarse antes de que terminen los primeros (a menos que uno quiera que eso ocurra).
- Así mismo permite controlar la acción a tomar después de un error del muestrador.

© JMA 2020. All rights reserved

Thread Group (predeterminado)

- Este es el Thread Group predeterminado que viene con JMeter y es el más utilizado para la mayoría de las pruebas de carga y resistencia.
- Funcionalidad: Ideal para simular un número fijo de usuarios que ejecutan un plan de prueba con una rampa de inicio y un número de iteraciones (o duración). Es la base para pruebas de carga, resistencia y, con ajustes, incluso estrés.
- Configuraciones clave:
  - Number of Threads (users): La cantidad de usuarios virtuales a simular.
  - Ramp-up period (seconds): El tiempo que JMeter tarda en iniciar todos los usuarios.
  - Loop Count: Cuántas veces cada usuario ejecutará el plan de prueba (puede ser infinito).
  - Scheduler (Opcional): Permite definir una duración específica para la prueba, anulando el Loop Count si es infinito.
- Fortalezas: Sencillo de configurar, cubre la mayoría de las necesidades básicas de pruebas de carga.
- Limitaciones: Menos flexible para escenarios complejos de rampas de carga/descarga o picos muy específicos sin la ayuda de Timers o Controllers adicionales.

© JMA 2020. All rights reserved

Configuración y desmontaje

- setUp Thread Group:
  - Un tipo especial de grupo de hilos que se puede utilizar para realizar acciones previas a la prueba. El comportamiento de estos hilos es idéntico al de un elemento de grupo de hilos normal. La diferencia radica en que estos hilos se ejecutan antes de que la prueba proceda a la ejecución de grupos de hilos normales.
- tearDown Thread Group:
  - Un tipo especial de grupo de hilos que se puede utilizar para realizar acciones posteriores a la prueba. El comportamiento de estos hilos es idéntico al de un elemento de grupo de hilos normal. La diferencia radica en que estos hilos se ejecutan después de que la prueba haya terminado de ejecutar sus grupos de hilos habituales.

© JMA 2020. All rights reserved

Ultimate Thread Group

(Plugin: Custom Thread Groups)

- Este es un plugin de JMeter muy popular que ofrece un control mucho más granular sobre el perfil de carga, especialmente útil para pruebas de estrés, picos o escenarios de carga mixta con varias fases.
- Funcionalidad: Permite definir el ciclo de vida de los usuarios mediante una tabla: cuántos usuarios arrancan, en qué tiempo, cuánto tiempo se mantienen y cuánto tiempo tardan en parar. Esto facilita la creación de perfiles de carga con rampas ascendentes y descendentes, mesetas y picos.
- Configuraciones clave (en una tabla):
  - Start Threads: Cuántos usuarios iniciar en esta fase.
  - Initial Delay: Retraso antes de que esta fase comience.
  - Startup Time: Tiempo para iniciar los hilos de esta fase (rampa de subida).
  - Hold Load For: Tiempo para mantener la carga estable.
  - Shutdown Time: Tiempo para detener los hilos de esta fase (rampa de bajada).
- Fortalezas:
  - Perfiles de carga complejos: Permite crear patrones de carga muy específicos y realistas, como subir la carga gradualmente, mantenerla estable y luego reducirla.
  - Múltiples fases: Puedes combinar varias filas en la tabla para crear perfiles de carga que simulen diferentes comportamientos a lo largo del tiempo.
- Limitaciones: Requiere la instalación del plugin "JMeter Plugins Manager".

© JMA 2020. All rights reserved

Concurrency Thread Group

(Plugin: Custom Thread Groups)

- Otro plugin de JMeter diseñado para mantener un número específico de usuarios concurrentes activos en el sistema, lo cual es muy útil para pruebas de rendimiento basadas en niveles de concurrencia.
- Funcionalidad: En lugar de solo definir el "Número de Hilos" y el "Ramp-up", este Thread Group se enfoca en que la herramienta intente activamente mantener el número de usuarios que están actualmente ejecutando transacciones dentro del sistema. JMeter gestionará la inyección de nuevos usuarios para reemplazar a los que terminan, buscando mantener la concurrencia objetivo.
- Configuraciones clave:
  - Target Concurrency: El número de usuarios concurrentes que JMeter intentará mantener.
  - Ramp Up Time: Tiempo para alcanzar la concurrencia objetivo.
  - Ramp-up Steps Count (Opcional): Si quieres subir la concurrencia en pasos definidos.
  - Hold Target Rate Time: Tiempo para mantener la concurrencia objetivo.
  - Thread Lifetime (Opcional): Duración máxima que un hilo individual vivirá.
- Fortalezas:
  - Control de concurrencia preciso: Ideal cuando tu objetivo principal es mantener un nivel exacto de usuarios activos procesando peticiones.
  - Simulación realista: Es muy útil para entornos donde el número de usuarios activos es la métrica clave.
- Limitaciones: También requiere la instalación del plugin "JMeter Plugins Manager". Puede ser más complejo de entender al principio que el Thread Group estándar.

© JMA 2020. All rights reserved

Controladores

- Los controladores conducen el procesamiento de una prueba. JMeter tiene dos tipos de controladores: muestreadores y controladores lógicos.
- Los muestreadores le dicen a JMeter que envíe solicitudes a un servidor. Por ejemplo, se agrega un muestreador de Petición HTTP si se desea que JMeter envíe una solicitud HTTP. También se puede personalizar una solicitud agregando uno o más elementos de configuración a un muestreador.
- Los controladores lógicos permiten personalizar la lógica que JMeter utiliza para decidir cuándo enviar solicitudes. Por ejemplo, puede agregar un controlador de lógica de intercalación para alternar entre dos muestreadores de solicitud HTTP.
- El Fragmento de prueba es un tipo especial de controlador que existe en el árbol del Plan de prueba al mismo nivel que el Grupo de subprocesos. Se distingue de un Grupo de subprocesos en que no se ejecuta a menos que sea referenciado por un Controlador de Módulo o de Include. Este elemento es únicamente para reutilizar el código dentro de los planes de prueba.

© JMA 2020. All rights reserved

MUESTREADORES

© JMA 2020. All rights reserved

Muestreadores

- Los muestreadores le dicen a JMeter que envíe solicitudes a un servidor y espere una respuesta. Se procesan en el orden en que aparecen en el árbol. Los controladores se pueden utilizar para modificar el número de repeticiones de una muestra.
- Los muestreadores JMeter incluyen:
  - Solicitud HTTP (también se puede utilizar para el servicio web SOAP o REST)
  - Solicitud de FTP
  - Solicitud de correo
  - Solicitud de TCP
  - Solicitud de objeto Java
  - Solicitud JDBC
  - Solicitud JMS
  - Solicitud de prueba de JUnit
  - Solicitud LDAP
  - Solicitud de proceso OS

© JMA 2020. All rights reserved

HTTP Request (Petición HTTP)

- Propósito: Este es, con diferencia, el Sampler más utilizado. Sirve para simular peticiones a servidores web que utilizan los protocolos HTTP o HTTPS.
- Comportamiento: Permite configurar todos los aspectos de una petición web:
  - Protocolo: http o https.
  - Servidor o IP: El dominio o la dirección IP del servidor web (ej., www.ejemplo.com).
  - Puerto: Opcional (por defecto 80 para HTTP, 443 para HTTPS).
  - Método: GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD, TRACE.
  - Ruta: La parte de la URL después del dominio (ej., /login, /api/productos).
  - Parámetros: Para enviar datos en la URL (GET) o en el cuerpo de la petición (POST).
  - Cuerpo de la Petición: Para enviar datos complejos como JSON, XML o texto plano (muy común para APIs REST).
  - Multipart/form-data: Para subir archivos o enviar formularios complejos.
- Uso Común: Pruebas de rendimiento de sitios web, APIs RESTful, microservicios, aplicaciones web modernas, cualquier aplicación que se comunique a través de navegadores o clientes HTTP.
- Consideraciones: A menudo se usa junto con HTTP Header Manager (para cabeceras personalizadas), HTTP Cookie Manager (para manejar sesiones) y HTTP Cache Manager (para simular el caché del navegador).

© JMA 2020. All rights reserved

Access Log Sampler (Muestreador del Log de Acceso)

- Propósito: Permite a JMeter "reproducir" patrones de tráfico de usuario basándose en datos reales capturados en archivos de log de acceso de servidores web (como Apache HTTP Server, Nginx, o IIS).
- Comportamiento: Lee líneas de un archivo de log de acceso, parsea la información (como la URL, el método HTTP, los parámetros) y construye peticiones HTTP dinámicamente para enviarlas a un servidor. Puede simular el tráfico tal como ocurrió en producción.
- Configuración Clave:
  - Log File: La ruta al archivo de log del servidor de acceso.
  - Parser: Selecciona el formato del log (ej., org.apache.jmeter.protocol.http.sampler.AccessLogSampler$TCLogParser para logs comunes de Apache).
  - Filter Options: Permite incluir o excluir ciertas peticiones (ej., por URL, por fecha/hora, por código de estado).
  - Server Name or IP: El servidor de destino al que se enviarán las peticiones.
- Uso Común:
  - Simular patrones de carga muy realistas derivados de datos de uso real.
  - Realizar pruebas de regresión de rendimiento con un perfil de tráfico histórico.
  - Validar el impacto de cambios en la infraestructura o el código bajo un escenario de carga conocido.

© JMA 2020. All rights reserved

TCP Sampler (Muestreador TCP)

- Propósito: Este Sampler es para interacciones de bajo nivel con servidores que se comunican directamente a través del protocolo TCP/IP. Es muy flexible pero requiere conocimiento del protocolo específico que se está probando.
- Comportamiento: Establece una conexión TCP con el servidor, envía un bloque de datos (texto o datos binarios) y opcionalmente espera una respuesta.
- Configuración Clave:
  - Nombre de Servidor o IP: La dirección del servidor TCP.
  - Puerto: El puerto del servicio TCP.
  - Texto a Enviar: Los datos que se enviarán al servidor. Puedes usar \n, \r para saltos de línea.
  - Caracteres de Fin de Línea (EOL): Define qué caracteriza el final de la respuesta del servidor (ej., un salto de línea, un carácter específico, o simplemente un timeout).
  - Timeout (ms): Cuánto tiempo esperar una respuesta.
- Uso Común:
  - Pruebas de servicios personalizados basados en sockets.
  - Aplicaciones de chat o juegos multijugador que usan comunicación TCP directa.
  - Depuración de comunicaciones de red de bajo nivel.
  - Protocolos propietarios no cubiertos por otros Samplers.

© JMA 2020. All rights reserved

FTP Request (Petición FTP)

- Propósito: Sirve para simular operaciones de cliente FTP (File Transfer Protocol), como cargar o descargar archivos desde un servidor FTP.
- Comportamiento: Establece una conexión con un servidor FTP y ejecuta una operación de transferencia de archivos.
- Configuración Clave:
  - Nombre de Servidor: La dirección del servidor FTP.
  - Puerto: Opcional (por defecto 21).
  - Nombre de Usuario y Contraseña: Credenciales para acceder al servidor FTP.
  - Archivo Remoto: La ruta y nombre del archivo en el servidor FTP.
  - Archivo Local: La ruta y nombre del archivo local (para operaciones de subida o descarga).
  - Operación:
    - PUT (Upload): Sube un archivo local al servidor FTP.
    - GET (Download): Descarga un archivo del servidor FTP al equipo local de JMeter.
- Uso Común: Probar el rendimiento de servidores FTP bajo carga, medir el rendimiento de la transferencia de archivos, verificar la disponibilidad de servicios FTP.

© JMA 2020. All rights reserved

SMTP Sampler (Muestreador SMTP)

- Propósito: Permite enviar correos electrónicos a través del protocolo SMTP (Simple Mail Transfer Protocol).
- Comportamiento: Simula el envío de un correo electrónico a un servidor SMTP especificado.
- Configuración Clave:
  - Host SMTP: La dirección del servidor SMTP (ej., smtp.gmail.com).
  - Puerto: Opcional (por defecto 25, 465 para SSL, 587 para TLS).
  - Remitente: La dirección de correo del remitente.
  - Destinatario(s): Las direcciones de correo de los destinatarios (To, CC, BCC).
  - Asunto: El asunto del correo.
  - Mensaje: El cuerpo del correo (puede ser texto plano o HTML).
  - Opciones de Autenticación/Seguridad: Nombre de usuario y contraseña, y si se usa SSL/TLS.
  - Adjuntos: Permite adjuntar archivos al correo.
- Uso Común: Probar el rendimiento de servidores de correo saliente, simular el envío masivo de correos electrónicos (ej., para campañas de marketing, notificaciones transaccionales).

© JMA 2020. All rights reserved

Mail Reader Sampler (Muestreador de Lector de Correo)

- Propósito: Permite leer correos electrónicos de un servidor de correo utilizando los protocolos POP3 o IMAP.
- Comportamiento: Se conecta a un buzón de correo, autentica al usuario y realiza operaciones como leer mensajes, obtener encabezados, o eliminar mensajes.
- Configuración Clave:
  - Protocolo: pop3 o imap.
  - Host del Servidor: La dirección del servidor de correo.
  - Puerto: Opcional (por defecto 110 para POP3, 143 para IMAP; para SSL, 995 POP3S, 993 IMAPS).
  - Nombre de Usuario y Contraseña: Credenciales del buzón de correo.
  - Opciones de Seguridad: Si se usa SSL.
  - Operación: Leer mensajes, obtener encabezados, buscar, etc.
  - Eliminar mensajes: Opción para borrar los correos después de leerlos.
- Uso Común:
  - Probar el rendimiento de servidores de correo entrante.
  - Verificar si los correos transaccionales (ej., confirmaciones de registro, restablecimiento de contraseña) se envían y se reciben correctamente durante una prueba de carga.
  - Simular la actividad de usuarios revisando su bandeja de entrada.

© JMA 2020. All rights reserved

JDBC Request (Petición JDBC)

- Propósito: Este Sampler está diseñado para interactuar directamente con bases de datos utilizando el estándar JDBC (Java Database Connectivity). Permite ejecutar comandos SQL y obtener resultados.
- Comportamiento: Se conecta a una base de datos a través de un driver JDBC (que debes haber configurado previamente en un elemento JDBC Connection Configuration). Una vez conectado, ejecuta la consulta o el comando SQL que especifiques y procesa la respuesta.
- Configuración Clave:
  - JDBC Connection Configuration: Debes tener un elemento JDBC Connection Configuration en tu Test Plan o Thread Group, donde defines la URL de la base de datos, el driver de la base de datos (por ejemplo, para MySQL, PostgreSQL, Oracle, SQL Server), nombre de usuario y contraseña, y la configuración del pool de conexiones.
  - Query Type: Define el tipo de operación SQL:
    - Select Statement: Para consultas de selección de datos.
    - Update Statement: Para insertar, actualizar o eliminar datos.
    - Callable Statement: Para ejecutar procedimientos almacenados.
    - Prepared Statement: Para ejecutar consultas precompiladas.
    - Commit/Rollback: Para controlar transacciones.
    - AutoCommit: Para habilitar/deshabilitar el autocommit.
  - SQL Query: El texto de la consulta SQL a ejecutar.
- Uso Común:
  - Probar el rendimiento del backend de la base de datos bajo carga.
  - Simular la carga que las operaciones de una aplicación generan en la DB.
  - Preparar o limpiar datos de prueba antes o después de un escenario (aunque para esto, los pre/post-procesadores también son opciones).
  - Verificar la integridad o el estado de los datos en la base de datos.

© JMA 2020. All rights reserved

MongoDB Script (DEPRECATED) (Script de MongoDB)

- Propósito: Este Sampler, que normalmente se instala como un plugin de JMeter, permite interactuar con bases de datos MongoDB ejecutando scripts de JavaScript (usando la shell de MongoDB) o comandos JSON directamente.
- Comportamiento: Se conecta a una instancia de MongoDB (requiere un elemento de configuración MongoDB Source Config para los detalles de conexión) y ejecuta el código JavaScript o el documento JSON que se le proporcione.
- Configuración Clave:
  - MongoDB Source Config: (Plugin necesario) Donde defines la URI de conexión de MongoDB, nombre de la base de datos, credenciales, etc.
  - Command (JSON): Puedes escribir un comando MongoDB en formato JSON (ej., {"find": "myCollection", "filter": {"name": "Test"}}).
  - Script (JS): Puedes escribir un script de JavaScript que interactúe con la base de datos de MongoDB.
- Uso Común:
  - Probar el rendimiento de las operaciones CRUD (Crear, Leer, Actualizar, Borrar) en una base de datos MongoDB.
  - Simular la carga en un sistema que utiliza MongoDB como su base de datos principal.
  - Validar la persistencia de datos en MongoDB.

© JMA 2020. All rights reserved

Bolt Request (Petición Bolt)

- Propósito: Este Sampler (generalmente un plugin) está diseñado para interactuar con bases de datos de grafos Neo4j utilizando el protocolo Bolt.
- Comportamiento: Se conecta a una instancia de Neo4j y ejecuta consultas en lenguaje Cypher o comandos del protocolo Bolt directamente.
- Configuración Clave:
  - Connection Configuration: (Plugin necesario) Detalles de conexión a Neo4j (URI de Bolt, credenciales).
  - Query: La consulta Cypher a ejecutar (ej., MATCH (n) RETURN n LIMIT 10).
  - Statement Parameters: Parámetros para la consulta Cypher.
  - Protocol Version: La versión del protocolo Bolt a usar.
- Uso Común:
  - Probar el rendimiento de las operaciones de lectura y escritura en una base de datos de grafos Neo4j bajo carga.
  - Simular la carga generada por una aplicación que utiliza Neo4j para sus operaciones de datos.

© JMA 2020. All rights reserved

JMS Publisher (Publicador JMS)

- Propósito: Se utiliza para enviar (publicar) mensajes a una cola (Queue) o un tema (Topic) JMS. Simula un productor de mensajes.
- Comportamiento: Se conecta a un servidor JMS (configurado previamente), crea un mensaje con el contenido y las propiedades que definas, y lo envía al destino JMS especificado. El mensaje es "fire and forget" (dispara y olvida), lo que significa que no espera una respuesta del destino.
- Configuración Clave:
  - Destination: El nombre de la cola o tema JMS al que se enviará el mensaje.
  - Message Type: El tipo de mensaje JMS (ej., Text Message, Map Message, Object Message).
  - Content: El contenido real del mensaje.
  - JMS Properties: Permite añadir propiedades personalizadas al encabezado del mensaje JMS.
- Uso Común:
  - Simular la actividad de productores de mensajes en arquitecturas basadas en eventos o microservicios.
  - Probar el rendimiento y la capacidad de un Message Broker para recibir y enrutar mensajes bajo carga.
  - Simular el envío de notificaciones o eventos a otros componentes del sistema.

© JMA 2020. All rights reserved

JMS Subscriber (Suscriptor JMS)

- Propósito: Se utiliza para recibir (suscribir) mensajes de una cola (Queue) o un tema (Topic) JMS. Simula un consumidor de mensajes.
- Comportamiento: Se conecta a un servidor JMS y espera mensajes del destino configurado. Puede configurarse para esperar un número fijo de mensajes o un tiempo determinado. Una vez recibidos, los mensajes pueden ser procesados (aunque el procesamiento real es interno de JMeter).
- Configuración Clave:
  - Destination: El nombre de la cola o tema JMS del que se esperan recibir mensajes.
  - Timeout (ms): El tiempo máximo de espera para la llegada de mensajes.
  - Receive Queue/Topic: Si el suscriptor es para una cola o un tema.
  - Number of Samples: El número de mensajes que se esperan recibir en este sampler.
- Uso Común:
  - Simular consumidores de mensajes y evaluar el rendimiento del procesamiento de mensajes por parte del Message Broker o de los componentes que consumen.
  - Verificar si los mensajes enviados por otros componentes se están entregando correctamente.
  - Probar la latencia de entrega de mensajes en el sistema.

© JMA 2020. All rights reserved

JMS Point-to-Point (JMS Punto a Punto)

- Propósito: Este Sampler es para modelar la comunicación en un sistema JMS que utiliza el modelo "punto a punto" (colas), donde un mensaje enviado a una cola es consumido por un solo receptor. Puede simular patrones de solicitud-respuesta.
- Comportamiento: Combina el envío y la recepción en un solo Sampler. Envía un mensaje a una cola de solicitud (Request Queue) y, opcionalmente, espera una respuesta de una cola de respuesta (Reply Queue).
- Configuración Clave:
  - Request Queue: La cola a la que se envía el mensaje inicial.
  - Receive Queue: (Opcional) La cola de la que se espera una respuesta.
  - Timeout (ms): El tiempo máximo para esperar la respuesta.
  - Message Type y Content (similares a JMS Publisher).
- Uso Común:
  - Probar el rendimiento de transacciones específicas que involucran un patrón de solicitud-respuesta a través de colas JMS.
  - Simular sistemas transaccionales donde un servicio envía una solicitud a otro y espera una respuesta asíncrona a través de un Message Broker.

© JMA 2020. All rights reserved

LDAP Request (Petición LDAP)

- Propósito: Permite realizar las operaciones más comunes y básicas de LDAP.
- Comportamiento: Se conecta a un servidor LDAP (configurado previamente) y ejecuta la operación LDAP especificada sobre una entrada de directorio.
- Configuración Clave:
  - Host y Puerto: La dirección y el puerto del servidor LDAP.
  - Base DN (Distinguished Name): El punto de partida en el árbol LDAP para la operación.
  - Username y Password: Credenciales para autenticarse en el servidor LDAP.
  - Operación:
    - Add: Añadir una nueva entrada.
    - Modify: Modificar una entrada existente.
    - Delete: Eliminar una entrada.
    - Search: Buscar entradas que coincidan con un filtro.
    - Compare: Comparar un valor de atributo.
    - Rename: Renombrar una entrada.
  - Parámetros específicos de la operación: (ej., filtro de búsqueda para Search, atributos a añadir/modificar para Add/Modify).
- Uso Común:
  - Probar el rendimiento de servidores LDAP bajo carga (ej., simular búsquedas de usuarios para autenticación).
  - Simular operaciones de gestión de usuarios o grupos en un directorio.
  - Verificar la disponibilidad y capacidad de respuesta de los servicios de directorio.

© JMA 2020. All rights reserved

LDAP Extended Request (Petición LDAP Extendida)

- Propósito: Ofrece opciones más avanzadas y detalladas para operaciones LDAP. Está diseñado para cubrir funcionalidades LDAP que no están disponibles en el Sampler LDAP Request básico.
- Comportamiento: Similar al LDAP Request, pero con una interfaz más flexible y campos adicionales para operaciones extendidas. A menudo se utiliza para operaciones que involucran controles LDAP, extensiones o esquemas personalizados.
- Configuración Clave:
  - Connection Parameters: Host, puerto, credenciales.
  - Search Parameters: Base DN, Scope, Deref Aliases, Time Limit, Size Limit, Filter (más opciones que en el básico).
  - Add / Modify / Delete Parameters: Detalles específicos para estas operaciones.
  - Extended Operation: Permite especificar la ID del OID (Object Identifier) de la operación extendida y sus datos.
- Uso Común:
  - Pruebas de rendimiento de características LDAP avanzadas o extensiones específicas.
  - Cuando el LDAP Request básico no proporciona suficiente control para un escenario de prueba particular.
  - Depuración de problemas complejos en el servidor LDAP.

© JMA 2020. All rights reserved

Java Request (Petición Java)

- 3. Propósito: Permite ejecutar código Java personalizado directamente dentro de tu script de JMeter como un Sampler.
- Comportamiento: Para usar este Sampler, necesitas escribir una clase Java que implemente la interfaz org.apache.jmeter.protocol.java.sampler.JavaSamplerClient. Esta clase debe ser compilada y su JAR debe ser colocado en el classpath de JMeter (generalmente en la carpeta lib/ext de JMeter). JMeter luego invocará los métodos definidos en tu clase (como runTest()).
- Configuración Clave:
  - Classname: El nombre completo de tu clase Java personalizada (ej., com.mycompany.jmeter.MyCustomSampler).
  - Parámetros: Puedes pasar parámetros a tu clase Java desde JMeter.
- Uso Común:
  - Probar el rendimiento de componentes Java específicos o librerías de negocio directamente, sin la capa de red.
  - Realizar lógica de prueba muy compleja o personalizada que no se puede lograr con los Samplers existentes o con lenguajes de scripting (como Groovy/JSR223) de manera eficiente.
  - Simular integraciones con sistemas que solo exponen una API Java.

© JMA 2020. All rights reserved

BeanShell Sampler (Muestreador BeanShell)

- Propósito: Permite ejecutar código de scripting en el lenguaje BeanShell como parte de tu script de JMeter.
- Comportamiento: JMeter ejecuta el código BeanShell que escribes en la ventana del Sampler. BeanShell es un lenguaje de scripting ligero compatible con Java. Tiene acceso a las variables y al contexto de JMeter, lo que permite una manipulación avanzada de datos, lógica condicional o incluso la creación de solicitudes personalizadas.
- Configuración Clave:
  - Script: El área de texto donde escribes tu código BeanShell.
- Uso Común:
  - Realizar tareas de pre-procesamiento o post-procesamiento que son demasiado complejas para los extractores o aserciones estándar.
  - Manipular variables de JMeter de formas complejas.
  - Implementar lógica de prueba personalizada.
- Importante: Aunque funcional, BeanShell es significativamente más lento que Groovy para scripts complejos. En versiones recientes de JMeter, el JSR223 Sampler con Groovy es el reemplazo recomendado y preferido para la mayoría de los casos de uso debido a su mucho mejor rendimiento. BeanShell está considerado "deprecated" para nuevos desarrollos.

© JMA 2020. All rights reserved

JSR223 Sampler (Muestreador JSR223)

- Propósito: Este es uno de los Samplers más potentes y flexibles de JMeter. Permite ejecutar código de scripting en cualquier lenguaje compatible con JSR223 (Java Specification Request 223), siendo Groovy el más popular y de mejor rendimiento para JMeter.
- Comportamiento: Ejecuta el código que escribas directamente en el área de script del Sampler. Este código tiene acceso completo a las variables de JMeter, propiedades, contexto de ejecución y APIs de Java, lo que permite una manipulación de datos extremadamente avanzada y la creación de lógica personalizada.
- Configuración Clave:
  - Language: Selecciona el lenguaje de scripting (Groovy, JavaScript, Python, etc.). Se recomienda Groovy por su rendimiento.
  - Script File: Opcional, puedes cargar el script desde un archivo externo.
  - Script: El área de texto donde escribes tu código.
- Uso Común:
  - Implementar lógica de prueba compleja que no se puede lograr con los Samplers o controladores estándar.
  - Manipular variables de JMeter de formas sofisticadas.
  - Realizar procesamiento de datos personalizado antes o después de una petición.
  - Probar APIs o sistemas que requieren una lógica de interacción muy específica o firmas criptográficas complejas.
  - Simular escenarios de carga que necesitan una gran flexibilidad en la interacción.
- Consideraciones: Es el reemplazo moderno y de alto rendimiento para el antiguo BeanShell Sampler.

© JMA 2020. All rights reserved

OS Process Sampler (Muestreador de Procesos del SO)

- Propósito: Permite ejecutar comandos o scripts del sistema operativo (ej., scripts de shell en Linux, comandos de CMD/PowerShell en Windows, scripts de Python o Perl) directamente desde tu script de JMeter.
- Comportamiento: JMeter lanza un nuevo proceso del sistema operativo para ejecutar el comando especificado. Captura la salida estándar (stdout) y el error estándar (stderr) del comando, y mide el tiempo que tarda en ejecutarse.
- Configuración Clave:
  - Command: El comando o la ruta al script ejecutable.
  - Working Directory: El directorio donde se ejecutará el comando.
  - Command Parameters: Argumentos a pasar al comando.
  - Environment Variables: Variables de entorno para el proceso hijo.
  - Timeout (ms): El tiempo máximo para esperar la finalización del proceso.
- Uso Común:
  - Integrar herramientas de línea de comandos externas en tu prueba de carga (ej., ejecutar un cliente de base de datos para una operación específica, o una herramienta de monitoreo).
  - Realizar tareas de preparación o limpieza del entorno de prueba (ej., reiniciar un servicio, limpiar una caché).
  - Probar el rendimiento de scripts o utilidades de línea de comandos.

© JMA 2020. All rights reserved

JUnit Request (Petición JUnit)

- Propósito: Permite integrar y ejecutar pruebas unitarias de Java (escritas con JUnit) directamente dentro de un escenario de prueba de rendimiento de JMeter.
- Comportamiento: JMeter cargará una clase de prueba JUnit específica y ejecutará los métodos de prueba definidos en ella. Los resultados de cada método de prueba se reportarán como un Sampler en los resultados de JMeter.
- Configuración Clave:
  - JUnit 4/5 or JUnit 3.x: Selecciona la versión de JUnit.
  - Classname: El nombre completo de la clase de prueba JUnit (ej., com.mycompany.tests.MyPerformanceUnitTest).
  - Test Method: Puedes ejecutar un método de prueba específico o todos los métodos de la clase.
  - Constructor and Test Method Parameters: Parámetros para pasar a la prueba.
- Uso Común:
  - Pruebas de rendimiento de componentes de código Java aislados o servicios internos (sin la capa de red).
  - Reutilizar pruebas unitarias o de integración existentes para incluirlas en un contexto de carga.
  - Garantizar que las operaciones de negocio complejas dentro del código Java se comporten bien bajo estrés.
- Consideraciones: Requiere que las clases de prueba JUnit estén en el classpath de JMeter.

© JMA 2020. All rights reserved

Flow Control Action (Acción de Control de Flujo)

- Propósito: Este Sampler, antes conocido como "Test Action", no envía peticiones a un servidor externo. Su función principal es controlar el flujo de ejecución de los hilos de JMeter.
- Comportamiento: Realiza una acción específica sobre el hilo actual o sobre toda la prueba.
- Configuración Clave:
  - Action:
    - Pause: Pausa la ejecución del hilo por un tiempo especificado (en milisegundos).
    - Stop: Detiene el hilo actual.
    - Stop Test: Detiene toda la prueba de JMeter (todos los hilos activos) al finalizar la iteración actual de cada hilo.
    - Stop Test Now: Detiene toda la prueba de JMeter inmediatamente, incluso interrumpiendo hilos si es necesario.
    - Go to next loop iteration: Hace que el hilo salte a la siguiente iteración del bucle más cercano (Thread Group o Loop Controller).
  - Duration (ms): Solo para la acción Pause.
- Uso Común:
  - Introducir pausas precisas en el script para simular "think time" o esperas forzadas.
  - Manejar errores: Si una condición de error grave se detecta, puedes detener el hilo o toda la prueba.
  - Controlar bucles: Saltar a la siguiente iteración bajo ciertas condiciones.

© JMA 2020. All rights reserved

Plugins: Samplers

- Es crucial mencionar que la flexibilidad de JMeter se amplía enormemente a través de su comunidad y el ecosistema de plugins. Existen muchísimos Samplers adicionales disponibles a través del JMeter Plugins Manager para probar una variedad aún mayor de protocolos y tecnologías, como:
  - GraphQL HTTP Request: Para probar APIs GraphQL.
  - MongoDB Sampler: Para interactuar con bases de datos MongoDB.
  - Redis Samplers: Para interactuar con Redis.
  - Kafka Samplers: Para productores y consumidores de Apache Kafka.
  - gRPC Sampler: Para probar servicios gRPC.
  - MQTT Sampler: Para probar brokers MQTT.
  - Cassandra Sampler: Para bases de datos Cassandra.
  - Y muchos otros para bases de datos NoSQL, servicios en la nube, etc.

© JMA 2020. All rights reserved

CONTROLADORES LÓGICOS

© JMA 2020. All rights reserved

Controladores lógicos

- Los controladores lógicos le permiten personalizar la lógica que JMeter utiliza para decidir cuándo enviar solicitudes. Los controladores lógicos pueden cambiar el orden de las solicitudes provenientes de sus elementos secundarios. Pueden modificar las solicitudes ellos mismos, hacer que JMeter repita las solicitudes, etc.
- Mediante los controladores lógicos se pueden establecer transacciones, orden, frecuencia y el flujo de control con condicionales, bucles, …
- Un Test Fragment (Fragmento de Prueba) es un tipo especial de Controlador Lógico (Logic Controller) que actúa como un módulo reutilizable de elementos del Plan de Pruebas. Su principal característica es que no se ejecuta por sí mismo; necesita ser referenciado por otros controladores o Samplers para que su contenido se ejecute.

© JMA 2020. All rights reserved

Logic Controllers

- Simple Controller
- Once Only Controller
- Transaction Controller
- Critical Section Controller
- If Controller
- Switch Controller
- Loop Controller
- ForEach Controller
- While Controller
- Interleave Controller
- Throughput Controller
- Random Controller
- Random Order Controller
- Module Controller
- Runtime Controller
- Include Controller
- Recording Controller

© JMA 2020. All rights reserved

Simple Controller (Controlador Simple)

- Propósito: Es el controlador lógico más básico y sirve principalmente para organizar tu script. Simplemente agrupa un conjunto de elementos (Samplers, otros controladores, pre/post-procesadores, etc.) bajo un nombre común.
- Comportamiento: Los elementos anidados dentro de un Simple Controller se ejecutan en secuencia, de arriba abajo, una vez por cada iteración del Thread Group o cualquier bucle padre en el que se encuentre. No añade ninguna lógica de control especial; solo estructura.
- Uso común:
  - Mejorar la legibilidad y mantenimiento de scripts grandes dividiéndolos en secciones lógicas (ej., "Paso de Inicio de Sesión", "Navegación por el Catálogo", "Proceso de Pago").
  - Agrupar elementos para que otros controladores puedan referirse a ellos (aunque para transacciones de tiempo, Transaction Controller es mejor).

© JMA 2020. All rights reserved

Recording Controller (Controlador de Grabación)

- Propósito: Actúa como un contenedor para las peticiones que JMeter graba automáticamente utilizando el HTTP(S) Test Script Recorder (el proxy de grabación de JMeter).
- Comportamiento: No tiene una lógica de control de flujo propia durante la ejecución. Su función principal es organizar los Samplers capturados por el proxy de grabación, colocándolos secuencialmente dentro de él.
- Uso común:
  - Grabar escenarios de usuario: Capturar interacciones del navegador para construir rápidamente un borrador de script de prueba.
  - Organizar las peticiones grabadas por flujos de negocio o pasos lógicos dentro de un Thread Group para facilitar la posterior edición, parametrización y adición de lógica.

© JMA 2020. All rights reserved

Once Only Controller (Controlador Una Sola Vez)

- Propósito: Garantiza que los elementos que contiene se ejecuten una y solo una vez por cada hilo (usuario virtual) durante toda la duración de la prueba.
- Comportamiento: Los Samplers y otros elementos dentro de un Once Only Controller se ejecutarán en la primera iteración del bucle padre (generalmente el Thread Group) para ese usuario específico. En todas las iteraciones subsiguientes del mismo usuario, el contenido de este controlador se ignorará.
- Uso común:
  - Simular acciones que un usuario realiza solo una vez al principio de su sesión, como un inicio de sesión (login), la aceptación de términos y condiciones, o la inicialización de alguna variable específica del usuario.
  - Configuración inicial de datos de prueba por usuario.

© JMA 2020. All rights reserved

Transaction Controller (Controlador de Transacciones)

- Propósito: Agrupa un conjunto de Samplers en una única "transacción lógica" y calcula el tiempo de respuesta combinado para esa transacción.
- Comportamiento: Cuando los Samplers dentro de un Transaction Controller se ejecutan, JMeter suma sus tiempos de respuesta individuales. Si la opción "Generate parent sample" está marcada, el Transaction Controller también generará un Sampler "padre" en los resultados que representará el tiempo total de esa transacción, lo cual es muy útil para el análisis.
- Configuración clave:
  - Generate parent sample: (Recomendado) Crea un Sampler resumen para toda la transacción en los reportes.
  - Include duration of timer and pre/post processors in generated sample: (Usualmente marcado) Incluye el tiempo que los temporizadores, pre-procesadores y post-procesadores añaden al tiempo total de la transacción.
- Uso común:
  - Medir el tiempo de respuesta de un flujo de negocio completo (ej., "Registro de Usuario" que incluye varias peticiones, "Añadir Producto al Carrito", "Proceso de Checkout").
  - Obtener métricas agregadas para partes significativas del negocio, no solo para peticiones individuales.

© JMA 2020. All rights reserved

Critical Section Controller (Controlador de Sección Crítica)

- Propósito: Asegura que solo un hilo (usuario) a la vez pueda ejecutar los elementos contenidos dentro de él. Actúa como un mecanismo de bloqueo (lock).
- Comportamiento: Si un hilo está ejecutando la sección crítica, cualquier otro hilo que intente acceder a ella esperará hasta que el primer hilo complete su ejecución de los elementos dentro del controlador. Esto previene condiciones de carrera en operaciones sensibles.
- Configuración clave:
  - Lock Name: Un nombre único para el bloqueo. Todos los Critical Section Controllers con el mismo "Lock Name" compartirán el mismo bloqueo.
- Uso común:
  - Realizar operaciones que deben ser atómicas o que solo pueden ser realizadas por un usuario a la vez (ej., la generación de un ID único y no repetible desde un sistema externo).
  - Interactuar con recursos compartidos donde la concurrencia podría llevar a corrupción de datos o errores.

© JMA 2020. All rights reserved

If Controller
(Controlador Condicional)

- Propósito: Ejecuta los elementos que contiene solo si una condición específica es verdadera. Es el equivalente de una sentencia if en programación.
- Comportamiento: Evalúa una expresión booleana. Si el resultado es true, los elementos anidados (Samplers, otros controladores) se ejecutan. Si el resultado es false, se omiten.
- Configuración clave:
  - Condition: La expresión booleana a evaluar. Puede ser una función de JMeter (ej., ${__jexl3(${responseCode} == 200)}), una variable, o cualquier expresión JavaScript/JEXL3.
  - Interpret Condition as Variable Expression?: (Recomendado) Marcar esta casilla si tu condición es una expresión JMeter que necesita ser evaluada. Si no está marcada, la condición se interpreta como código JavaScript/Groovy directamente.
  - Evaluate for all children?: (Usualmente no se marca) Si está marcado, la condición se reevalúa para cada hijo directo en lugar de solo al inicio del controlador.
- Uso común:
  - Simular flujos condicionales en la lógica de la aplicación (ej., "si el login es exitoso, ir a la página principal; si no, ir a la página de error").
  - Realizar acciones específicas solo cuando se cumple una condición (ej., "si la variable de stock es cero, no intentar añadir al carrito").

© JMA 2020. All rights reserved

Switch Controller
(Controlador de Condicional Múltiple)

- Propósito: Ejecuta uno de sus elementos anidados basándose en un valor de entrada específico, similar a una estructura switch o case en programación.
- Comportamiento: Evalúa el valor de una variable o una cadena que le proporciones. Si ese valor coincide con el nombre (o número de índice) de uno de sus hijos directos, solo ese hijo se ejecuta. Si no hay coincidencia, nada se ejecuta por defecto, a menos que definas un caso predeterminado (el primer hijo puede configurarse como tal).
- Configuración clave: Switch Value: La variable o el valor literal que el controlador usará para decidir qué hijo ejecutar (ej., ${userType}, páginaB).
- Uso común: Simular diferentes rutas de usuario basadas en un estado, tipo de usuario o valor dinámico (ej., si el usuario es "admin" va por una ruta, si es "cliente" por otra).

© JMA 2020. All rights reserved

Loop Controller (Controlador de Bucle Fijo)

- Propósito: Permite repetir la ejecución de un conjunto de elementos (Samplers, otros controladores) un número específico de veces o indefinidamente.
- Comportamiento: Los elementos dentro de este controlador se ejecutarán la cantidad de veces configurada por cada iteración de su padre (generalmente un Thread Group o un Loop Controller anidado).
- Configuración clave: Loop Count: El número de repeticiones (ej., 5 para cinco veces, o Forever para un bucle infinito).
- Uso común: Simular usuarios que realizan una acción repetitiva (ej., navegar por 5 páginas en un catálogo, añadir 3 ítems diferentes al carrito).

© JMA 2020. All rights reserved

ForEach Controller (Controlador de Iteración)

- Propósito: Itera sobre un grupo de variables que comparten un prefijo común, ejecutando los elementos anidados para cada valor de la variable.
- Comportamiento: Toma un conjunto de variables numeradas con el mismo prefijo (ej., producto_1, producto_2, producto_3), las procesa una por una, y asigna el valor actual a una nueva variable de salida. Los elementos anidados se ejecutan una vez por cada valor procesado.
- Configuración clave:
  - Input Variable Prefix: El prefijo común de las variables a iterar (ej., producto).
  - Output Variable Name: El nombre de la variable donde se almacenará el valor actual de la iteración (ej., currentProduct).
- Uso común: Procesar listas de ítems obtenidas de una respuesta anterior (ej., ítems en un carrito, IDs de productos de una búsqueda) y realizar una acción con cada uno de ellos.

© JMA 2020. All rights reserved

While Controller
(Controlador Bucle Mientras)

- Propósito: Ejecuta sus elementos anidados repetidamente mientras una condición específica sea verdadera. Es análogo a un bucle while en lenguajes de programación.
- Comportamiento: La condición se evalúa antes de cada iteración. Si la condición es true, los elementos se ejecutan. Si es false, el controlador se salta y el hilo continúa con los elementos posteriores.
- Configuración clave: Condition: La expresión booleana a evaluar (ej., ${__jexl3(${userFound} == "false")}, ${LAST_SAMPLE_OK} para continuar mientras la última petición fue exitosa).
- Uso común: Bucle continuo hasta que se cumpla una condición (ej., hasta que se reciba un código de respuesta específico, o hasta que un valor de variable cambie).

© JMA 2020. All rights reserved

Interleave Controller
(Controlador Intercalado)

- Propósito: Ejecuta solo uno de sus elementos anidados en cada iteración del bucle padre, rotando entre ellos.
- Comportamiento: En la primera iteración, ejecuta el primer elemento hijo. En la segunda, el segundo, y así sucesivamente, volviendo al principio después de ejecutar el último.
- Configuración clave: Exclude Sub Controllers: Si está marcada, solo intercala entre Samplers directamente dentro de él, ignorando los Samplers dentro de sub-controladores.
- Uso común: Cuando quieres que diferentes acciones se ejecuten en una secuencia rotativa por cada paso del usuario, simular un comportamiento donde los usuarios acceden a diferentes secciones en cada "clic" general.

© JMA 2020. All rights reserved

Throughput Controller (Controlador de Rendimiento)

- Propósito: Permite controlar la frecuencia con la que se ejecutan los elementos anidados, ya sea por un porcentaje de veces o por un número absoluto de ejecuciones.
- Comportamiento:
  - Percent Executions: Ejecuta los elementos anidados un cierto porcentaje de las veces (ej., "solo el 10% de los usuarios visitan la página de contacto").
  - Total Executions: Ejecuta los elementos anidados un número fijo de veces en total a lo largo de toda la prueba (útil para un número absoluto de ocurrencias).
- Uso común: Modelar escenarios donde solo un subconjunto de usuarios realiza ciertas acciones, o para limitar la carga en una parte específica de la aplicación.

© JMA 2020. All rights reserved

Random Controller (Controlador Aleatorio)

- Propósito: Ejecuta uno solo de sus elementos anidados de forma completamente aleatoria en cada iteración.
- Comportamiento: De todos los Samplers o Logic Controllers que contiene directamente como hijos, elige uno al azar para ejecutarlo en la iteración actual.
- Uso común: Simular un comportamiento de navegación más natural donde los usuarios eligen diferentes caminos aleatoriamente (ej., "visitar página A o página B o página C").

© JMA 2020. All rights reserved

Random Order Controller (Controlador de Orden Aleatorio)

- Propósito: Ejecuta todos sus elementos anidados, pero en un orden aleatorio y diferente en cada iteración.
- Comportamiento: A diferencia del Random Controller que elige solo uno, este ejecuta todos los hijos, pero los desordena aleatoriamente en cada paso.
- Uso común: Cuando necesitas que todas las acciones se realicen, pero el orden no es fijo o debe simular variaciones del usuario para evitar patrones de prueba predecibles.

© JMA 2020. All rights reserved

Runtime Controller (Controlador de Tiempo de Ejecución)

- Propósito: Ejecuta sus elementos anidados durante un período de tiempo especificado, en lugar de un número fijo de veces.
- Comportamiento: Los elementos dentro de este controlador se ejecutarán continuamente durante la "Runtime" (tiempo de ejecución) configurada. Si el tiempo se agota, el controlador detiene la ejecución de sus hijos y el hilo continúa con los elementos posteriores.
- Configuración clave: Runtime (seconds): La duración en segundos durante la cual los elementos anidados deben ejecutarse.
- Uso común: Para simular un período de actividad constante para una sección específica de tu script (ej., "navegar por el catálogo durante 5 minutos"), o cuando necesitas que una parte de la prueba se ejecute por una duración fija.

© JMA 2020. All rights reserved

Test Fragment

- El propósito fundamental de un Test Fragment es mejorar la:
  - Reutilización de código: Si tienes una secuencia de acciones (por ejemplo, "login", "añadir al carrito", "logout") que se repite en varias partes de tu prueba o en diferentes Thread Groups, puedes definirla una vez en un Test Fragment y luego "llamarlo" donde sea necesario.
  - Modularidad: Divide un Plan de Pruebas grande y complejo en módulos más pequeños y manejables. Esto facilita la depuración, el mantenimiento y la colaboración en proyectos grandes.
  - Legibilidad: Un Plan de Pruebas con Test Fragments bien definidos es más fácil de entender, ya que las secuencias de acciones complejas se encapsulan en un solo elemento.
  - Organización: Ayuda a mantener tu árbol de elementos JMeter más ordenado y menos anidado.
  - Dentro del Test Fragment, colocas los Samplers, Controladores Lógicos, Timers, Pre/Post-Procesadores, y Aserciones que componen la secuencia de acciones que quieres reutilizar.
- Dentro del Test Fragment, colocas los Samplers, Controladores Lógicos, Timers, Pre/Post-Procesadores, y Aserciones que componen la secuencia de acciones que quieres reutilizar.
- Para que el contenido de un Test Fragment se ejecute, debes "llamarlo" desde otro lugar en tu Plan de Pruebas. Hay dos formas principales de hacerlo: con un Module Controller o con Include Controller.

© JMA 2020. All rights reserved

Module Controller (Controlador de Módulos)

- Propósito: Permite la reutilización de partes de un script que están definidas en un Test Fragment o en otro Thread Group.
- Comportamiento: Cuando se ejecuta, el Module Controller llama y ejecuta los Samplers y controladores definidos en el módulo referenciado. Es como una "llamada a función" o "include" dentro de tu plan de pruebas.
- Configuración clave: Seleccionar el "Módulo" (el Test Fragment o Thread Group) al que se quiere hacer referencia.
- Uso común: Modularizar scripts grandes, promover la reutilización de código (ej., un módulo de login que se usa en varios Thread Groups), y facilitar el mantenimiento.

© JMA 2020. All rights reserved

Include Controller (Controlador de Inclusión)

- Propósito: Permite incluir y ejecutar partes de un Plan de Pruebas que están definidas en un archivo .jmx externo.
- Comportamiento: Actúa como un "puntero" a otro archivo .jmx. Cuando el Include Controller se ejecuta, JMeter carga y ejecuta los elementos del Thread Group o Test Fragment que se especifique en el archivo externo.
- Configuración clave: Path to Included Test Plan: La ruta al archivo .jmx que contiene el fragmento de prueba a incluir.
- Uso común:
  - Reutilización de scripts completos o fragmentos en múltiples proyectos o planes de pruebas.
  - Modularización de planes grandes en archivos más pequeños, mejorando la organización, la colaboración y el mantenimiento.

© JMA 2020. All rights reserved

ELEMENTOS DE CONFIGURACIÓN

© JMA 2020. All rights reserved

Elementos de configuración

- Los elementos de configuración se pueden usar para configurar valores predeterminados y variables para su uso posterior por parte de los muestreadores. Hay que tener en cuenta que estos elementos se procesan al inicio del alcance en el que se encuentran, es decir, antes de cualquier muestreador en el mismo alcance.
- Están disponibles configuradores para:
  - Valores predeterminados para peticiones HTTP, FTP, Java, LDAP
  - Configuración de conexión JDBC, del almacén de claves, de muestra de TCP, de inicio de sesión
  - Variables definidas por el usuario, aleatorias, contadores, conjuntos de datos CSV
  - Administrador de caché HTTP y DNS
  - Administrador de autorización, cookies y encabezados HTTP

© JMA 2020. All rights reserved

Configuration Elements

- CSV Data Set Config
- FTP Request Defaults
- DNS Cache Manager
- HTTP Authorization Manager
- HTTP Cache Manager
- HTTP Cookie Manager
- HTTP Request Defaults
- HTTP Header Manager
- Java Request Defaults
- JDBC Connection Configuration
- Keystore Configuration
- Login Config Element
- LDAP Request Defaults
- LDAP Extended Request Defaults
- TCP Sampler Config
- User Defined Variables
- Random Variable
- Counter
- Simple Config Element
- MongoDB Source Config (DEPRECATED)
- Bolt Connection Configuration

© JMA 2020. All rights reserved

Propiedades y variables

- Las propiedades son globales para jmeter, y se usan principalmente para definir algunos de los valores predeterminados que usa JMeter. Por ejemplo, la propiedad remote_hosts define los servidores que JMeter intentará ejecutar de forma remota.
- Se puede hacer referencia a las propiedades en los planes de prueba, pero no se puede usar para valores específicos de hilos. Las propiedades de JMeter se definen en los ficheros jmeter.properties y user.properties o en la línea de comandos.
- Las variables de JMeter son locales para cada hilo. Los valores pueden ser los mismos para cada hilo, o pueden ser diferentes. Si una variable es actualizada por un hilo, solo se cambia la copia de la variable en el hilo. Por ejemplo, el Postprocesador de Expresión Regular Extractor establecerá sus variables de acuerdo con la muestra que su hilo haya leído, y estas pueden usarse más adelante por el mismo hilo.
- Los valores definidos por el plan de prueba y el configurador de variables definidas por el usuario (UDV) están disponibles para todo el plan de prueba desde el inicio. Si la misma variable está definida por múltiples UDV, solo tendrá efecto la última.

© JMA 2020. All rights reserved

Propiedades y variables

- Una vez que un hilo ha comenzado, el conjunto inicial de variables se copia a cada hilo. Otros elementos, como el Preprocesador de parámetros de usuario o el Postprocesador de extracción de expresiones regulares, se pueden usar para redefinir las mismas variables (o crear nuevas). Estas redefiniciones solo se aplican al hilo actual.
- Las variables no tienen por que variar, se pueden definir una vez y, si se dejan solas, no cambiarán de valor. Por lo tanto, se pueden usar como abreviatura para expresiones que aparecen con frecuencia en un plan de prueba o para los elementos que son constantes durante una ejecución pero que pueden variar entre ejecuciones. Por ejemplo, el nombre de un host.
- Tanto las variables como las propiedades distinguen entre mayúsculas y minúsculas.

© JMA 2020. All rights reserved

Funciones

- Las funciones de JMeter son valores calculados que pueden rellenar los campos de cualquier muestreador u otro elemento en un árbol de prueba.
- Hay dos tipos de funciones: valores estáticos definidos por el usuario (o variables) y funciones incorporadas.
- Los valores estáticos definidos por el usuario permiten al usuario definir las variables que se reemplazarán con su valor estático cuando se compila un árbol de prueba y se envía para que se ejecute. Este reemplazo ocurre una vez al comienzo de la prueba.
- Con las funciones integradas, se pueden calcular nuevos valores en tiempo de ejecución dependiendo de los datos de respuesta anteriores, en qué hilo se encuentra la función, la hora y muchas otras fuentes. Se generan nuevos valores para cada solicitud a lo largo del curso de la prueba.

© JMA 2020. All rights reserved

Funciones

- Las funciones y variables se pueden escribir en cualquier campo de cualquier componente de prueba.
- Para invocar las funciones:
  ${__functionName (var1, var2, var3)}
- Si un parámetro de función contiene una coma, entonces asegúrese de escapar con " \ ", de lo contrario JMeter lo tratará como un delimitador de parámetros. Las funciones que no requieren parámetros pueden omitir los paréntesis.
- Las variables se referencian de la siguiente manera:
  ${VARIABLE}
- Las propiedades deben ser referenciadas utilizando la función __P o __property:
  ${__property(user.dir)}
- Si se hace referencia a una función o variable no definida, JMeter NO informa/registra un error: la referencia se devuelve sin cambios.

© JMA 2020. All rights reserved

Orden de ejecución

1. Elementos de configuración
2. Preprocesadores
3. Temporizadores
4. Controladores
5. Postprocesadores (si SampleResult no es nulo)
6. Aserciones (si SampleResult no es nulo)
7. Receptores (si SampleResult no es nulo)

   – Los temporizadores, las aserciones, los preprocesadores y los postprocesadores solo se procesan si hay un muestreador al que aplicarlo.
   – Los controladores lógicos y los muestreadores se procesan en el orden en que aparecen en el árbol.
   – Otros elementos de prueba se procesan de acuerdo con el alcance en el que se encuentran y el tipo de elemento de prueba.

© JMA 2020. All rights reserved

Reglas de alcance

- El árbol de la prueba JMeter contiene elementos que son jerárquicos y están ordenados. Algunos elementos en los árboles de prueba son estrictamente jerárquicos (receptores, elementos de configuración, postprocesadores, preprocesadores, aserciones, temporizadores) y algunos están ordenados por principio (controladores, muestreadores).
- Cuando se crea el plan de prueba, se creará una lista ordenada de solicitudes de muestra (a través de muestreadores) que representa un conjunto de pasos a ejecutar. Estas solicitudes a menudo se organizan dentro de los controladores que también se ordenan.
- Otros elementos son jerárquicos. Una aserción, por ejemplo, es jerárquica en el árbol de prueba. Si su padre es una solicitud, entonces se aplica a esa solicitud. Si su padre es un controlador, afecta a todas las solicitudes que contiene.

© JMA 2020. All rights reserved

User Defined Variables (Variables Definidas por el Usuario)

- Propósito: Definir un conjunto de variables con valores fijos al inicio del Plan de Pruebas o de un Grupo de Hilos.
- Comportamiento: Las variables se evalúan una sola vez al comienzo del ámbito donde se colocan (Plan de Pruebas, Grupo de Hilos, etc.). El valor de la variable permanece constante durante toda la ejecución del test para ese hilo específico. Es decir, si un hilo entra a un Thread Group que tiene User Defined Variables, esas variables se inicializan una vez y conservan sus valores para todas las iteraciones de ese hilo.
- Configuración Clave:
  - Tabla con dos columnas: Name (Nombre de la variable) y Value (Valor de la variable).
- Uso Común:
  - Centralizar parámetros de configuración: Definir la URL base de tu aplicación, el puerto, o cualquier valor constante que se use en múltiples lugares del script. Si cambian, solo necesitas modificar un lugar.
  - Valores de entorno: Definir variables específicas para diferentes entornos de prueba (ej., QA_URL, PROD_URL).
  - Valores de sesión iniciales: Datos que no cambian por iteración, como el tipo de navegador simulado.
- Diferencias/Notas Importantes:
  - Son estáticas y se evalúan una sola vez. No son adecuadas para generar valores que cambien en cada iteración del bucle. Para eso, usarías CSV Data Set Config, Random Variable o Counter.
  - Son útiles para variables globales o de grupo que no necesitan cambiar dinámicamente.

© JMA 2020. All rights reserved

Random Variable (Variable Aleatoria)

- Propósito: Generar un valor numérico aleatorio dentro de un rango especificado, que puede cambiar con cada iteración o con cada hilo.
- Comportamiento: Cada vez que el Random Variable se ejecuta (lo cual depende de su alcance), genera un nuevo número aleatorio. Si se coloca en un Thread Group, se generará un nuevo valor para cada iteración de cada hilo.
- Configuración Clave:
  - Variable Name (Nombre de la Variable): El nombre de la variable de JMeter donde se almacenará el valor aleatorio (ej., randomId).
  - Output Format (Formato de Salida): (Opcional) Define el formato numérico (ej., 000 para tener 3 dígitos, user_00 para prefijar).
  - Minimum Value (Valor Mínimo): El límite inferior del rango para el número aleatorio.
  - Maximum Value (Valor Máximo): El límite superior del rango para el número aleatorio.
  - Seed (Semilla): (Opcional) Un valor para inicializar el generador de números aleatorios. Si usas la misma semilla, la secuencia de números aleatorios será la misma en cada ejecución, lo cual es útil para reproducibilidad en depuración. Si está vacío, usa una semilla aleatoria (más común para pruebas de carga reales).
- Uso Común:
  - Generar IDs de usuario/producto aleatorios: Simular usuarios que acceden a diferentes IDs dentro de un rango.
  - Valores numéricos aleatorios para campos de formulario: Ej., edades, cantidades, calificaciones.
  - Simular navegación aleatoria: Combinado con un Switch Controller o If Controller, puedes generar un número aleatorio para elegir una de varias rutas posibles.
- Diferencias/Notas Importantes:
  - Ideal para valores numéricos aleatorios. Si necesitas valores aleatorios de una lista predefinida (ej., nombres, tipos de producto), el CSV Data Set Config con un archivo de una sola columna y el Sharing Mode adecuado puede ser más flexible, o usar funciones __RandomFromList().

© JMA 2020. All rights reserved

Counter (Contador)

- Propósito: Generar un valor numérico incremental con cada iteración de un hilo o a nivel global para todos los hilos.
- Comportamiento: Mantiene un contador interno que se incrementa en un valor fijo (Increment) cada vez que se le "llama". Puedes configurarlo para que sea global (para todos los hilos) o local (por hilo). También puedes definir un valor inicial y un valor máximo.
- Configuración Clave:
  - Start (Inicio): El valor inicial del contador.
  - Increment (Incremento): La cantidad en la que se incrementa el contador en cada paso.
  - Maximum (Máximo): (Opcional) El valor máximo que alcanzará el contador. Si se supera, el contador se reinicia (si Track Counter Independently for each User no está marcado y Per User es falso) o se mantiene en el máximo.
  - Variable Name (Nombre de la Variable): El nombre de la variable de JMeter donde se almacenará el valor actual del contador (ej., userId).
  - Track Counter Independently for each User (Seguimiento del contador independientemente para cada usuario):
    - Marcado (True): Cada hilo tendrá su propio contador que empieza en Start y se incrementa de forma independiente.
    - No marcado (False): Habrá un único contador global que todos los hilos comparten y se incrementa con cada acceso al Counter.
  - Reset on each Thread Group Iteration (Reiniciar en cada iteración del grupo de hilos): Si está marcado y el contador es independiente por usuario, se reinicia para cada iteración de su Thread Group padre.
- Uso Común:
  - Generar IDs únicos y secuenciales: Para crear usuarios de prueba, IDs de pedido, o cualquier dato que necesite ser único y seguir una secuencia.
  - Simular IDs de usuario logueados: Cada hilo puede tomar un userId único (usando el modo "independiente para cada usuario").
  - Controlar el número de iteraciones o un índice: Aunque el Loop Controller es más común para esto, un Counter global puede ser útil para tareas de setup/teardown.
- Diferencias/Notas Importantes:
  - Asegúrate de entender la diferencia entre "independiente por usuario" y el contador global, ya que afecta significativamente el comportamiento y la unicidad de los valores.
  - Ideal para datos secuenciales y únicos.

© JMA 2020. All rights reserved

CSV Data Set Config (Configuración de Conjunto de Datos CSV)

- Propósito: Leer líneas de datos desde un archivo de texto plano (generalmente CSV) y asignarlas a variables de JMeter. Cada línea del archivo representa un conjunto de datos que puede ser utilizado por un hilo o por varias iteraciones.
- Comportamiento:
  - Este elemento lee el archivo CSV especificado y asigna los valores de cada columna a las variables de JMeter que definas.
  - En cada nueva iteración (o por cada nuevo hilo, dependiendo de la configuración), se lee la siguiente línea del archivo.
  - Cuando se llega al final del archivo, puedes configurar si se reinicia desde el principio (Recycle on EOF) o si la prueba debe detenerse (Stop on EOF).

© JMA 2020. All rights reserved

CSV Data Set Config (Configuración de Conjunto de Datos CSV)

- Configuración Clave:
  - Filename (Nombre de archivo): La ruta completa (o relativa al bin de JMeter) del archivo .csv (o .txt) que contiene tus datos.
  - File encoding (Codificación del archivo): La codificación de caracteres del archivo (ej., UTF-8, ISO-8859-1).
  - Variable Names (comma-delimited) (Nombres de variables (separados por comas)): Una lista separada por comas de los nombres de las variables de JMeter a las que se asignarán los valores de cada columna del CSV. El orden debe coincidir con el orden de las columnas en el archivo. Ejemplo: Si tu CSV tiene username,password, aquí escribirías usuario,clave.
  - Delimiter (Delimitador): El carácter que separa las columnas en tu archivo CSV (\t para tabulador , ;).
  - Recycle on EOF? (¿Reciclar al final del archivo?):
    - True: Cuando todos los datos del archivo se han leído, el lector de archivos vuelve al principio y reutiliza los datos.
    - False: El lector se detiene al final del archivo.
  - Stop on EOF? (¿Detener al final del archivo?):
    - True: Si se llega al final del archivo, el hilo actual se detiene (o la prueba completa si todos los hilos han terminado). Es útil para pruebas funcionales con un conjunto de datos finito.
    - False: Si se llega al final del archivo, si Recycle on EOF? es True, el hilo continuará, sino el hilo obtendrá valores null o vacíos de las variables CSV.
  - Sharing Mode (Modo de Compartición): ¡Muy importante! Define cómo se comparten los datos del archivo entre los hilos:
    - All threads (Todos los hilos): Por defecto. Hay un único puntero al archivo para todos los hilos. Cada hilo que necesita una línea obtiene la siguiente línea del archivo. Esto asegura que cada hilo obtenga una línea única hasta que el archivo se agote.
    - Current thread group (Grupo de hilos actual): Un puntero al archivo por cada Thread Group. Los hilos dentro del mismo grupo de hilos comparten el puntero.
    - Current thread (Hilo actual): Cada hilo tiene su propio puntero al archivo. Esto significa que cada hilo lee el archivo desde el principio, obteniendo los mismos datos en las mismas iteraciones si tienen la misma secuencia de ejecución.
    - Edit (Editar): Permite definir un modo de compartición personalizado.

© JMA 2020. All rights reserved

CSV Data Set Config (Configuración de Conjunto de Datos CSV)

- Uso Común:
  - Autenticación con múltiples usuarios: Proporcionar diferentes nombres de usuario y contraseñas para simular múltiples usuarios loggeándose.
  - Datos de formulario dinámicos: Rellenar campos de formulario con diferentes valores (ej., direcciones, números de teléfono, productos a buscar).
  - Datos de búsqueda: Proporcionar términos de búsqueda o IDs de elementos para consultas.
  - Conjuntos de datos grandes y complejos: Cuando necesitas una gran cantidad de datos estructurados que no se pueden generar fácilmente de forma aleatoria o secuencial.
- Diferencias/Notas Importantes con otros Elementos de Configuración de Datos:
  - vs. User Defined Variables: CSV Data Set Config proporciona valores variables por iteración/hilo desde un archivo externo, mientras que User Defined Variables son estáticas y se inicializan una sola vez.
  - vs. Random Variable / Counter: CSV Data Set Config utiliza datos predefinidos que tú proporcionas en un archivo. Random Variable y Counter generan datos de forma aleatoria o secuencial respectivamente, sin necesidad de un archivo externo.
  - Control del flujo: Stop on EOF? es una característica poderosa para detener la prueba una vez que se han utilizado todos los datos únicos.
  - Consideraciones de rendimiento: Para archivos CSV muy grandes, JMeter puede consumir más memoria para precargarlos o tener una sobrecarga de E/S al leerlos. El Sharing Mode es crucial para evitar condiciones de carrera o que los hilos obtengan los mismos datos de forma incorrecta.

© JMA 2020. All rights reserved

TEMPORIZADORES

© JMA 2020. All rights reserved

Temporizadores

- De forma predeterminada, un hilo de JMeter ejecuta los muestreadores en secuencia sin pausa. Es recomendable especificar un retraso agregando uno de los temporizadores disponibles al grupo de subprocesos, si no, JMeter podría saturar al servidor al realizar demasiadas solicitudes en un período de tiempo muy corto.
- Un temporizador hará que JMeter demore un cierto tiempo antes de cada muestra que esté dentro de su alcance.
- Si se agrega más de un temporizador a un grupo de subprocesos, JMeter toma la suma de los temporizadores y se detiene durante ese tiempo antes de ejecutar los muestreadores a los que se aplican los temporizadores. Los temporizadores se pueden agregar como hijos de muestreadores o controladores para restringir a los que se aplican.
- Para proporcionar una pausa en un solo lugar en un plan de prueba, se puede usar el Muestreador de acciones de prueba.

© JMA 2020. All rights reserved

Timers

- Constant Timer
- Gaussian Random Timer
- Uniform Random Timer
- Constant Throughput Timer
- Precise Throughput Timer
- Synchronizing Timer
- BeanShell Timer
- JSR223 Timer
- Poisson Random Timer

© JMA 2020. All rights reserved

Constant Timer (Temporizador Constante)

- Propósito: Pausar cada hilo por una cantidad de tiempo fija y predeterminada antes de ejecutar la siguiente solicitud.
- Comportamiento: Cada usuario virtual espera la duración especificada (Delay) antes de enviar la siguiente petición.
- Configuración Clave:
  - Delay (in milliseconds) (Retraso en milisegundos).
- Uso Común:
  - Simular un tiempo de pensamiento uniforme y predecible.
  - Cuando necesitas un control muy estricto sobre el intervalo entre peticiones.

© JMA 2020. All rights reserved

Gaussian Random Timer (Temporizador Aleatorio Gaussiano)

- Propósito: Pausar cada hilo por una cantidad de tiempo aleatoria que sigue una distribución normal (Gaussiana). Esto significa que la mayoría de las pausas estarán cerca del promedio, con menos pausas en los extremos.
- Comportamiento: Calcula un retraso que se distribuye alrededor de una media (Constant Delay Offset) con una cierta desviación estándar (Deviation).
- Configuración Clave:
  - Deviation (in milliseconds): La desviación estándar del tiempo de espera.
  - Constant Delay Offset (in milliseconds): El valor central alrededor del cual se agruparán los retrasos.
- Uso Común:
  - Simular un comportamiento de usuario más realista, ya que los tiempos de pensamiento de las personas no son perfectamente uniformes.
  - Cuando se sabe que los tiempos de pensamiento se distribuyen normalmente en tu aplicación.

© JMA 2020. All rights reserved

Poisson Random Timer (Temporizador Aleatorio de Poisson)

- 9.
- Propósito: Pausar cada hilo por una cantidad de tiempo aleatoria que sigue una distribución de Poisson. Esta distribución es útil para modelar la ocurrencia de eventos independientes en un intervalo fijo, como la llegada de clientes a un servicio.
- Comportamiento: Genera retrasos que tienden a agruparse alrededor de una media, pero con una "cola" más larga hacia tiempos de espera más largos que la distribución Gaussiana. Esto puede simular eventos "raros" o impredecibles.
- Configuración Clave:
  - Lambda (in milliseconds): El parámetro Lambda de la distribución de Poisson, que se relaciona con el valor promedio esperado del retraso.
  - Delay Offset (in milliseconds): Un valor fijo que se añade al retraso generado por la distribución de Poisson.
- Uso Común:
  - Simular un comportamiento de usuario donde las acciones ocurren de forma impredecible y no uniforme.
  - Modelar eventos que ocurren a una cierta "tasa media" pero con variaciones significativas (ej., el tiempo entre clics de un usuario cuando navega libremente).

© JMA 2020. All rights reserved

Uniform Random Timer (Temporizador Aleatorio Uniforme)

- Propósito: Pausar cada hilo por una cantidad de tiempo aleatoria dentro de un rango específico, donde cualquier valor dentro de ese rango tiene la misma probabilidad de ocurrir.
- Comportamiento: Genera un retraso aleatorio entre un valor mínimo (Random Delay Maximum) y un valor basado en un desplazamiento constante (Constant Delay Offset).
- Configuración Clave:
  - Random Delay Maximum (in milliseconds): El valor máximo del componente aleatorio del retraso.
  - Constant Delay Offset (in milliseconds): Un valor fijo que se añade al retraso aleatorio, estableciendo el mínimo efectivo.
    - Nota: El retraso final será entre Constant Delay Offset y (Constant Delay Offset + Random Delay Maximum).
- Uso Común:
  - Simular un tiempo de pensamiento con una variabilidad predecible pero dentro de límites claros.
  - Cuando quieres una distribución uniforme de los tiempos de espera.

© JMA 2020. All rights reserved

Constant Throughput Timer (Temporizador de Rendimiento Constante)

- Propósito: Intentar mantener un rendimiento (throughput) constante y objetivo de solicitudes por minuto, independientemente de la velocidad del servidor o la capacidad de JMeter.
- Comportamiento: Calcula dinámicamente el retraso necesario para alcanzar el Target Throughput especificado. Si el servidor es lento o JMeter no puede generar la carga, este Timer esperará menos o nada; si el servidor es rápido, esperará más para no sobrepasar el objetivo.
- Configuración Clave:
  - Target Throughput (in samples per minute): El número deseado de Samplers por minuto que quieres que se ejecuten.
  - Calculate Throughput based on: Define cómo se calcula el throughput (ej., this thread only, all active threads in current thread group, all active threads).
- Uso Común:
  - Controlar la carga que llega al servidor para que sea constante y medible.
  - Asegurarse de que el servidor reciba un número específico de transacciones por unidad de tiempo.
  - Estudios de capacidad donde se busca la capacidad máxima del sistema.

© JMA 2020. All rights reserved

Precise Throughput Timer (Temporizador de Rendimiento Preciso)

- Propósito: Una versión más avanzada y precisa del Constant Throughput Timer, diseñada para manejar mejor las distribuciones de carga y la concurrencia, especialmente con muchos hilos.
- Comportamiento: Ofrece un algoritmo de distribución de carga más sofisticado que el Constant Throughput Timer. Minimiza la deriva de la tasa de aciertos y es más robusto bajo diferentes escenarios de carga y variabilidad del tiempo de respuesta del servidor.
- Configuración Clave: Más opciones que el Constant Throughput Timer, incluyendo:
  - Target Throughput (requests/sec): Rendimiento objetivo en solicitudes por segundo.
  - Throughput Period: El período durante el cual se distribuirá el rendimiento.
  - Test Duration: La duración total de la prueba para la cual se calculará el throughput.
  - Batch Size: Cuántas muestras se procesan en un "lote".
- Uso Común:
  - Control de carga de alta precisión en escenarios de pruebas de rendimiento a gran escala.
  - Cuando el Constant Throughput Timer no proporciona la precisión deseada.
  - Necesitas una tasa de inyección de carga muy estable.

© JMA 2020. All rights reserved

Synchronizing Timer (Temporizador Sincronizador)

- Propósito: Actúa como un punto de encuentro (rendezvous point) para un número específico de hilos. Bloquea a los hilos hasta que el número configurado de hilos haya llegado, y luego los libera todos simultáneamente.
- Comportamiento: Los hilos que llegan al Synchronizing Timer esperan en una "puerta" hasta que un Group Size (tamaño del grupo) de hilos esté presente. Una vez que se alcanza el tamaño del grupo, todos esos hilos se liberan al mismo tiempo.
- Configuración Clave: Group Size (number of threads to group by): El número de hilos que deben acumularse antes de que se liberen todos.
- Uso Común:
  - Simular un pico repentino de actividad de usuario (ej., el momento exacto en que se abren las inscripciones para un evento popular, un "flash sale").
  - Pruebas de concurrencia donde necesitas que un gran número de usuarios golpeen una función específica al mismo tiempo.

© JMA 2020. All rights reserved

BeanShell Timer (Temporizador BeanShell)

- Propósito: Permite ejecutar código de scripting en el lenguaje BeanShell para calcular el retraso de un hilo.
- Comportamiento: JMeter ejecuta el script BeanShell que escribes. El valor numérico (en milisegundos) devuelto por el script se utiliza como el tiempo de pausa. Proporciona acceso a variables de JMeter y al contexto.
- Configuración Clave: Script: El área de texto donde escribes tu código BeanShell.
- Uso Común:
  - Cuando necesitas un retraso dinámico y complejo que depende de variables o lógica de negocio.
  - Para retrasos que se calculan basándose en los resultados de Samplers previos.
- Importante: Similar al BeanShell Sampler, este Timer es más lento que el JSR223 Timer con Groovy. Para nuevos desarrollos o para un rendimiento óptimo, se prefiere el JSR223 Timer.

© JMA 2020. All rights reserved

JSR223 Timer (Temporizador JSR223)

- Propósito: Permite ejecutar código de scripting en cualquier lenguaje compatible con JSR223 (principalmente Groovy) para calcular el retraso. Es el reemplazo preferido y de alto rendimiento para el BeanShell Timer.
- Comportamiento: El script JSR223 se ejecuta, y el valor numérico (en milisegundos) que devuelve se utiliza como el tiempo de pausa. Ofrece un rendimiento significativamente mejor que BeanShell y acceso completo a las APIs de Java y al contexto de JMeter.
- Configuración Clave:
  - Language: Selecciona el lenguaje (se recomienda Groovy).
  - Script File: Carga el script desde un archivo externo (opcional).
  - Script: El área de texto para tu código.
- Uso Común: Para cualquier escenario que requiera un retraso dinámico, complejo y de alto rendimiento, basado en la lógica de negocio, variables, o cálculos personalizados.

© JMA 2020. All rights reserved

ASERCIONES

© JMA 2020. All rights reserved

Aserciones

- Las aserciones permiten afirmar hechos sobre las respuestas recibidas del servidor que se está probando, en caso de no cumplirse JMeter lo marcará como una solicitud fallida. Permiten comprobar que la aplicación está devolviendo los resultados esperados.
- Se pueden establecer aserciones sobre el contenido, tamaño y duración de las respuestas. Hay disponibles aserciones especificas para determinados muestreadores.
- Las aserciones se aplican a todos los muestreadores que se encuentran en su alcance. Para restringir una aserción a un solo muestreador se le debe agregar como elemento secundario.
- Para ver los resultados de la aserción, se agrega un receptor de resultado de aserción. Las aserciones fallidas también se mostrarán en la vista de árbol y en los receptores de la tabla, y contarán para el % de error en los informes de agregados y resumen.

© JMA 2020. All rights reserved

Assertions

- Response Assertion
- Duration Assertion
- Size Assertion
- XML Assertion
- BeanShell Assertion
- MD5Hex Assertion
- HTML Assertion
- XPath Assertion
- XPath2 Assertion
- XML Schema Assertion
- JSR223 Assertion
- Compare Assertion
- SMIME Assertion
- JSON Assertion
- JSON JMESPath Assertion

© JMA 2020. All rights reserved

Response Assertion (Aserción de Respuesta)

- Propósito: La aserción más utilizada y versátil. Permite validar el contenido de la respuesta (cuerpo, cabeceras, código de respuesta, mensaje de respuesta, URL, etc.) contra patrones definidos.
- Comportamiento: Compara una parte específica de la respuesta con uno o más patrones de prueba utilizando expresiones regulares, cadenas literales o patrones inversos.
- Configuración clave:
  - Apply to: Dónde aplicar la aserción (Main sample only, Sub-samples, Both, etc.).
  - Fields to Test: La parte de la respuesta a evaluar (ej., Text Response para el cuerpo, Response Code, Response Headers, URL Sampled, Response Message).
  - Pattern Matching Rules: Cómo coincidir (ej., Contains, Matches, Equals, Starts With, Substring, Not).
  - Patterns to Test: Los patrones (cadenas, expresiones regulares) que la aserción buscará en la respuesta.
- Uso común:
  - Verificar que una página contiene un texto esperado (ej., "Bienvenido", "Producto añadido").
  - Asegurarse de que el código de respuesta HTTP sea 200 (OK) o 302 (Redirección).
  - Confirmar que las cabeceras de respuesta contienen valores específicos.

© JMA 2020. All rights reserved

Duration Assertion (Aserción de Duración)

- Propósito: Verificar que el tiempo de respuesta de un Sampler no exceda una duración máxima predefinida.
- Comportamiento: Mide el tiempo que tarda el Sampler en recibir la respuesta completa del servidor y lo compara con el Duration configurado. Si el tiempo de respuesta es mayor, la aserción falla.
- Configuración clave: Duration (milliseconds): El tiempo máximo de respuesta permitido en milisegundos.
- Uso común:
  - Establecer umbrales de rendimiento para solicitudes críticas (ej., "el login debe responder en menos de 500 ms").
  - Identificar cuellos de botella cuando el servidor tarda demasiado en responder.

© JMA 2020. All rights reserved

Size Assertion (Aserción de Tamaño)

- Propósito: Verificar que el tamaño de la respuesta (en bytes) esté dentro de un rango esperado.
- Comportamiento: Mide el tamaño de la respuesta recibida y lo compara con el tamaño (Size) y el tipo de comparación (Type of Comparison) configurados.
- Configuración clave:
  - Size (bytes): El tamaño de referencia en bytes.
  - Type of Comparison: Cómo comparar el tamaño (ej., Equals, Not Equals, Greater Than, Less Than, Greater Than or Equals, Less Than or Equals).
- Uso común:
  - Detectar errores donde las respuestas son inesperadamente grandes o pequeñas (ej., un archivo descargado que es demasiado pequeño, una página que devuelve una respuesta vacía cuando no debería).
  - Validar la integridad de los datos o el contenido de las respuestas.

© JMA 2020. All rights reserved

Compare Assertion (Aserción de Comparación)

- Propósito: Comparar la respuesta actual con una respuesta de referencia (guardada previamente en un archivo o con la respuesta de una muestra anterior).
- Comportamiento: Requiere una respuesta de referencia. Puede comparar el contenido, las cabeceras o los códigos de respuesta. Es útil para asegurar que los cambios en el sistema no alteran respuestas que deberían permanecer constantes.
- Configuración clave:
  - Compare content: Comparar el cuerpo de la respuesta.
  - Compare response headers: Comparar las cabeceras de respuesta.
  - Compare response code: Comparar el código de respuesta.
  - Compare response message: Comparar el mensaje de respuesta.
  - Compare response time: Comparar los tiempos de respuesta.
  - File to compare: La ruta a un archivo que contiene la respuesta de referencia.
  - Save First Sample to File: Permite guardar la primera respuesta para usarla como referencia.
- Uso común:
  - Pruebas de regresión: Asegurarse de que un cambio en el código no haya alterado una parte específica de la respuesta que debería ser la misma.
  - Validar la consistencia de los datos en respuestas repetitivas.

© JMA 2020. All rights reserved

MD5Hex Assertion (Aserción MD5Hex)

- Propósito: Verificar que el hash MD5 del cuerpo de la respuesta coincide con un valor esperado.
- Comportamiento: Calcula el hash MD5 del cuerpo de la respuesta y lo compara con el hash MD5 configurado. Si no coinciden, la aserción falla.
- Configuración clave: MD5Hex: El valor hexadecimal del hash MD5 esperado.
- Uso común:
  - Validar la integridad de archivos descargados o de grandes volúmenes de datos.
  - Asegurarse de que el contenido de una respuesta no ha sido alterado.
  - Útil para verificar que un recurso estático (ej., una imagen grande) es exactamente el que se espera.

© JMA 2020. All rights reserved

HTML Assertion (Aserción HTML)

- Propósito: Verificar la sintaxis y el cumplimiento de estándares del HTML en la respuesta.
- Comportamiento: Utiliza un parser HTML (basado en JTidy o NekoHTML) para analizar el cuerpo de la respuesta HTML. Puedes configurar umbrales para el número de errores, advertencias o errores fatales. Si el HTML no es válido o supera los umbrales configurados, la aserción falla.
- Configuración clave:
  - Doctype: El tipo de documento HTML a validar (ej., Strict HTML 4.01, XHTML 1.0 Transitional).
  - Errors/Warnings/Fatals thresholds: El número máximo permitido de cada tipo de problema.
  - Show warnings/errors: Mostrar los detalles de los problemas.
- Uso común:
  - Asegurar la calidad del código HTML generado por la aplicación bajo carga.
  - Detectar errores de generación de HTML que podrían afectar la representación en el navegador o el SEO.
  - Verificar que la aplicación no está generando HTML inválido o malformado bajo estrés.

© JMA 2020. All rights reserved

JSON Assertion (Aserción JSON)

- Propósito: Validar el contenido de una respuesta que está en formato JSON, permitiéndote verificar la presencia de rutas JSON específicas y sus valores.
- Comportamiento: Permite especificar una "JSON Path Expression" (expresión de ruta JSON) y un valor esperado. La aserción verifica si la ruta existe en el JSON de la respuesta y, opcionalmente, si su valor coincide con el patrón esperado.
- Configuración clave:
  - JSON Path Expression: La ruta JSON al elemento que quieres verificar (ej., $.data.user.name, $..productId).
  - Validation:
    - Assert JSON Path exists: Solo verifica si la ruta JSON existe en la respuesta.
    - Expected Value: El valor exacto que esperas que tenga el elemento en esa ruta.
  - Additionally check expected value: Si marcas esta opción, el valor del elemento en la ruta JSON DEBE coincidir con el Expected Value.
  - Match as regular expression: Si el Expected Value debe interpretarse como una expresión regular.
  - Expect null: Si esperas que el valor en la ruta JSON sea null.
  - Invert assertion (Not): Invertir el resultado (falla si la condición se cumple).
- Uso común:
  - Asegurarse de que ciertos campos JSON estén presentes en una respuesta de API REST.
  - Validar que el valor de un campo JSON específico es el esperado (ej., $.status es "success").
  - Verificar que un array JSON contiene al menos un elemento ($.items[0]).

© JMA 2020. All rights reserved

JSON JMESPath Assertion (Aserción JSON JMESPath)

- Propósito: Una aserción más poderosa y flexible para validar respuestas JSON, utilizando el lenguaje de consulta JMESPath. JMESPath es más avanzado que JSONPath para realizar transformaciones y filtros complejos.
- Comportamiento: Evalúa una expresión JMESPath sobre la respuesta JSON. Puedes especificar un valor esperado para la salida de esa expresión o simplemente verificar que la expresión produce algún resultado.
- Configuración clave:
  - JMESPath Expression: La expresión JMESPath a evaluar (ej., users[?age > '30'].name).
  - Expected value: El valor que esperas que la expresión JMESPath devuelva.
  - Assert that JSON is valid: (Opcional) Verifica que la respuesta es un JSON válido antes de intentar la aserción JMESPath.
  - Validate against expected value: Si se marca, el resultado de la expresión JMESPath debe coincidir con el Expected value.
  - Match as regular expression: Si el Expected value es una expresión regular.
  - Expect null: Si esperas que el resultado de la expresión sea null.
  - Invert assertion (Not): Invertir el resultado.
- Uso común:
  - Realizar validaciones más complejas en JSON que requieren filtrado, transformación o selección de múltiples elementos (ej., "asegúrate de que al menos un usuario en la lista tenga el rol de 'admin'").
  - Cuando las capacidades de JSON Assertion son insuficientes para tus necesidades de validación.

© JMA 2020. All rights reserved

XML Assertion (Aserción XML)

- Propósito: Validar que el cuerpo de la respuesta es un XML bien formado y, opcionalmente, verificar el contenido utilizando XPath.
- Comportamiento: Primero, valida que la respuesta es un documento XML sintácticamente correcto. Luego, opcionalmente, puedes añadir expresiones XPath para verificar la presencia de nodos o sus valores.
- Configuración clave:
  - XML validation: Si se debe validar que el XML es bien formado.
  - XPath NameSpaces (Tabla): Para definir prefijos de namespaces si tu XML los usa.
  - Validation Checkbox (por cada XPath): Marca para activar la validación con XPath.
  - XPath expression: La expresión XPath a evaluar (ej., //user/name, /catalog/book[price > 50]).
  - Expected value: El valor esperado del nodo XPath.
- Uso común:
  - Asegurarse de que las respuestas de servicios web SOAP o APIs que devuelven XML son válidas y están estructuradas correctamente.
  - Verificar la existencia de elementos XML o sus valores específicos.

© JMA 2020. All rights reserved

XPath Assertion (Aserción XPath)

- Propósito: Validar el contenido de una respuesta XML o XHTML utilizando expresiones XPath. Es más simple que XML Assertion si solo necesitas XPath sin validación XML completa.
- Comportamiento: Toma una expresión XPath y verifica si esa expresión encuentra un nodo en el documento XML/XHTML de la respuesta. Puedes verificar la existencia del nodo o que su valor coincida con un patrón.
- Configuración clave:
  - XPath expression: La expresión XPath (ej., //book[@id='bk101']/author).
  - Match as regular expression: Si el valor esperado debe interpretarse como una expresión regular.
  - Test content: Si se debe validar el contenido del nodo encontrado.
  - Expected value: El valor esperado del nodo.
  - Validate XML: (Opcional) Si también se debe validar que el XML es bien formado.
  - Namespace (Tabla): Para definir namespaces.
- Uso común:
  - Verificar la existencia de elementos específicos en respuestas XML o HTML (si es XHTML bien formado).
  - Extraer y validar el texto de nodos XML.

© JMA 2020. All rights reserved

XPath2 Assertion (Aserción XPath2)

- Propósito: Similar a XPath Assertion, pero soporta la especificación XPath 2.0 y XSLT 2.0, ofreciendo más funcionalidades y un manejo de tipos de datos más robusto.
- Comportamiento: Evalúa expresiones XPath 2.0 sobre el XML de la respuesta.
- Configuración clave: Muy similar a XPath Assertion, pero con la capacidad de usar la sintaxis y las funciones de XPath 2.0 (ej., doc(), funciones para fechas/horas, secuencias).
- Uso común:
  - Cuando tus validaciones XML son más complejas y requieren las capacidades avanzadas de XPath 2.0.
  - Trabajar con documentos XML más modernos o esquemas complejos.

© JMA 2020. All rights reserved

XML Schema Assertion (Aserción de Esquema XML)

- Propósito: Validar un documento XML contra un Esquema XML (XSD) para asegurar que el XML cumple con la estructura y las reglas definidas por el esquema.
- Comportamiento: Necesitas proporcionar la ruta a un archivo XSD. La aserción entonces verifica si el XML de la respuesta es válido de acuerdo con ese esquema.
- Configuración clave:
  - Schema files (.xsd): La ruta a tu archivo XSD (puedes añadir múltiples si el XML importa otros esquemas).
- Uso común:
  - Asegurar la conformidad de las respuestas XML con las especificaciones de un contrato de servicio (ej., en servicios web SOAP).
  - Verificar la integridad y validez estructural de los mensajes XML.

© JMA 2020. All rights reserved

SMIME Assertion (Aserción SMIME)

- Propósito: Validar mensajes de correo electrónico firmados o cifrados utilizando el estándar S/MIME (Secure/Multipurpose Internet Mail Extensions).
- Comportamiento: Trabaja con respuestas que contienen mensajes S/MIME. Puede verificar la firma digital, el cifrado y la cadena de certificados. Requiere que JMeter tenga acceso a los certificados y claves necesarios.
- Configuración clave: Relacionadas con la verificación de firmas, certificados, etc.
- Uso común:
  - Pruebas de seguridad y funcionalidad en sistemas que manejan correos electrónicos firmados o cifrados (ej., sistemas de mensajería seguros, plataformas de e-commerce que envían recibos cifrados).
  - Asegurar que los mensajes S/MIME son válidos y confiables bajo carga.

© JMA 2020. All rights reserved

JSR223 Assertion (Aserción JSR223)

- Propósito: La aserción más flexible y potente. Permite escribir código de scripting personalizado (preferiblemente Groovy) para realizar cualquier tipo de validación en la respuesta.
- Comportamiento: Ejecuta el script que escribas. El script tiene acceso al objeto SampleResult (que contiene toda la información de la respuesta, como el cuerpo, las cabeceras, el código de estado) y al contexto de JMeter. El script debe establecer Failure = true y/o FailureMessage = "..." si la aserción debe fallar.
- Configuración clave:
  - Language: Selecciona el lenguaje de scripting (se recomienda Groovy por su rendimiento).
  - Script File: Carga el script desde un archivo externo (opcional).
  - Script: El área de texto para tu código personalizado.
- Uso común:
  - Validaciones complejas o condicionales que no pueden ser manejadas por otras aserciones (ej., validar si el valor de un campo es mayor que el de otro, realizar cálculos, o validar múltiples condiciones a la vez).
  - Cuando necesitas interactuar con variables de JMeter o llamar a funciones Java para la validación.
  - Validar formatos no estándar o lógicas de negocio muy específicas.

© JMA 2020. All rights reserved

BeanShell Assertion (Aserción BeanShell)

- Propósito: Similar a JSR223 Assertion, permite escribir código de scripting en BeanShell para validar la respuesta.
- Comportamiento: Ejecuta el script BeanShell. Al igual que con JSR223, el script accede al SampleResult y al contexto. Se debe establecer Failure = true y/o FailureMessage = "..." para indicar un fallo.
- Configuración clave: Script: El área de texto para tu código BeanShell.
- Uso común:
  - Validaciones personalizadas cuando no existían las aserciones de formato específico o el JSR223 Assertion no estaba tan optimizado.
- Importante: Se prefiere el JSR223 Assertion con Groovy sobre BeanShell Assertion para nuevos desarrollos debido a su rendimiento significativamente superior. BeanShell es un lenguaje interpretado más lento.

© JMA 2020. All rights reserved

PROCESSORS

© JMA 2020. All rights reserved

Processors

Pre Processors

- HTML Link Parser
- HTTP URL Re-writing Modifier
- User Parameters
- BeanShell PreProcessor
- JSR223 PreProcessor
- JDBC PreProcessor
- RegEx User Parameters
- Sample Timeout

Post Processors

- Regular Expression Extractor
- CSS Selector Extractor (was: CSS/JQuery Extractor)
- XPath2 Extractor
- XPath Extractor
- JSON JMESPath Extractor
- Result Status Action Handler
- BeanShell PostProcessor
- JSR223 PostProcessor
- JDBC PostProcessor
- JSON Extractor
- Boundary Extractor

© JMA 2020. All rights reserved

Elementos del preprocesador

- Un preprocesador ejecuta alguna acción antes de que se realice una solicitud de muestra.
- Si se adjunta un preprocesador a un elemento muestreador, entonces se ejecutará justo antes de que se ejecute ese elemento.
- Un preprocesador se utiliza principalmente para modificar la configuración de una solicitud de muestra justo antes de que se ejecute, o para actualizar las variables que no se extraen del texto de respuesta.

© JMA 2020. All rights reserved

User Parameters (Parámetros de Usuario)

- Propósito: Definir variables que cambian su valor en cada iteración para cada hilo. Es una alternativa a User Defined Variables cuando necesitas que los valores se actualicen por iteración.
- Comportamiento: Permite definir un conjunto de variables con múltiples valores. En cada nueva iteración del bucle, el Pre-Procesador asigna el siguiente conjunto de valores a las variables.
- Configuración clave:
  - Tabla Variables: Define los nombres de las variables.
  - Tabla User_X: Define los valores que tomará cada variable en cada iteración.
- Uso común:
  - Cuando un hilo necesita usar diferentes credenciales o datos en cada una de sus iteraciones, y la cantidad de datos es pequeña o conocida de antemano (a diferencia de CSV Data Set Config para grandes volúmenes).
  - Simular la rotación de datos por cada iteración.

© JMA 2020. All rights reserved

RegEx User Parameters (Parámetros de Usuario RegEx)

- Propósito: Similar a User Parameters, pero los valores se definen utilizando expresiones regulares.
- Comportamiento: Permite definir un conjunto de variables. Cada variable tiene un campo donde puedes introducir una expresión regular, y el pre-procesador generará un valor que coincida con esa expresión para cada iteración o usuario.
- Uso común: Para generar cadenas de texto que siguen un patrón específico (ej., códigos alfanuméricos, nombres de archivo con un formato particular) pero que no son puramente numéricos o aleatorios simples.

© JMA 2020. All rights reserved

HTML Link Parser (Analizador de Enlaces HTML)

- Propósito: Extraer automáticamente todos los enlaces (URLs) de una respuesta HTML anterior y permitir que las peticiones subsiguientes utilicen esos enlaces de forma dinámica.
- Comportamiento: No se asocia directamente a un Sampler saliente, sino que se asocia a un Sampler previo cuya respuesta contiene el HTML a analizar. Captura todos los enlaces HTML (etiquetas <a>, <img>, <iframe>, script con src, link con href, etc.) y los almacena en variables. Luego, puedes configurar los Samplers siguientes para que "sigan" estos enlaces.
- Uso común: Simular la navegación real de un navegador que carga todos los recursos de una página. Muy útil en conjunto con un HTTP Request para cargar recursos embebidos o con ForEach Controller para iterar sobre enlaces encontrados. Raramente usado hoy en día; la opción "Retrieve All Embedded Resources" en el HTTP Request Sampler es más directa para la mayoría de los casos.

© JMA 2020. All rights reserved

HTTP URL Re-writing Modifier (Modificador de Reescritura de URL HTTP)

- Propósito: Modificar las URLs de las solicitudes HTTP para incluir parámetros de sesión que se han reescrito en la URL (en lugar de usarse como cookies o parámetros POST).
- Comportamiento: Busca un nombre de parámetro de sesión específico en la respuesta anterior y lo añade automáticamente a las URLs de las solicitudes siguientes.
- Uso común: Cuando las aplicaciones gestionan la sesión incrustando el ID de sesión directamente en la URL (ej., jsessionid en Tomcat), y no se pueden manejar con el HTTP Cookie Manager. Permite a JMeter pasar este ID de sesión entre las peticiones.

© JMA 2020. All rights reserved

Sample Timeout (Tiempo de Espera de Muestra)

- Propósito: Establecer un tiempo límite (timeout) para la ejecución del Sampler al que está asociado.
- Comportamiento: Si el Sampler tarda más del tiempo configurado en completarse, el Pre-Procesador lo detendrá y lo marcará como fallido.
- Configuración clave: Timeout (milliseconds): El tiempo máximo permitido para el Sampler.
- Uso común:
  - Asegurar que las peticiones individuales no se queden "colgadas" indefinidamente.
  - Definir un umbral de rendimiento máximo para una sola petición, similar a una aserción de duración pero que detiene la petición.

© JMA 2020. All rights reserved

JDBC PreProcessor (Pre-Procesador JDBC)

- Propósito: Ejecutar una consulta o comando SQL en una base de datos antes de que se ejecute el Sampler asociado.
- Comportamiento: Se conecta a una base de datos (utilizando una JDBC Connection Configuration previa) y ejecuta la consulta SQL. Puedes almacenar los resultados de la consulta en variables de JMeter para usarlos en el Sampler siguiente.
- Configuración clave:
  - Variable Name of Pool (configured in JDBC Connection Configuration): El nombre del pool de conexiones configurado.
  - Query Type: Tipo de consulta SQL (Select, Update, Callable, Prepared).
  - SQL Query: La consulta SQL a ejecutar.
  - Result Variable Name: El nombre de la variable donde se almacenarán los resultados de la consulta.
- Uso común:
  - Obtener datos de prueba específicos de una base de datos (ej., un ID de usuario recién creado, el estado de un pedido) que son necesarios para la siguiente solicitud HTTP.
  - Preparar el estado de la base de datos (ej., insertar un registro) antes de probar una funcionalidad que lo requiere.

© JMA 2020. All rights reserved

JSR223 PreProcessor (Pre-Procesador JSR223)

- Propósito: Uno de los Pre-Procesadores más potentes y flexibles. Permite ejecutar código de scripting personalizado (preferiblemente Groovy) justo antes de que se envíe el Sampler asociado.
- Comportamiento: El script que definas se ejecuta justo antes del Sampler. Tiene acceso completo al Sampler actual, al log, al contexto de JMeter y a las variables, lo que permite modificar dinámicamente la solicitud, generar datos complejos o interactuar con el entorno.
- Configuración clave:
  - Language: El lenguaje de scripting (Groovy es el más recomendado).
  - Script File: Cargar el script desde un archivo externo (opcional).
  - Script: El área de texto para tu código personalizado.
- Uso común:
  - Generar firmas criptográficas para cabeceras o cuerpos de solicitud antes de enviarlos.
  - Calcular valores dinámicos complejos para parámetros de solicitud.
  - Modificar la URL o los parámetros de un Sampler de forma programática.
  - Interactuar con bases de datos o sistemas externos para obtener datos necesarios para la próxima solicitud.
  - Realizar tareas de precondición complejas.

© JMA 2020. All rights reserved

BeanShell PreProcessor (Pre-Procesador BeanShell)

- Propósito: Similar al JSR223 PreProcessor, permite ejecutar código de scripting en BeanShell justo antes de un Sampler.
- Comportamiento: El script BeanShell se ejecuta, teniendo acceso al Sampler y al contexto de JMeter para realizar modificaciones o generar datos.
- Configuración clave: Script: El área de texto para tu código BeanShell.
- Uso común: Los mismos casos que el JSR223 PreProcessor.
- Importante: Al igual que con otros elementos BeanShell, el JSR223 PreProcessor con Groovy es el reemplazo recomendado debido a su rendimiento significativamente superior.

© JMA 2020. All rights reserved

Elementos del postprocesador

- Un postprocesador ejecuta alguna acción después de que se haya realizado una solicitud de muestra.
- Si un postprocesador está conectado a un elemento Sampler, se ejecutará justo después de que se ejecute ese elemento.
- Un postprocesador se utiliza para procesar los datos de respuesta, a menudo para extraer valores de él.

© JMA 2020. All rights reserved

Result Status Action Handler

- Propósito: Realizar una acción basada en el estado del resultado de la muestra (éxito o fallo).
- Comportamiento: Define una acción a realizar si el Sampler tiene éxito o falla. Las acciones incluyen detener el hilo, detener la prueba, ir al siguiente bucle, etc.
- Configuración clave:
  - Main sample only / Sub-samples only: Afecta solo a la muestra principal o a las sub-muestras.
  - Sampler status: JMeter thread is OK o JMeter thread is NOT OK.
  - Action to be taken: Continue, Stop Thread, Stop Test, Go to next loop.
- Uso común: Detener un hilo si una petición crítica falla, o continuar al siguiente bucle si la petición tiene éxito.

© JMA 2020. All rights reserved

Boundary Extractor

- Propósito: Extraer una cadena de texto delimitada por un prefijo y un sufijo.
- Comportamiento: Busca la primera coincidencia de la cadena delimitada y la extrae.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable.
  - Left Boundary: El prefijo.
  - Right Boundary: El sufijo.
  - Match No., Default Value: Similar al Regular Expression Extractor.
- Uso común: Extraer valores simples que están claramente delimitados en la respuesta. Es menos flexible que las expresiones regulares, pero más fácil de usar para casos sencillos.

© JMA 2020. All rights reserved

CSS/JQuery Extractor (Extractor CSS/JQuery)

- Propósito: Extraer datos de respuestas HTML o XHTML utilizando selectores CSS (o JQuery).
- Comportamiento: Utiliza selectores CSS para encontrar elementos en el HTML y extraer su texto, atributos u otras propiedades.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable.
  - CSS/JQuery expression: El selector CSS a utilizar (ej., div.product h2, a[href]).
  - Attribute: El atributo a extraer (ej., href, src, value). Si está vacío, extrae el texto del elemento.
  - Match No., Default Value: Similar al Regular Expression Extractor.
- Uso común: Extraer enlaces, títulos, valores de campos de formulario, o cualquier dato de una página HTML basándose en su estructura.

© JMA 2020. All rights reserved

Regular Expression Extractor (Extractor de Expresiones Regulares)

- Propósito: Extraer datos de la respuesta utilizando expresiones regulares. Es uno de los extractores más versátiles.
- Comportamiento: Busca una coincidencia con la expresión regular que definas en la parte de la respuesta que especifiques (cuerpo, cabeceras, URL, etc.). Puede extraer uno o múltiples valores.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable de JMeter donde se almacenará el valor extraído.
  - Regular Expression: La expresión regular a utilizar.
  - Template: Cómo combinar múltiples grupos de la expresión regular (ej., 1, $1$$2$).
  - Match No.: 0 para aleatorio, 1 para la primera coincidencia, etc.
  - Default Value: El valor que se usará si no se encuentra ninguna coincidencia.
  - Scope: Dónde buscar la coincidencia (cuerpo, cabeceras, etc.).
- Uso común: Extraer IDs de sesión, tokens de autenticación, valores dinámicos de formularios, o cualquier dato que siga un patrón.

© JMA 2020. All rights reserved

JSON Extractor

- Propósito: Extraer valores de una respuesta JSON utilizando una sintaxis similar a JavaScript.
- Comportamiento: Permite especificar una expresión para seleccionar elementos del JSON y guardarlos en variables.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable.
  - JSON Path expressions: La expresión para seleccionar el elemento (ej., $.items[0].name).
  - Match No., Default Values: Similar al Regular Expression Extractor.
- Uso común: Extraer datos de APIs REST que devuelven JSON.

© JMA 2020. All rights reserved

JSON JMESPath Extractor

- Propósito: Extraer datos de respuestas JSON utilizando expresiones JMESPath. JMESPath es un lenguaje poderoso para consultar JSON.
- Comportamiento: Evalúa una expresión JMESPath sobre el JSON de la respuesta y almacena el resultado.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable.
  - JMESPath Expression: La expresión JMESPath (ej., data.items[0].name).
  - Default Value: El valor por defecto.
  - Match No.: Similar al Regular Expression Extractor.
- Uso común: Extraer datos de APIs REST que devuelven JSON, especialmente cuando se necesitan filtros o transformaciones complejas.

© JMA 2020. All rights reserved

XPath Extractor

- Propósito: Extraer datos de respuestas XML o XHTML utilizando expresiones XPath 1.0.
- Comportamiento: Similar al XPath2 Extractor, pero utiliza XPath 1.0.
- Configuración clave: Similar al XPath2 Extractor, pero con sintaxis XPath 1.0.
- Uso común: Extraer datos de respuestas XML, especialmente en servicios web SOAP.

© JMA 2020. All rights reserved

XPath2 Extractor

- Propósito: Extraer datos de respuestas XML o XHTML utilizando expresiones XPath 2.0.
- Comportamiento: Evalúa una expresión XPath 2.0 y almacena el resultado en una variable.
- Configuración clave:
  - Name of created variable(s): El nombre de la variable.
  - XPath2 Query: La expresión XPath 2.0.
  - Default Value: El valor por defecto.
  - Match No.: Similar al Regular Expression Extractor.
- Uso común: Extraer datos de respuestas XML complejas donde XPath 1.0 es insuficiente.

© JMA 2020. All rights reserved

JDBC PostProcessor

- Propósito: Ejecutar una consulta o comando SQL en una base de datos después de recibir una respuesta.
- Comportamiento: Se conecta a una base de datos (usando una JDBC Connection Configuration previa) y ejecuta la consulta.
- Configuración clave: Similar al JDBC PreProcessor.
- Uso común: Actualizar la base de datos basándose en la respuesta (ej., marcar un pedido como completado), o realizar alguna acción de limpieza.

© JMA 2020. All rights reserved

BeanShell PostProcessor

- Propósito: Ejecutar código de scripting en BeanShell después de recibir una respuesta.
- Comportamiento: El script BeanShell se ejecuta después del Sampler. Tiene acceso al SampleResult (la respuesta), al contexto de JMeter y a las variables.
- Configuración clave: Script: El código BeanShell a ejecutar.
- Uso común: Procesamiento de datos complejo, manipulación de variables, o lógica personalizada después de la respuesta.
- Importante: Se prefiere el JSR223 PostProcessor con Groovy sobre BeanShell PostProcessor para nuevos desarrollos debido a su rendimiento superior.

© JMA 2020. All rights reserved

JSR223 PostProcessor

- Propósito: Ejecutar código de scripting (Groovy, etc.) después de recibir una respuesta. Es más potente y flexible que BeanShell.
- Comportamiento: El script JSR223 se ejecuta después del Sampler, con acceso completo al SampleResult, al contexto y a las variables.
- Configuración clave:
  - Language: El lenguaje de scripting (Groovy es el más común).
  - Script File: Cargar el script desde un archivo externo (opcional).
  - Script: El código a ejecutar.
- Uso común: Los mismos casos que el BeanShell PostProcessor, pero con mejor rendimiento y acceso a más funcionalidades.

© JMA 2020. All rights reserved

RECEPTORES

© JMA 2020. All rights reserved

Receptores

- Los receptores proporcionan acceso a la información que JMeter recopila sobre los casos de prueba mientras JMeter se ejecuta. La mayoría de los receptores realizan varias funciones además de "escuchar" los resultados de la prueba. También proporcionan medios para ver, guardar y leer los resultados de las pruebas guardadas.
- Se pueden agregar receptores en cualquier parte de la prueba, incluso directamente en el plan de prueba. Recopilarán datos solo de elementos en el que están asociado o por debajo de su nivel.
- Todos los receptores guardan los mismos datos, la única diferencia está en la forma en que se presentan los datos en pantalla.
- Los receptores pueden dirigir los datos a un archivo para su uso posterior, proporcionan un campo para indicar el archivo para almacenar datos y permiten elegir qué campos guardar, así como el formato: CSV o XML.
- Los receptores pueden presentar los datos en formato grafico, tabular, árbol, especializado y para depurar. Algunos proporcionan información de resumen o agregación.

© JMA 2020. All rights reserved

Archivo de desultados .jtl (JMeter Test Log)

- El JMeter Test Log es el formato de salida fundamental y principal de JMeter. Todos los demás informes y Listeners se basan en este tipo de archivos.
- Un archivo JTL es un archivo de texto plano que contiene los resultados detallados de cada muestra (Sampler) y sub-muestra ejecutada durante la prueba. Puedes configurar su formato como CSV (Comma Separated Values) o XML (Extensible Markup Language).
- Las características de cada formato son:
  - CSV (por defecto): Es el formato más ligero y recomendado para pruebas de carga a gran escala debido a su menor tamaño de archivo y mejor rendimiento de escritura. No guarda los datos de respuesta (response data) por defecto.
  - XML: Es más verboso y consume más espacio en disco, pero puede incluir más detalles como los datos completos de la solicitud y la respuesta si se configura para ello (ej., jmeter.save.saveservice.output_format=xml y jmeter.save.saveservice.response_data=true en user.properties o como argumento de línea de comandos -J).
- Por defecto, un archivo JTL en formato CSV incluye columnas como timeStamp, elapsed, label, responseCode, responseMessage, threadName, success, bytes, grpThreads, allThreads, URL, Latency, etc. Puedes personalizar qué campos se guardan en el archivo .jtl a través de la configuración del Listener (ej., Simple Data Writer o View Results Tree con el botón "Configure") o mediante propiedades en jmeter.properties/user.properties.

© JMA 2020. All rights reserved

Listeners

- Sample Result Save Configuration
- Graph Results
- Assertion Results
- View Results Tree
- Aggregate Report
- View Results in Table
- Simple Data Writer
- Aggregate Graph
- Response Time Graph
- Mailer Visualizer
- BeanShell Listener
- Summary Report
- Save Responses to a file
- JSR223 Listener
- Generate Summary Results
- Comparison Assertion Visualizer
- Backend Listener

© JMA 2020. All rights reserved

Summary Report (Informe Resumen)

- Propósito: Proporciona una tabla concisa que resume las métricas clave de rendimiento para cada Sampler (etiqueta de solicitud) único en tu plan de pruebas. Agrupa los resultados de todas las iteraciones y hilos.
- Comportamiento: Calcula y muestra estadísticas como el número total de muestras, el tiempo de respuesta promedio, el rendimiento (Throughput), el porcentaje de error, los percentiles (ej., línea del 90%, 95%, 99%), entre otros.
- Casos de uso:
  - Obtener una visión general rápida y de alto nivel del rendimiento de tu aplicación durante o después de una prueba.
  - Identificar rápidamente qué transacciones o páginas están funcionando más lento o generando más errores.
  - Generar un informe tabular básico para compartir con los interesados.

© JMA 2020. All rights reserved

Generate Summary Results (Generar Resultados Resumen)

- Propósito: Este no es un visualizador en la GUI, sino un Listener diseñado específicamente para imprimir un resumen conciso de los resultados de la prueba directamente en la consola (o en el archivo de log de JMeter) mientras la prueba se está ejecutando.
- Comportamiento: Periódicamente (cada 30 segundos por defecto), imprime una línea en la consola con estadísticas clave como el rendimiento (Throughput), el tiempo de respuesta promedio y el porcentaje de error para el intervalo de tiempo transcurrido desde el último resumen. También imprime un resumen acumulativo.
- Casos de uso:
  - Monitorear rápidamente el progreso de la prueba y el rendimiento general en un entorno no-GUI (cuando JMeter se ejecuta desde la línea de comandos).
  - Obtener un feedback inmediato sobre el estado de la prueba sin la sobrecarga de un Listener gráfico o tabular.
  - Confirmar que la prueba se está ejecutando y produciendo carga según lo esperado.

© JMA 2020. All rights reserved

Aggregate Report (Informe Agregado)

- Propósito: Proporcionar un resumen conciso y tabular de las métricas clave de rendimiento para todas las etiquetas de solicitud (Samplers) únicas en tu plan de pruebas. Agrega datos de todos los hilos e iteraciones.
- Comportamiento: Calcula y muestra datos estadísticos para cada etiqueta de Sampler única. Es uno de los Listeners más utilizados para resumir los resultados de las pruebas.
- Métricas mostradas:
  - #Muestras: Número total de solicitudes.
  - Promedio: Tiempo de respuesta promedio.
  - Mediana: El percentil 50 de tiempo de respuesta.
  - Línea del 90%, Línea del 95%, Línea del 99%: Tiempos de respuesta por debajo de los cuales caen el 90%, 95% o 99% de las muestras (importante para entender la consistencia de la experiencia del usuario).
  - Mín/Máx: Tiempos de respuesta mínimo y máximo.
  - % de Errores: Porcentaje de muestras que fallaron.
  - Rendimiento (Throughput): Solicitudes por segundo/minuto.
  - KB/seg. recibidos, KB/seg. enviados: Tráfico de red.
- Casos de uso:
  - Resumen de alto nivel de los resultados de la prueba.
  - Identificar Samplers/transacciones lentas.
  - Informar sobre el rendimiento general de la prueba a las partes interesadas.
  - Comparar el rendimiento entre diferentes ejecuciones de prueba.

© JMA 2020. All rights reserved

Aggregate Graph (Gráfico Agregado)

- Propósito: Este Listener es una extensión del Aggregate Report que visualiza los resultados en un gráfico de barras, haciendo que los resúmenes estadísticos sean más intuitivos de comparar.
- Comportamiento: Muestra los valores de métricas clave (como el tiempo de respuesta promedio, la mediana y la línea del 90%) en un gráfico de barras para cada Sampler único (etiqueta) de tu prueba. Esto te permite comparar rápidamente el rendimiento entre diferentes transacciones o páginas.
- Métricas mostradas: Barras que representan los tiempos de respuesta promedio, medianos, 90%, 95% y 99% para cada Sampler. También puede mostrar el rendimiento (throughput) y el porcentaje de error.
- Casos de uso:
  - Comparar rápidamente el rendimiento de diferentes partes de tu aplicación o diferentes transacciones de negocio.
  - Identificar visualmente dónde se encuentran los cuellos de botella más significativos en tu aplicación.
  - Presentar un resumen gráfico de alto nivel de los resultados de la prueba a un público no técnico.
- Consideración: Al igual que otros Listeners gráficos, no se recomienda usarlo durante la ejecución de pruebas de carga grandes en modo GUI debido al consumo de recursos. Es ideal para cargar y analizar un archivo .jtl después de la prueba.

© JMA 2020. All rights reserved

View Results Tree (Vista del Árbol de Resultados)

- Propósito: Mostrar los resultados detallados de cada muestra individual en una estructura de árbol jerárquica. Este es probablemente el Listener más potente para la depuración.
- Comportamiento: Para cada Sampler que se ejecuta, captura y muestra la solicitud completa enviada, la respuesta completa recibida, los códigos de estado, los mensajes de respuesta, las cabeceras y los resultados de cualquier aserción.
- Secciones mostradas:
  - Resultado de la muestra: Resumen (estado, tiempo transcurrido, latencia, tamaño, etc.).
  - Solicitud (Request): Los datos completos de la solicitud enviada al servidor.
  - Datos de respuesta (Response data): El cuerpo completo de la respuesta recibida del servidor.
  - Cabeceras de respuesta: Todas las cabeceras de la respuesta del servidor.
  - Resultados de aserciones: Mensajes detallados de éxito/fallo para cada aserción.
- Limitaciones: Extremadamente intensivo en recursos. Almacenar todos estos detalles consume mucha memoria y CPU. Nunca uses este Listener durante la ejecución de una prueba de carga real en modo GUI o en los motores de prueba. Solo debe usarse para depurar pruebas a pequeña escala o para el análisis posterior de un archivo .jtl guardado.
- Casos de uso:
  - Depuración de scripts de prueba: Esencial para entender por qué una solicitud está fallando o por qué un extractor no funciona.
  - Analizar los detalles de las muestras individuales después de una ejecución de prueba (desde un archivo .jtl).
  - Validar la correlación y parametrización.

© JMA 2020. All rights reserved

View Results in Table (Vista Resultados en Tabla)

- Propósito: Mostrar una tabla simple de resultados resumidos para cada muestra individual, fila por fila.
- Comportamiento: Lista los resultados de cada muestra (marca de tiempo, etiqueta, tiempo de muestra, estado, bytes, latencia, etc.) en un formato tabular. Es menos intensivo en recursos que "View Results Tree" pero aún puede consumir mucha memoria si se muestran un gran número de muestras.
- Limitaciones: Puede volverse difícil de manejar con un número muy alto de muestras.
- Casos de uso:
  - Monitorear una prueba en curso (en modo GUI para pruebas pequeñas, pero siempre con precaución).
  - Escanear rápidamente los tiempos y estados de las muestras individuales.
  - Exportar una lista simple de resultados de muestras a un archivo CSV.

© JMA 2020. All rights reserved

Graph Results (Resultados Gráficos)

- Propósito: Representar visualmente métricas clave de rendimiento (como tiempos de respuesta, rendimiento) a lo largo del tiempo en formato gráfico.
- Comportamiento: Grafica el tiempo de respuesta promedio para cada muestra (o otras métricas seleccionadas) frente a la duración de la prueba. Esto ofrece una visión rápida de las tendencias de rendimiento.
- Métricas mostradas (comunes): Tiempo de respuesta promedio (a lo largo del tiempo), rendimiento (Throughput), desviación.
- Limitaciones: Puede consumir mucha memoria y CPU, especialmente en pruebas grandes. No se recomienda usarlo durante la ejecución de la prueba en modo GUI ya que puede afectar la prueba en sí. Es mejor para analizar archivos .jtl guardados.
- Casos de uso:
  - Identificar rápidamente tendencias de degradación o mejora del rendimiento.
  - Presentar información de rendimiento de alto nivel a partes interesadas no técnicas.

© JMA 2020. All rights reserved

Response Time Graph
(Gráfico de Tiempo de Respuesta)

- Propósito: Este Listener genera un gráfico de líneas que muestra cómo los tiempos de respuesta cambian a lo largo del tiempo durante la ejecución de la prueba.
- Comportamiento: Traza los tiempos de respuesta (promedio, mínimo, máximo, desviación estándar) en un eje Y, con el tiempo transcurrido de la prueba en el eje X. Esto te permite observar tendencias y fluctuaciones en el rendimiento.
- Métricas mostradas: Líneas que representan el tiempo de respuesta promedio, mínimo, máximo y/o la desviación estándar a lo largo del tiempo. Puedes seleccionar cuáles mostrar.
- Casos de uso:
  - Identificar tendencias de rendimiento: Ver si el tiempo de respuesta empeora, mejora o se mantiene estable a medida que avanza la prueba o a medida que aumenta la carga.
  - Detectar picos de rendimiento o caídas repentinas.
  - Correlacionar visualmente los cambios en el rendimiento con eventos específicos de la prueba o con datos de monitoreo del servidor.
- Consideración: Útil para análisis post-prueba de archivos .jtl.

© JMA 2020. All rights reserved

Assertion Results
(Resultados de Aserción)

- Propósito: Mostrar solo los resultados de las Aserciones (éxito o fallo) para cada muestra. Proporciona una lista concisa de qué aserciones pasaron y cuáles fallaron.
- Comportamiento: Para cada Sampler ejecutado dentro de su ámbito, verifica las aserciones asociadas. Si una aserción falla, listará el nombre del Sampler y el mensaje específico de fallo de la aserción. Si todas las aserciones pasan, la muestra generalmente no se lista, o se lista brevemente como exitosa.
- Limitaciones: No muestra los datos brutos de solicitud/respuesta; solo los resultados de las aserciones.
- Casos de uso:
  - Identificar rápidamente fallos funcionales durante una prueba de rendimiento.
  - Depurar por qué un paso de prueba específico está fallando funcionalmente en lugar de simplemente ser lento.
  - Asegurarse de que la aplicación devuelve el contenido correcto bajo carga.

© JMA 2020. All rights reserved

Comparison Assertion Visualizer (Visualizador de Aserción de Comparación)

- Propósito: Este Listener (generalmente disponible a través de plugins) está diseñado para visualizar los resultados de la "Comparison Assertion" y ayudarte a depurar aserciones complejas que implican comparar dos conjuntos de datos.
- Comportamiento: Funciona en conjunto con una Comparison Assertion (otro elemento de JMeter que compara dos valores, como una respuesta actual con una esperada, o dos respuestas entre sí). El visualizador muestra las diferencias de forma clara, resaltando dónde no coinciden los valores esperados y reales.
- Casos de uso:
  - Depuración de aserciones complejas que involucran grandes cuerpos de respuesta (ej., JSON o XML).
  - Verificar que los datos devueltos por el servidor coinciden exactamente con lo esperado.
  - Identificar cambios sutiles en las respuestas que podrían indicar regresiones funcionales.

© JMA 2020. All rights reserved

Save Responses to a file (Guardar Respuestas a un archivo)

- Propósito: Permite guardar los cuerpos de las respuestas recibidas de los servidores en archivos individuales en tu sistema de archivos local.
- Comportamiento: Por cada Sampler que se ejecuta y genera una respuesta, este Listener crea un nuevo archivo (con un nombre basado en la etiqueta del Sampler y un número de secuencia) y guarda el cuerpo de la respuesta dentro de él.
- Configuraciones clave:
  - Filename prefix (Prefijo del nombre de archivo): Un prefijo común para todos los archivos generados (ej., respuesta_).
  - Directorio: La carpeta donde se guardarán los archivos.
  - Don't add number to prefix: Para que no se añada un número secuencial al nombre.
  - Add timestamp: Añade una marca de tiempo al nombre del archivo.
  - XML / HTML / JSON: Formatos específicos que puedes seleccionar para cómo se guardan los archivos.
- Casos de uso:
  - Depuración detallada: Si necesitas inspeccionar el contenido exacto de las respuestas para entender por qué una prueba falla o para analizar los datos recibidos.
  - Guardar evidencia de lo que el servidor devolvió durante una prueba de carga.
  - Cuando se trabaja con respuestas binarias o muy grandes que son difíciles de ver en el View Results Tree.

© JMA 2020. All rights reserved

Simple Data Writer (Escritor de Datos Simple)

- Propósito: Este Listener está diseñado para guardar los resultados completos de la prueba en un archivo, permitiendo una configuración detallada del formato y los datos a incluir. Es esencial para análisis posterior fuera de JMeter o para pruebas de carga grandes donde la visualización en tiempo real no es práctica.
- Comportamiento: Escribe los datos de cada muestra (sampler) en un archivo a medida que la prueba se ejecuta. Puedes elegir los campos específicos a incluir (ej., marca de tiempo, tiempo transcurrido, código de respuesta, etc.) y el formato de salida (CSV para análisis tabular, o XML para un formato más estructurado).
- Configuraciones clave:
  - Filename (Nombre de archivo): La ruta y el nombre del archivo donde se guardarán los resultados (ej., resultados.jtl).
  - Configure (Configurar): Abre una ventana donde puedes seleccionar qué campos de datos de la muestra quieres guardar (ej., duración, estado de éxito, mensaje de respuesta, cuerpo de la solicitud/respuesta, etc.). Es crucial ajustar esto para equilibrar el nivel de detalle y el tamaño del archivo.
- Casos de uso:
  - Guardar datos brutos para análisis posterior: La forma más común de registrar los resultados de una prueba de carga en modo no-GUI.
  - Generar archivos de entrada para herramientas de análisis personalizadas o scripts de generación de informes.
  - Proporcionar un registro completo y auditable de cada muestra ejecutada.

© JMA 2020. All rights reserved

Mailer Visualizer (Notificaciones por correo)

- Propósito: Este Listener tiene una función única: enviar notificaciones por correo electrónico en función de los resultados de la prueba, especialmente cuando se cumplen ciertas condiciones.
- Comportamiento: Monitorea continuamente los resultados de la prueba y, si se supera un umbral de errores o si se detecta un fallo de Sampler, puede enviar un correo electrónico a una lista de destinatarios configurada.
- Configuraciones clave:
  - Host SMTP, Puerto, Autenticación: Detalles del servidor de correo saliente.
  - Remitente, Destinatarios (To, CC, BCC): Quién envía y quién recibe el correo.
  - Subject (Asunto): El asunto del correo.
  - Failure/Error Threshold (Umbral de Fallo/Error): El porcentaje de errores que, si se supera, activará el envío del correo.
  - Failure/Error Message: El contenido del mensaje de correo que se enviará.
- Casos de uso:
  - Alertar a los equipos de desarrollo u operaciones cuando una prueba de carga falla críticamente o cuando el sistema comienza a degradarse.
  - Proporcionar resúmenes automatizados de los resultados de las pruebas por correo electrónico al finalizar la ejecución.
  - Integración en pipelines de CI/CD para notificaciones automáticas de rendimiento.

© JMA 2020. All rights reserved

Backend Listener (Oyente de Backend)

- Propósito: Envía los resultados de la prueba a un sistema de monitoreo o base de datos de series de tiempo externa para un monitoreo en tiempo real y un análisis a largo plazo más potente.
- Comportamiento: En lugar de procesar y mostrar los resultados dentro de JMeter, este Listener los recolecta y los transmite a un sistema backend (como InfluxDB, Graphite, Prometheus, ElasticSearch) a través de sus respectivos clientes o APIs. Esto permite usar herramientas de visualización como Grafana para crear paneles de control dinámicos.
- Configuraciones clave:
  - Backend Listener Implementation: Selecciona el cliente para el sistema backend (ej., InfluxdbBackendListenerClient, GraphiteBackendListenerClient).
  - Parameters: Parámetros específicos para la conexión y configuración del backend (ej., URL de la base de datos, nombre de la base de datos, métricas a enviar).
- Casos de uso:
  - Monitoreo en tiempo real de pruebas de carga a través de paneles de control externos (ej., Grafana).
  - Almacenamiento de datos de rendimiento a largo plazo para análisis de tendencias y comparaciones históricas.
  - Integración de los resultados de JMeter en un ecosistema de monitoreo y APM (Application Performance Management) más amplio.
  - Reduce la sobrecarga de JMeter al no tener que procesar y mostrar gráficos/tablas internamente durante la ejecución de la prueba.

© JMA 2020. All rights reserved

Sample Result Save Configuration (Configuración de Guardado de Resultados de Muestras)

- Este no es un Listener en el sentido tradicional de mostrar resultados, sino un elemento de configuración que define cómo se guardan los resultados. Esto impacta directamente el nivel de detalle que otros Listeners pueden mostrar o que se puede analizar al cargar un archivo de resultados.
- Propósito: Definir qué puntos de datos específicos (ej., encabezados de respuesta, cuerpo de solicitud, resultados de aserciones) debe guardar JMeter para cada resultado de muestra, especialmente al escribir en un archivo.
- Comportamiento: Se suele añadir al nivel del Plan de Pruebas o bajo un Listener específico. Controla las propiedades sample.save_configuration en jmeter.properties.
- Configuraciones clave: Puedes seleccionar casillas para guardar varios datos:
  - Marca de tiempo, tiempo transcurrido, éxito, etiqueta, código de respuesta, mensaje, nombre del hilo: Métricas básicas de la prueba.
  - Resultados de aserciones: Si se almacenan los detalles de éxito/fallo de las aserciones.
  - Cabeceras de solicitud, cabeceras de respuesta, cuerpo de solicitud, cuerpo de respuesta: Crucial para la depuración.
  - URL, Latencia, Tiempo de Conexión: Detalles de tiempo de red.
- Casos de uso:
  - Depuración: Habilitar el guardado de cabeceras y cuerpos de solicitud/respuesta para solucionar problemas de muestras fallidas.
  - Optimización del tamaño del archivo: Deshabilitar el guardado de datos innecesarios al ejecutar pruebas grandes para reducir el tamaño del archivo .jtl y la sobrecarga de E/S.

© JMA 2020. All rights reserved

BeanShell Listener (Oyente BeanShell)

- Propósito: Permite ejecutar código de scripting personalizado en BeanShell (un lenguaje similar a Java) para procesar los resultados de las muestras.
- Comportamiento: A medida que cada Sampler termina su ejecución, el código BeanShell definido en este Listener se ejecuta. Este script tiene acceso al objeto SampleResult (que contiene todos los detalles de la muestra) y al contexto de JMeter, permitiendo una manipulación avanzada de los datos.
- Configuraciones clave:
  - Script: El área de texto donde escribes tu código BeanShell.
- Casos de uso:
  - Realizar análisis de datos de muestra complejos y personalizados en tiempo real.
  - Integrar los resultados de JMeter con sistemas externos que no tienen un Listener dedicado (ej., enviar métricas a una base de datos personalizada, llamar a una API externa con los resultados).
  - Generar informes personalizados que JMeter no ofrece por defecto.
  - Implementar lógica de post-procesamiento avanzada para los resultados.
- Consideración: Al igual que el BeanShell Sampler, el BeanShell Listener puede ser más lento que el equivalente JSR223 Listener (que soporta Groovy) para scripts complejos. Para nuevos desarrollos, se prefiere el JSR223 Listener con Groovy por su mejor rendimiento.

© JMA 2020. All rights reserved

JSR223 Listener (Oyente JSR223)

- Propósito: Este es un Listener extremadamente flexible y potente que te permite ejecutar código de scripting personalizado (utilizando Groovy, JavaScript, Python, etc.) para procesar los resultados de cada muestra.
- Comportamiento: A medida que cada Sampler completa su ejecución, el script que definas en este Listener se ejecuta. El script tiene acceso al objeto SampleResult (que contiene todos los detalles de la muestra, como tiempo, éxito, respuesta) y al contexto completo de JMeter, permitiendo una manipulación avanzada de datos o interacciones externas.
- Configuraciones clave:
  - Language (Lenguaje): Selecciona el lenguaje de scripting (se recomienda Groovy por su rendimiento).
  - Script File (Archivo de script): Puedes cargar el script desde un archivo externo.
  - Script: El área de texto donde escribes tu código personalizado.
- Casos de uso:
  - Realizar análisis de datos de muestra complejos y personalizados que no están disponibles en otros Listeners.
  - Integrar JMeter con sistemas externos que no tienen un Listener dedicado (ej., enviar métricas a una base de datos personalizada, invocar una API externa con los resultados de la prueba).
  - Generar informes personalizados o procesar datos de formas únicas.
  - Implementar lógica de post-procesamiento avanzada para los resultados.

© JMA 2020. All rights reserved

Plugins

- jp@gc - Graphs Generator / Custom Graphs (Generador de Gráficos / Gráficos Personalizados):
  - Este paquete de plugins (jmeter-plugins-cmn-jp@gc y jmeter-plugins-graphs-basic) añade una gran cantidad de gráficos que van más allá del Graph Results básico. Incluyen:
    - Response Times Over Time (Tiempos de Respuesta a lo Largo del Tiempo): Muestra el promedio de los tiempos de respuesta de los Samplers en un gráfico de líneas continuo.
    - Transactions Per Second (Transacciones Por Segundo): Muestra el rendimiento (throughput) de las transacciones exitosas y fallidas por segundo.
    - Active Threads Over Time (Hilos Activos a lo Largo del Tiempo): Muestra cuántos usuarios activos hay en cada grupo de hilos en cada momento. Esto es crucial para entender la carga aplicada.
    - Hits Per Second (Hits Por Segundo): Muestra el número de "hits" o solicitudes individuales al servidor por segundo, lo que es diferente de las transacciones (que pueden contener múltiples hits).
    - Bytes Throughput Over Time (Rendimiento de Bytes a lo Largo del Tiempo): Muestra el volumen de datos (KB/seg) enviados y recibidos.
  - Uso: Proporcionan una visualización mucho más rica y detallada de la mayoría de las métricas clave de rendimiento, ayudando a identificar tendencias y problemas de manera más efectiva.

© JMA 2020. All rights reserved

Plugins

- jp@gc - PerfMon Metrics Collector (Recolector de Métricas PerfMon):
  - Propósito: Este es uno de los plugins más cruciales. No solo es un Listener, sino que funciona con un agente (ServerAgent) que debes instalar y ejecutar en los servidores bajo prueba.
  - Función: Recopila métricas a nivel de sistema operativo (CPU, RAM, uso de disco, E/S de red) de los servidores de aplicación y base de datos durante la prueba.
  - Uso: Correlacionar el rendimiento de la aplicación con la salud de la infraestructura. Te permite ver si los cuellos de botella están en tu código o en los recursos del servidor. Muestra gráficos en tiempo real dentro de JMeter.
- jp@gc - JSON Path Tester (Probador de Ruta JSON):
  - Aunque es principalmente una herramienta de depuración para extractores (Post-Processor), se usa en el contexto de los Listeners al analizar respuestas. Te permite probar expresiones JSONPath sobre una respuesta JSON para verificar que estás extrayendo los datos correctos. No es un Listener per se, pero es invaluable cuando trabajas con APIs JSON.
- JTL Reporter Backend Listener (Oyente de Backend JTL Reporter):
  - Un ejemplo de un Backend Listener específico que envía datos a un servicio de informes externo (JTL Reporter).

© JMA 2020. All rights reserved

EJECUCIÓN

© JMA 2020. All rights reserved

Proceso de Prueba

- Preguntas iniciales¶
  - ¿Cuál es el número promedio anticipado de usuarios (carga normal)?
  - ¿Cuál es el número máximo de usuarios previsto?
  - ¿Cuándo es un buen momento para cargar y probar la aplicación (es decir, fuera de horario o fines de semana), teniendo en cuenta que esto puede muy bien bloquear uno o más de los servidores?
  - ¿La aplicación tiene estado? Si es así, ¿cómo la administra la aplicación (cookies, reescritura de URL o algún otro método)?
  - ¿Cuál es el objetivo que se pretende lograr con la prueba?
  - ¿Quién sabe cómo funciona la aplicación?
- La secuencia normal es:
  - Prueba (volumen bajo, ¿podemos comparar nuestra aplicación?)
  - Rendimiento (el número promedio de usuarios)
  - Carga (el número máximo de usuarios)
  - Estrés (¿cuál es el límite?)
- El proceso de prueba puede pasar de la prueba de caja negra a la prueba de caja blanca (la diferencia es que la primera no requiere conocimiento de la aplicación [se trata como una "caja negra"] mientras que la segunda requiere algún conocimiento de la aplicación). No es raro que se descubran problemas con la aplicación durante este proceso, así que prepárate para defender tu trabajo.

© JMA 2020. All rights reserved

Ejecutar Plan de Prueba

- En el GUI:
  - Crear y Validar Plan de Pruebas
  - Quitar/deshabilitar Receptores
  - Cerrar
- Ejecutar el plan en modo comando (no-gui) y exportar resultados:
  - jmeter -n *-f* -t demo.jmx -l demo.jtl -e -o demoinf
    - Sumario cada 30 segundos, totalizados
- Analizar resultados en el GUI:
  - Crear plan con receptores.
  - Cargar los ficheros de resultados
  - Ver informe HTML de resultados

© JMA 2020. All rights reserved

Pruebas distribuidas

- En los sistemas esclavos, ejecutar jmeter/bin/ejecute jmeter-server.bat (jmeter-server en unix).
- En el sistema maestro, modificar el fichero de propiedades jmeter/bin/jmeter.properties:

  remote_hosts=192.168.1.10,192.168.1.11,192.168.1.12

  server.rmi.ssl.disable=true
- En el sistema maestro, iniciar JMeter y abrir el plan de prueba que se desea utilizar.
- Para ejecutar la prueba:
  - Iniciar un solo cliente: Run > Remote Start > 192.168.1.10
  - Iniciar todos los clientes: Run > Remote Start All
- Para detener los sistemas esclavos, Run > Remote Stop > 192.168.1.10 ó Run > Remote Stop All

© JMA 2020. All rights reserved

Apache JMeter Dashboard

- Generar un informe HTML es una de las características más valiosas de JMeter, ya que permite transformar los resultados crudos de tus pruebas de rendimiento (archivos .jtl) en un dashboard interactivo y profesional, fácil de compartir y analizar. Este informe se crea con la herramienta de generación de informes que JMeter, no con un Listener, que se puede ejecutar con el plan de pruebas o después de que tu prueba haya finalizado:
  - Generación al ejecutar el plan de prueba:
    - jmeter -n -f -t <archivo de prueba JMX> -l <archivo de registro de prueba> **-e -o <Ruta a la carpeta de salida>**
  - Generación a partir de un archivo de registro de muestra existente
    - jmeter -g <archivo de registro> -o <Ruta a la carpeta de salida>
  - Generación en el GUI a partir de un archivo de registro CSV de muestra existente
    - Tools > Generate HTML report
- Cuando JMeter genera el informe, creará una serie de archivos y subdirectorios dentro del directorio de salida que especificaste (el archivo principal para abrirlo es index.html). Todas las propiedades del generador de informes se encuentran en el archivo /jmeter/bin/reportgenerator.properties. Para personalizarlas, se copian en el archivo user.properties y modifíquelas.

© JMA 2020. All rights reserved

Contenido y estructura del informe

- Dashboard/Overview (Panel de Control/Visión General):
  - Un resumen de alto nivel de la prueba.
  - Métricas clave como el número total de Samplers, el porcentaje de errores, el rendimiento (Throughput) y el tiempo promedio de respuesta.
  - Información sobre la prueba (fecha, duración, etc.).
- Charts (Gráficos): Una serie de gráficos interactivos que muestran tendencias y distribuciones de rendimiento:
  - Response Time Over Time (Tiempos de Respuesta a lo Largo del Tiempo): Cómo evolucionan los tiempos de respuesta.
  - Response Time Percentiles (Percentiles de Tiempo de Respuesta): Gráfico de los percentiles (50%, 90%, 95%, 99%).
  - Active Threads Over Time (Hilos Activos a lo Largo del Tiempo): Cómo cambia el número de usuarios activos.
  - Hits Per Second (Hits Por Segundo): Número de solicitudes enviadas al servidor.
  - Transactions Per Second (Transacciones Por Segundo): Rendimiento de las transacciones.
  - Bytes Throughput (Rendimiento de Bytes): Datos enviados y recibidos.
  - Latencies Over Time (Latencias a lo Largo del Tiempo): Tiempo de espera para la primera respuesta.
  - Errors (Errores): Gráfico de errores por Sampler y a lo largo del tiempo.
- Statistics (Estadísticas):
  - Una tabla detallada (similar al Aggregate Report) con todas las métricas estadísticas (promedio, mediana, percentiles, min, max, errores, throughput) para cada Sampler (etiqueta de transacción).
- Errors (Errores):
  - Una tabla que lista los errores específicos que ocurrieron, con el código de respuesta y el mensaje.

© JMA 2020. All rights reserved

Respondedores de informe

- El receptor de resultados gráficos genera un gráfico simple que representa todos los tiempos de muestra. En la parte inferior del gráfico, la muestra actual (negro), el promedio actual de todas las muestras (azul), la desviación estándar actual (rojo) y la tasa de rendimiento actual (verde) se muestran en milisegundos.
- El informe agregado crea una fila de tabla para cada solicitud con un nombre diferente en la prueba. Para cada solicitud, totaliza la información de respuesta y proporciona el recuento de solicitudes, mínimo, máximo, promedio, tasa de error, rendimiento aproximado (solicitud / segundo) y rendimiento de Kilobytes por segundo. Una vez que se realiza la prueba, el rendimiento es el real a lo largo de toda la prueba.
- El gráfico agregado es similar al informe agregado. La principal diferencia es que el gráfico agregado proporciona una manera fácil de generar gráficos de barras exportables a PNG.
- El informe de resumen crea una fila de tabla para cada solicitud con un nombre diferente en su prueba. Similar al Informe agregado, pero usando menos memoria.
- El gráfico de tiempo de respuesta dibuja un gráfico de líneas que muestra la evolución del tiempo de respuesta durante la prueba, para cada solicitud etiquetada. Si existen muchas muestras para la misma marca de tiempo, se muestra el valor medio.

© JMA 2020. All rights reserved

https://www.soapui.org

SOAPUI

© JMA 2020. All rights reserved

Contenidos

- Instalación
- Estructura de proyectos
- Preparación de la prueba
- Service Mocking
- Pruebas funcionales: Pasos, Aserciones, Propiedades
- Ejecución
- Pruebas de carga
- Informes
- Data Driven Testing
- TestRunner Command-Line

© JMA 2020. All rights reserved

Introducción

- SoapUI es una herramienta para probar servicios web que pueden ser servicios web SOAP, servicios web RESTful u otros servicios basados en HTTP.
- SoapUI es una herramienta de código abierto y completamente gratuita con una versión comercial, ReadyAPI (anteriormente SoapUI Pro), que tiene una funcionalidad adicional para empresas con servicios web de misión crítica.
- SoapUI se considera el estándar de facto para las Pruebas de servicio API. Esto significa que hay mucho conocimiento en la red sobre la herramienta y blogs para obtener más información sobre el uso de SoapUI en la vida real.
- SoapUI permite realizar pruebas funcionales, pruebas de rendimiento, pruebas de interoperabilidad, pruebas de regresión y mucho más. Su objetivo es que la prueba sea bastante fácil de comenzar, por ejemplo, para crear una Prueba de carga, simplemente hay que hacer clic derecho en una prueba funcional y ejecutarla como una prueba de carga.
- SoapUI puede simular servicios web (mocking). Se puede grabar pruebas y usarlas más tarde.
- SoapUI puede crear apéndices de código desde el WSDL. Incluso puede crear especificaciones REST (WADL) a partir de la comunicación grabada.

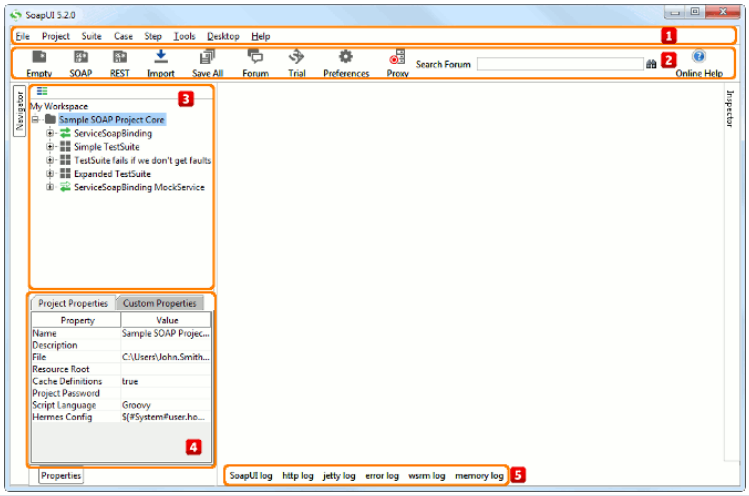
© JMA 2020. All rights reserved

Instalación

- SoapUI está basado en Java, por lo que se ejecuta en la mayoría de los sistemas operativos. Ha sido probado en varias versiones de Windows, así como en Mac y múltiples dialectos de Linux.
- SoapUI requiere una versión 1.6+ de JRE (Java Runtime Environment), se recomienda al menos 1 GB de memoria y aproximadamente 100 MB de espacio en disco.
- Para descargar e instalar accedemos a su página oficial:
  - https://www.soapui.org/downloads/soapui.html
- Si se está instalando con el instalador o las distribuciones independientes, el JRE está incluido y no es necesario en el sistema. De lo contrario, hay que asegúrese de que esté instalado y la variable de entorno JAVA_HOME esté configurada.

© JMA 2020. All rights reserved

Interfaz de SoapUI



© JMA 2020. All rights reserved

Log

- Las diferentes pestañas de log disponibles con las siguientes:
  - SoapUI log: Notificaciones generales y mensajes.
  - Http log: Muestra los datos enviados y recibidos por http. Deshabilitado durante las pruebas de stress.
  - Jetty log: Relacionado con las notificaciones de estado del mock-service.
  - Script log: Los scripts lanzan estos mensajes usando el objeto log disponible (está deshabilitado durante las pruebas de stress pero puede ser habilitado desde /File/Preferencess/UI settings).
  - Error log: Es un log con información sobre los errores ocurridos durante la ejecución. No tienen porque ser solo errores de soapUI, sino que pueden ser producidos por algún servicio o servidor que no esté disponible.
  - Memory Log: Muestra información del uso de la memoria.
- SoapUI utiliza log4j para crear los logs, es posible adaptar la configuración de log4j, renombrando el archivo log4j.xml, llamándolo “soapui-log4j.xml” y posteriormente moviéndolo al directorio bin de soapUI.

© JMA 2020. All rights reserved

Espacio de trabajo

- En SoapUI, el trabajo se organiza en espacios de trabajo y proyectos. Un proyecto puede contener cualquier número de pruebas funcionales, pruebas de carga y simulaciones de servicio requeridas para los propósitos de prueba.
- SoapUI ofrece dos formatos de proyecto:
  - Proyectos independientes (predeterminado): los almacena como un único archivo XML que contiene todos los artefactos del proyecto, como interfaces, pruebas, servicios simulados, scripts, etc.
  - Proyectos compuestos (ReadyAPI): para equipos de prueba, esto permite que varias personas trabajen en el proyecto al mismo tiempo.

© JMA 2020. All rights reserved

Proyectos

- Los proyectos SOAP se pueden crear desde un archivo WSDL o una llamada de servicio individual. Se puede usar estos proyectos para probar cada aspecto de sus servicios SOAP, verificar que los servicios sean compatibles con los estándares de uso común como WS-Security, WS-Addressing y MTOM, crear pruebas funcionales y de carga, y mucho más.
- Los proyectos REST se pueden crear desde un archivo WADL o, directamente, URI y sus parámetros. Se puede usar estos proyectos para probar servicios RESTful, crear varias solicitudes y verificar la información que recibe, probar una multitud de métodos y operaciones, etc.
- Se pueden importar proyectos existentes locales, compuestos, comprimidos o remotos. Al importar, SoapUI lo comprueba para verificar que sea coherente y que tenga todas las dependencias externas necesarias disponibles. Este proceso se llama resolución .

© JMA 2020. All rights reserved

Propiedades del proyecto

- La propiedad de raíz de recursos controla cómo SoapUI maneja las rutas para los recursos del proyecto.
  - Un camino absoluto: Usa este camino absoluto.
  - $ {projectDir}: Resuelve archivos relativos a la carpeta del proyecto.
  - $ {workspaceDir}: Resuelve archivos relativos a la carpeta que contiene el archivo del espacio de trabajo
- Se pueden crear propiedades personalizadas.
- Se puede usar la propiedad del proyecto Contraseña del proyecto para cifrar todo el contenido del archivo del proyecto (el icono del proyecto contendrá un pequeño carácter E para indicar que se ha cifrado). Al abrir el proyecto SoapUI pedirá dicha contraseña. Para eliminar el cifrado, hay que borrar el valor de la contraseña del proyecto y guardar el proyecto. SoapUI no proporciona ningún medio para recuperar un archivo de proyecto cifrado si se ha perdido la contraseña.

© JMA 2020. All rights reserved

Estructura del proyecto

- Interfaces
  - SOAP
  - REST
- Suite de pruebas (TestSuite)
  - Casos de prueba
    - Pasos de la prueba (pruebas funcionales)
    - Pruebas de carga
    - Pruebas de seguridad
- Simulaciones de servicio
  - SOAP MockServer
  - REST MockServer

© JMA 2020. All rights reserved

Preparación de la prueba

- Importar definición y configurar endpoints:
  - Añadir WSDL (http://www.dneonline.com/calculator.asmx?wsdl)
  - Añadir WADL
  - Importar Swagger (https://petstore.swagger.io/v2/swagger.json)
- Explorar el servicio:
  - Cada servicio basado en WSDL expone una serie de operaciones que tienen un formato de mensaje de solicitud y respuesta (ambos opcionales).
  - Cada servicio REST expone una serie de URLs y el método de solicitud determina la operación a realizar. Las básicas son: POST, GET, PUT y DELETE; aunque SoapUI también admite solicitudes HEAD, OPTIONS, TRACE, PATCH, PROPFIND, LOCK, UNLOCK, COPY y PURGE.
- Preparar y probar peticiones al servicio
  - Para invocar una operación, puede agregar cualquier número de objetos de solicitud a una operación en el árbol del navegador.

© JMA 2020. All rights reserved

Explorar el servicio

- Si al crear el proyecto se activa "Create Request" muestrea las posibles peticiones extraídas de las definiciones.
- Todas las peticiones se pueden completar con:
  - Auth: permite establecer el mecanismo de autenticación y la identidad con la que se realizará la petición.
  - Encabezados HTTP: permite agregar los valores a los encabezado de la solicitud
  - Adjuntos: permite agregar ficheros que se enviaran adjuntos con la solicitud
- Las peticiones SOAP adicionalmente se pueden completar con:
  - WS-A: las propiedades utilizadas para agregar encabezados WS-A a Request / MockResponse de acuerdo con la especificación WS Addressing.
  - WS-RM: las propiedades utilizadas para configurar y usar una secuencia WS-RM para la solicitud de acuerdo con la especificación de WS Reliable Messaging.

© JMA 2020. All rights reserved

Parámetros REST

- Tipos de parámetros:
  - CONSULTA (QUERY): aparecen en la URL después del signo de interrogación (?) después del nombre del recurso (nombre=valor).
  - ENCABEZAMIENTO (HEADER): se pasan en los encabezados de las solicitudes salientes (encabezado: valor).
  - PLANTILLA (TEMPLATE): estos parámetros aparecen en la ruta encerrados entre {}.
  - MATRIX: estos parámetros también van en la URL de la solicitud. Residen entre la ruta del recurso y los parámetros QUERY, y están separados de la ruta del recurso por un punto y coma (;).
  - PLAIN: estos parámetros están presentes en el editor de solicitudes, pero SoapUI no los incluye en las solicitudes.
- Nivel de parámetro:
  - El nivel de RECURSO significa que el parámetro que cree se agregará a todos los elementos de método y solicitud en el elemento de recurso.
  - El nivel METHOD significa que el parámetro se agregará a los elementos de la solicitud debajo del elemento del método solamente. No afectará al recurso ni a otros elementos del método.

© JMA 2020. All rights reserved

Opciones de parámetros

- SoapUI utiliza las siguientes opciones para los parámetros para configurar la solapa Forms y seleccionar el valor de una lista desplegable (solo en ReadyAPI):
  - Required: Establece si los pasos de prueba deben especificar el valor del parámetro o si puede omitirlo
  - Type: Especifica el tipo de datos del parámetro.
  - Options: Una lista de posibles valores para el parámetro que aparecen en la lista desplegable.
  - Description: Un texto libre que describe el parámetro y proporcionar una pista rápida para el parámetro en los editores de pasos de prueba.
  - Disable Encoding: Si los valores de un parámetro van en una URL y contienen algunos símbolos especiales como espacios o barras diagonales, SoapUI de forma predeterminada reemplazará estos símbolos con sus códigos (como %20 o %2F). En determinados casos, es posible que se desee desactivar dicha codificación marcando la casilla.

© JMA 2020. All rights reserved

Representaciones

- Establece los diferentes formatos MIME en el que se puede solicitar u obtener las peticiones REST:
  - Type: Tipo de representación, puede tomar los siguientes valores:
    - REQUEST: formato del cuerpo de la solicitud
    - RESPONSE: formato del cuerpo de las respuestas correctas (2xx)
    - FAULT: formato del cuerpo de las respuestas fallidas (4xx y 5xx)
  - Media-Type: Formato MIME
  - Status Codes: Código numérico del estado HTTP (no aplicable en el tipo REQUEST)

© JMA 2020. All rights reserved

Pruebas funcionales

- En SoapUI, las pruebas funcionales se pueden usar para validar requisitos funcionales, tanto para invocar los Servicios Web propios (=”pruebas unitarias”) como para una secuencia de peticiones (= " pruebas de integración"). Además, es posible añadir lógica a las pruebas mediante scripts de Groovy, lo que permite, por ejemplo, interactuar con una base de datos o realizar un flujo de pruebas complejo.
- Las pruebas funcionales, en SoapUI, se pueden usar para una variedad de propósitos:
  - Pruebas Unitarias: valida que cada operación del Servicio Web funciona como se indica
  - Pruebas de compatibilidad: valida que el resultado devuelto por el Servicio Web es compatible con su definición
  - Prueba de procesos: valida que una secuencia de invocaciones de Servicios Web ejecuta un proceso de negocio requerido
  - Pruebas guiadas por datos: valida que cualquiera de los anteriores funciona como requerimiento de datos de entrada procedentes de fuentes externas (por ejemplo, una base de datos u otro servicio Web)

© JMA 2020. All rights reserved

Pruebas funcionales

- SoapUI soporta pruebas funcionales de Servicios Web suministrando un caso de prueba con un número de pasos que pueden ser ejecutados en secuencia. En la actualidad, hay seis tipos de pasos que proporcionan muchas posibilidades de prueba. Los casos de prueba están organizados en un grupo de pruebas y en un mismo proyecto se pueden crear varios grupos de pruebas.
- SoapUI estructura las pruebas funcionales en tres niveles: TestSuites, TestCases y TestSteps.
- Un TestSuite es una colección de TestCases que se pueden usar para agrupar pruebas funcionales en unidades lógicas. Se puede crear cualquier número de TestSuites dentro de un proyecto de SoapUI para admitir escenarios de pruebas masivas.
- Un TestCase es una colección de TestSteps que se ensamblan para probar algunos aspectos específicos de sus servicios. Puede agregar cualquier número de TestCases a el TestSuite que lo contenga e incluso modularizarlos para llamarse entre sí para escenarios de prueba complejos.
- TestSteps son los "bloques de construcción" de las pruebas funcionales en SoapUI. Se agregan a un TestCase y se utilizan para controlar el flujo de ejecución y validar la funcionalidad de los servicios que se probarán.

© JMA 2020. All rights reserved

Tipos de Paso

- SOAP Request: Envía una petición SOAP y permite que la respuesta sea validada usando una variedad de aserciones
- REST Request: Ejecuta una Request Rest definida en el proyecto
- HTTP Request: Ejecuta una llamada http
- JDBC Request: Ejecuta una petición a una BBDD
- AMF Request: Ejecuta una petición AMF (es el formato de mensajería Adobe ActionScript utilizado por las aplicaciones Flash / Flex para interactuar con un servidor back-end).
- GraphQL Request: Ejecuta una consulta o una mutación en GraphQL.
- Properties: Se utiliza para definir propiedades globales que pueden ser leídas desde una fuente externa.
- Property Transfer: Utilizadas para transferir los valores de propiedad entre dos TestSteps
- Run TestCase Step: Ejecuta otro TestCase dentro de uno ya existente
- Conditional Goto: Permite cualquier número de saltos condicionales en la ruta de ejecución del TestCase.
- Delay Step: Pausa la ejecución de una TestCase durante un número especificado de milisegundos.
- Groovy Script: Ejecuta un script Groovy que puede hacer más o menos "cualquier cosa"
- Mock Response: Escucha un petición que se valida y devuelve una respuesta mock
- Manual TestStep: Se utiliza cuando se necesita interacción manual del tester dentro de la prueba
- Message Queuing Telemetry Transport: Receive MQTT Message, Publish using MQTT, Drop MQTT Connection

© JMA 2020. All rights reserved

TestRequests

- Los TestRequests son una de las principales características cuando se trabaja con SoapUI.
- Extiende peticiones estándar con la posibilidad de añadir cualquier número de aserciones que se aplicarán a la respuesta recibida por la petición. Esto es, comprueba que la respuesta contenga lo que se espera que contenga.
- Los TestRequests son enviados manualmente a través de las acciones de envío de los editores o cuando se ejecuta el TestCase que contiene la petición.
- La respuesta de la petición es validada contra las aserciones de peticiones y el icono de la petición cambia para reflejar el resultado de la validación; verde significa que todas las validaciones fueron bien y rojo que algunas fallaron.
- Un icono con el fondo gris indica que la petición todavía no ha sido enviada para validar, un fondo blanco indica que los TestRequests carecen de aserciones.

© JMA 2020. All rights reserved

Petición

- SOAP Request:
  - Vinculada a operación del interfaz contiene el fragmento SOAP de la petición en XML para su edición.
- REST Request:
  - Vinculada a un método del interfaz contiene los parámetros de solicitud HTTP para la asignación de sus valores.
- HTTP Request:
  - Permite crear una petición a una URL usando el método indicado y parámetros de solicitud HTTP para la asignación de sus valores.

© JMA 2020. All rights reserved

Petición

- Todas las peticiones se pueden completar con:
  - Auth : permite establecer el mecanismo de autenticación y la identidad con la que se realizará la petición.
  - Encabezados HTTP: permite agregar los valores a los encabezado de la solicitud
  - Adjuntos: permite agregar ficheros que se enviaran adjuntos con la solicitud
- Las peticiones SOAP adicionalmente se pueden completar con:
  - WS-A : las propiedades utilizadas para agregar encabezados WS-A a Request / MockResponse de acuerdo con la especificación WS Addressing.
  - WS-RM : las propiedades utilizadas para configurar y usar una secuencia WS-RM para la solicitud de acuerdo con la especificación de WS Reliable Messaging.

© JMA 2020. All rights reserved

Respuesta

- Una vez ejecutada manualmente la petición se mostrara la respuesta. Admite diferentes formatos: XML, JSON, HTML, Raw.
- Así mismo permite inspeccionar:
  - Encabezados HTTP : muestra los encabezado HTTP de la respuesta.
  - Adjuntos: muestra los adjuntos de la respuesta asociada.
  - Información SSL: muestra la respuesta detallada SSL para la respuesta actual.
  - WS-A: muestra las propiedades del encabezado WS-Addressing.
  - WSS: muestra las propiedades del encabezado WS-Security

© JMA 2020. All rights reserved

Aserciones

- Las aserciones se utilizan para validar el mensaje recibido por un TestStep durante la ejecución, generalmente comparando partes del mensaje (o el mensaje completo) con algún valor esperado.
- Se puede agregar cualquier cantidad de aserciones a una muestra de TestStep, cada una validando algún aspecto o contenido diferente de la respuesta.
- Después de que se ejecuta una muestra TestStep, todas las aserciones se aplican a la respuesta recibida y si alguna de ellas falla, el TestStep se marca como fallido en la vista del TestCase y se muestra una entrada FALLO correspondiente en el Registro de ejecución de prueba.
- Las aserciones se pueden deshabilitar para que no se apliquen.
- Las aserciones se dividen en varias categorías para facilitar la gestión. Solo se habilitan las categorías que contienen aserciones aplicables para un tipo específico de muestra.

© JMA 2020. All rights reserved

Categoría: contenido de propiedad

- Contains: busca la existencia de un token de cadena en el valor de la propiedad, admite expresiones regulares. Aplicable a cualquier propiedad.
- Not Contains: busca la inexistencia de un token de cadena en el valor de la propiedad, admite expresiones regulares. Aplicable a cualquier propiedad.

- XPath Match: utiliza una expresión XPath para seleccionar contenido de la propiedad de destino y compara el resultado con un valor esperado. Aplicable a cualquier propiedad que contenga XML.
- XQuery Match: utiliza una expresión XQuery para seleccionar contenido de la propiedad de destino y compara el resultado con un valor esperado. Aplicable a cualquier propiedad que contenga XML.

© JMA 2020. All rights reserved

Categoría: contenido de propiedad

- JsonPath Count: usa una expresión JsonPath para contar las ocurrencias de un elemento. Aplicable a cualquier propiedad que contenga JSON.
- JsonPath Existence Match: utiliza una expresión JsonPath para seleccionar contenido de la propiedad de destino y compara el resultado con un valor esperado. Aplicable a cualquier propiedad que contenga JSON.
- JsonPath Match: utiliza una expresión JsonPath para seleccionar contenido de la propiedad de destino y compara el resultado con un valor esperado. Si los valores coinciden con las afirmaciones pasadas supera la prueba, de lo contrario falla. Aplicable a cualquier propiedad que contenga JSON.
- JsonPath RegEx Match: utiliza una expresión JsonPath para seleccionar contenido de la propiedad de destino y compara el resultado con una expresión regular especificada. Aplicable a cualquier propiedad que contenga JSON.

© JMA 2020. All rights reserved

Categoría: Cumplimiento, estado y estándares

- HTTP Download all resource: descarga todos los recursos referidos como un documento HTML (imágenes, scripts, etc.) y valida que estén disponibles. Aplicable a cualquier propiedad que contenga HTML.
- Valid HTTP Status Codes: comprueba que se recibió un resultado HTTP con un código de estado en la lista de códigos definidos. Aplicable a cualquier TestStep que recibe mensajes HTTP.
- Invalid HTTP Status Codes: comprueba que se recibió un resultado HTTP con un código de estado que no figura en la lista de códigos definidos. Aplicable a cualquier TestStep que recibe mensajes HTTP
- SOAP Response: valida que la última respuesta recibida es una respuesta SOAP válida. Aplicable solo a los pasos de solicitud de prueba SOAP.
- SOAP Request: valida que la última solicitud recibida es una solicitud SOAP válida. Aplicable solo a MockResponse TestSteps.
- Not SOAP Fault: valida que el último mensaje recibido no es un fallo SOAP. Aplicable a SOAP TestSteps.

© JMA 2020. All rights reserved

Categoría: Cumplimiento, estado y estándares

- SOAP Fault: valida que el último mensaje recibido es un fallo SOAP. Aplicable a la solicitud SOAP TestSteps
- Schema Compliance: valida que el último mensaje recibido sea compatible con la definición de esquema WSDL o WADL asociada. Aplicable a SOAP y REST TestSteps.
- WS-Addressing Request: valida que la última solicitud recibida contiene encabezados de direccionamiento WS válidos. Aplicable solo a MockResponse TestSteps.
- WS-Addressing Response: valida que la última respuesta recibida contiene encabezados de WS-Addressing válidos. Aplicable solo a los pasos de solicitud de prueba SOAP.
- WS-Security Status: valida que el último mensaje recibido contenía encabezados WS-Security válidos. Aplicable a SOAP TestSteps.

© JMA 2020. All rights reserved

Otras categorías

- Script
  - Script Assertion: ejecuta una secuencia de comandos personalizada para realizar validaciones arbitrarias. Aplicable solo a TestSteps (es decir, no a las propiedades).
- SLA
  - Response SLA: valida que el último tiempo de respuesta recibido estuvo dentro del límite definido. Aplicable a Script TestSteps y TestSteps que envían solicitudes y reciben respuestas.
- JMS
  - JMS Status: valida que la solicitud JMS del TestStep objetivo se ejecutó correctamente. Aplicable a Request TestSteps con un punto final JMS.
  - JMS Timeout: valida que la instrucción JMS del TestStep de destino no tardó más de la duración especificada. Aplicable a Request TestSteps con un punto final JMS.
- JDBC
  - JDBC Status: valida que la instrucción JDBC del TestStep objetivo se ejecutó correctamente. Aplicable solo a JDBC TestSteps.
  - JDBC Timeout: valida que la instrucción JDBC del TestStep de destino no tardó más de la duración especificada. Aplicable solo a JDBC TestSteps.
- Security
  - Sensitive Information Exposure: comprueba que el último mensaje recibido no expone información confidencial sobre el sistema de destino. Aplicable a REST, SOAP y HTTP TestSteps.

© JMA 2020. All rights reserved

Verificación SOAP

- SOAP Response
- Schema Compliance
- XPath Match
  - XPath Expression:

    declare namespace ns1='http://tempuri.org/';

    //ns1:AddResult
- XQuery Math
  - XQuery Expression:

    count(//Row)
- SOAP Fault

© JMA 2020. All rights reserved

Verificación REST

- JsonPath Count
  - JsonPath Expression: $[*]
- JsonPath RegEx Match:
  - JsonPath Expression: $[0]
- JsonPath Existence Match
  - JsonPath Expression: $.id
- JsonPath RegEx Match:
  - JsonPath Expression: $.id
  - Regular Expression: \d
- Valid HTTP Codes:
  - Codes: 200, 201, 204
- Schema Compliance

© JMA 2020. All rights reserved

Propiedades

- Las propiedades son un aspecto central de las pruebas más avanzadas con SoapUI.
- En lo que respecta a las propiedades de pruebas funcionales, se utilizan para parametrizar la ejecución y la funcionalidad de sus pruebas, por ejemplo:
  - Las propiedades se pueden utilizar para mantener los puntos finales de sus servicios, lo que facilita el cambio de los puntos finales reales utilizados durante la ejecución de la prueba (consulte el ejemplo a continuación).
  - Las propiedades se pueden usar para mantener las credenciales de autenticación, lo que facilita su administración en un lugar central o en un archivo externo.
  - Las propiedades se pueden usar para transferir y compartir identificadores de sesión durante la ejecución de la prueba, por lo que múltiples pasos de prueba o casos de prueba pueden compartir las mismas sesiones.
- Las propiedades se pueden definir en varios niveles en SoapUI:
  - En el nivel Proyecto, TestSuite y TestCase
  - En un Properties TestStep
  - En un DataGen TestStep
  - Como parte de una configuración de TestStep (DataSource TestStep y DataSink TestStep)
- Además, la mayoría de los otros TestSteps exponen propiedades para lectura y escritura, por ejemplo:
  - Script TestStep expone una propiedad "Result" que contiene el valor de cadena devuelto por el último script ejecutado.
  - Todos los pasos de Request exponen una propiedad con la última respuesta recibida.
- Las propiedades pueden leerse y escribirse fácilmente desde scripts y también transferirse entre TestSteps con Property-Transfer TestStep

© JMA 2020. All rights reserved

Properties TestStep

- El Properties TestStep se utiliza para definir propiedades personalizadas que se utilizarán dentro de un TestCase.
- Sus principales ventajas sobre la definición de propiedades en el nivel TestCase son:
  - Se pueden organizar las propiedades en varios Properties TestStep (si se tienen muchas de ellas).
  - Se puede especificar los nombres de los archivos de origen y destino que se usarán para leer y escribir las propiedades contenidas cuando se ejecute TestStep. En formato .properties:

    Propiedad1=Valor1

    Propiedad2=Valor2

© JMA 2020. All rights reserved

Property Transfer TestStep

- Los Property Transferes son TestStep que transfieren propiedades entre los contenedores de propiedades dentro del mismo alcance que el Property Transfer TestStep (es decir, que contenga su TestCase, su TestSuite, el proyecto y las propiedades del destino).
- El paso puede contener un número arbitrario de "transferencias" especificando una propiedad de origen y destino con expresiones opcionales de XPath/XQuery.
- Property Transferes utiliza el mismo motor de Saxon XPath/XQuery descrito para las aserciones XPath y XQuery.
- Tras la ejecución de un TestCase, cada transferencia se realiza mediante la selección de las propiedades especificadas por el paso de origen, por la propiedad y expresión XPath opcional y copiando su valor a la propiedad especificada por el paso de destino pudiendo usar una expresión Xpath opcional.
- Si las expresiones XPath están especificadas, SoapUI tratará de sustituir el nodo destino con el nodo de origen si son del mismo tipo. Si no (por ejemplo, cuando se asigna texto () a un @atributo), SoapUI hará todo lo posible para copiar el valor.
- El origen de la transferencia puede ser la respuesta de un paso anterior.
- El destino puede ser el contenido de un paso posterior, pero ser mas cómodo el uso de la expansión de propiedades.

© JMA 2020. All rights reserved

Propiedades de Expansión

- SoapUI proporciona una sintaxis común para insertar dinámicamente ("expandir") valores de propiedad durante el procesamiento. La sintaxis es la siguiente:
  - ${[scope]propertyName[#xpath-expression]}
- donde alcance puede ser uno de los siguientes valores literales:
  - #Project#: hace referencia a una propiedad del Proyecto
  - #TestSuite#: hace referencia a una propiedad en el TestSuite que lo contiene
  - #TestCase#: hace referencia a una propiedad en el TestCase que lo contiene
  - #MockService#: hace referencia a una propiedad en el MockService que lo contiene
  - #Global#: hace referencia a una propiedad global en todos los proyectos. En File>Preferences>Global Properties.
  - #System#: hace referencia a una propiedad del sistema. En Help>System properties.
  - #Env#: hace referencia a una variable de entorno
  - [Nombre de TestStep]#: hace referencia a una propiedad TestStep
- Si no se especifica ningún alcance, la propiedad se resuelve de la siguiente manera:
  - Mira en el contexto actual (por ejemplo, TestRunContext) una propiedad con el nombre coincidente
  - Busca una propiedad global coincidente
  - Comprueba si hay una propiedad del sistema coincidente

© JMA 2020. All rights reserved

Propiedades de Expansión

- Si la expansión de la propiedad incluye además una expresión XPath, se usará para seleccionar el valor correspondiente del valor de la propiedad referenciada (que debe contener XML):
  - ${Search Request#Response#//ns1:Item[1]/n1:Author[1]/text()}
- SoapUI 2.5 introdujo la posibilidad de escribir scripts directamente dentro de una PropertyExpansion; si se prefija el contenido con un '=' el contenido restante hasta la llave de cierre se evaluará como un script y se insertará su resultado:
  - ${=(int)(Math.random()*1000)}
- Dependiendo del contexto de la expansión, las variables relevantes estarán disponibles para acceder al modelo de objeto SoapUI.
  - $ {= request.name}
- Las siguientes variables están (casi) siempre disponibles en estos scripts:
  - log: un Logger log4j que registra en la ventana de registro
  - modelItem: el modelItem actual (por ejemplo, Request, MockResponse, etc.).
  - context: el contexto de ejecución actual (por ejemplo, un TestCase o MockService)

© JMA 2020. All rights reserved

JDBC Request

- SoapUI 3.5 presento un nuevo TestStep para recuperar datos de una base de datos utilizando JDBC. El resultado está formateado como XML y se puede afirmar o procesar de la manera estándar (XPath , Xquery, …), pero también hay dos aserciones adicionales disponibles:
  - Afirmación de estado JDBC
  - La aserción JDBC Timeout verificará que el SQL se ejecutó dentro del tiempo de espera configurado.
- Para utilizar JDBC TestStep se deberá agregar el controlador JDBC a la carpeta soapui_home/bin/ext y reiniciar la aplicación.
- Para configurar la conexión:
  - Driver: com.mysql.jdbc.Driver
  - Connection String: jdbc:mysql://localhost:3306/sakila?user=root&password=root
- La consulta SQL, los parámetros pueden ser referenciados mediante la expansión de propiedades:
  - SELECT count(*) FROM `sakila`.`actor` WHERE `actor_id`=:id

© JMA 2020. All rights reserved

Run TestCase TestStep

- Si los escenarios de prueba se vuelven cada vez más complejos, es posible que se desee compartir una serie de TestSteps entre diferentes TestCases, tal vez para configurar algunos requisitos previos (inicio de sesión, etc.) o para realizar ciertos pasos en diferentes contextos.
- Run TestCase TestStep presenta un enfoque flexible para la modularización; ejecuta el TestCase objetivo configurado opcionalmente, pasando y recuperando propiedades antes y después de la ejecución, por ejemplo, se podría usar esto para llamar a una secuencia compleja de TestSteps para realizar y validar un procedimiento de inicio de sesión, pasando las credenciales requeridas y recibiendo el sessionid resultante.

© JMA 2020. All rights reserved

Conditional Goto TestStep

- Los pasos de condicional Goto evalúan un número arbitrario de condiciones XPath del mensaje de respuesta de la petición anterior y transfieren la ejecución del TestCase al TestStep asociado con la primera condición que se evalúe a verdadero.
- Esto permite la ejecución condicional de rutas de TestCase, donde el resultado de algunas peticiones controla cómo moverse por el TestCase.
- Si no hay condiciones que coincidan con la respuesta actual, la ejecución del TestCase continúa después del paso Goto de forma habitual.
- Ejemplo de escenarios:
  - Ramificaciones que dependen de los resultados devueltos por una petición
  - Reiniciar después de un largo retraso en las pruebas de vigilancia
  - En varias ocasiones esperar y chequear el valor de estado antes de continuar (por ejemplo en un proceso por lotes)
- Las condiciones usan el mismo motor que Saxon XPath descrito para las aserciones Xpath (una condición debe devolver un valor booleano para ser válida)

© JMA 2020. All rights reserved

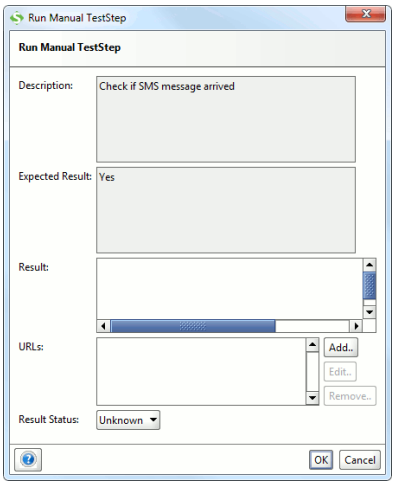
Delay TestStep

- Los TestStep de retardo se pueden insertar en cualquier posición en un TestCase para pausar la ejecución de un TestCase durante un número determinado de milisegundos.
- Ejemplos de escenarios:
  - Repetir varias veces el TestCase o determinados TestStep después de una demora de tiempo.
  - Demorar los siguientes pasos para dar tiempo a que un proceso ajeno se complete.

© JMA 2020. All rights reserved

Manual TestStep

- Manual TestStep se utiliza cuando se necesita interacción manual del tester dentro de la prueba. Algunos ejemplos:
  - Iniciar dispositivos como teléfonos movil
  - Realizar una acción en el sitio web o en el servidor
  - Verificar manualmente la base de datos / registros para
- Para definirlo se puede especificar una descripción y el resultado esperado. Cuando la ejecución del caso de prueba alcanza un paso de prueba Manual, se muestra una ventana emergente con la descripción y el resultado esperado y permite agregar los detalles de los resultados reales y especificar el botón de opción Pasar-Fallar-N/A para establecer el estado del paso de prueba. Al hacer clic en continuar, SoapUI va al siguiente paso.
- Estos pasos de prueba manuales, se omiten cuando se inicia a través de testrunner ya que no hay forma de interactuar con ventanas emergentes.



© JMA 2020. All rights reserved

Script Step

- SoapUI ofrece amplias opciones para la creación de secuencias de comandos, utilizando Groovy o Javascript (desde SoapUI 3.0) como su lenguaje de secuencias de comandos, que se utiliza para establecer el nivel del proyecto en la pestaña de detalles del proyecto en la parte inferior izquierda.
- Todos los scripts tienen acceso a una serie de variables específicas de la situación, que siempre incluyen un objeto de registro para iniciar sesión en Groovy Log y un objeto de contexto para realizar PropertyExpansions específicas de contexto o manejo de propiedades, si corresponde.
- Una variable específica de contexto siempre está disponible para acceder directamente al modelo de objetos SoapUI.

© JMA 2020. All rights reserved

Script Step

- Los scripts se pueden usar en los siguientes lugares en SoapUI:
  - Como parte de un TestCase con Script TestStep, que permite que las pruebas realicen prácticamente cualquier funcionalidad deseada
  - Antes y después de ejecutar un TestCase o TestSuite para inicializar y limpiar antes o después de ejecutar sus pruebas.
  - Al iniciar/detener un MockService para inicializar o limpiar el estado de MockService
  - Para proporcionar despacho dinámico de MockOperation o crear contenido dinámico de MockResponse
  - Al abrir / cerrar un proyecto, para inicializar o limpiar configuraciones relacionadas con el proyecto
  - Como un DataSource o DataSink dinámico con los pasos de prueba de DataSource / DataSink correspondientes
  - Para crear aserciones arbitrarias con la aserción de script
  - Para extender SoapUI y agregar funcionalidad arbitraria al núcleo SoapUI.

© JMA 2020. All rights reserved

SOAP Mock Response

- El paso SOAP Mock Response escucha una solicitud SOAP y devuelve una respuesta preconfigurada antes de continuar. La solicitud entrante se puede validar con las mismas aserciones de las solicitudes SOAP, específicamente con SOAP Request y Schema Compliance se puede validar la petición entrante.
- Cuando la ejecución de un caso de prueba llega al paso de prueba SOAP Mock Response, la ejecución de la prueba se detiene y espera una solicitud al recurso especificado en el puerto configurado con la ruta especificada. Una vez que se recibe una solicitud, SOAP Mock Response devuelve el resultado y continua con siguiente paso en el caso de prueba.
- Dichos servicios mock temporales pueden capturar solicitudes enviadas por webhooks. De esta manera, se puede configurar rápidamente un servicio temporal para capturar una sola solicitud, o hacer que esto sea parte de la prueba sin configurar una aplicación o una API virtual separada para manejar la solicitud.

© JMA 2020. All rights reserved

Pruebas de carga

- Las pruebas de rendimiento son una de las tareas más comunes en las pruebas de servicios web. Lo que se llama una prueba de carga en SoapUI es en realidad una prueba de rendimiento.
- Las pruebas de rendimiento crean o simulan artificialmente la carga y miden cómo la maneja el entorno.
- Esto significa que no necesariamente tiene que ser el rendimiento de un sistema bajo una carga alta, sino también el rendimiento bajo la carga base o la carga esperada.
- En SoapUI, se crean las pruebas de carga sobre la base de pruebas funcionales existentes. Esto ayuda a crear rápida y fácilmente pruebas de rendimiento para el servicio web. Luego se puede validar el rendimiento del servicio web utilizando diferentes estrategias de carga, verificar que su funcionalidad no se rompa bajo carga, ejecutar varias pruebas de carga simultáneamente para ver cómo se afectan entre sí y mucho más.

© JMA 2020. All rights reserved

Pruebas de carga

- Pruebas de línea de base
  - Técnicamente hablando, esto se puede definir como pruebas de rendimiento puro. Las pruebas de línea de base examinan cómo funciona un sistema bajo la carga esperada o normal y crea una línea de base con la que se pueden comparar otros tipos de pruebas.
  - Objetivo: encontrar métricas para el rendimiento del sistema bajo carga normal.
- Prueba de carga
  - La prueba de carga incluye aumentar la carga y ver cómo se comporta el sistema bajo una carga mayor. Durante las pruebas de carga, puede monitorear los tiempos de respuesta, el rendimiento, las condiciones del servidor y más. Sin embargo, el objetivo de las pruebas de carga no es romper el entorno de destino.
  - Objetivo: encontrar métricas para el rendimiento del sistema bajo alta carga.
- Prueba de esfuerzo o estrés
  - La prueba de esfuerzo consiste en aumentar la carga hasta encontrar el volumen de carga donde el sistema realmente se rompe o está cerca de romperse.
  - Objetivo: encontrar el punto de ruptura del sistema.

© JMA 2020. All rights reserved

Pruebas de carga

- Pruebas de remojo
  - Para encontrar inestabilidades del sistema que ocurren con el tiempo, debe ejecutar pruebas durante un período prolongado. Para eso están las pruebas de remojo; ejecute pruebas de carga o incluso pruebas de línea de base durante un largo período de tiempo y vea cómo el entorno de destino maneja los recursos del sistema y si funciona correctamente. El defecto más común encontrado en las pruebas de remojo son las pérdidas de memoria. El escenario más común para las pruebas de remojo es activar una serie de pruebas cuando sale de su oficina un viernes y lo deja correr durante el fin de semana.
  - Objetivo: Asegurarse de que no surja ningún comportamiento no deseado durante un largo período de tiempo.
- Pruebas de escalabilidad
  - El propósito de las pruebas de escalabilidad es verificar si su sistema se adapta adecuadamente a la carga cambiante. Por ejemplo, un mayor número de solicitudes entrantes debería causar un aumento proporcional en el tiempo de respuesta. El aumento no proporcional indica problemas de escalabilidad.
  - Objetivo: buscar métricas y verificar si el rendimiento del sistema cambia adecuadamente a la carga.

© JMA 2020. All rights reserved

Factores que afectan al rendimiento

- Si observamos características únicas del rendimiento del servicio web, se destacan dos aspectos: una gran cantidad de procesamiento XML / JSON en el lado del servidor (análisis y serialización XML / JSON) y procesamiento de cargas (cuerpos de solicitud y respuesta). Las razones por las cuales esto falla pueden ser múltiples; puede estar en la plataforma en forma de, por ejemplo, debilidades en el servidor de aplicaciones, y en la implementación en forma de WSDL innecesariamente complejos. También podría ser que el código está haciendo una solicitud a una base de datos que responde lentamente.
- Un factor más que afecta el rendimiento con frecuencia es la seguridad. Aquellos que realizan pruebas de rendimiento web saben que los sitios HTTPS tienen un rendimiento considerablemente menor que sus análogos HTTP, y en las pruebas de servicios web, podemos agregar una capa de WS-Security a la capa de seguridad HTTP, disminuyendo aún más el rendimiento.
- La complejidad de analizar las cargas útiles de XML / JSON significa que tenemos que poner un enfoque adicional en las pruebas de escalabilidad, mientras que el problema de la seguridad significa que tendremos que enfocarnos un poco más en hacer pruebas de solicitudes que son seguras. Si todo el servicio web es seguro, esto significa que la prueba de carga es más importante, especialmente si se utiliza WS-Security y el manejo de tokens.
- También significa que tenemos que examinar la especificación API de cerca. Si las solicitudes y respuestas son complejas o grandes, o si incluyen archivos adjuntos grandes, debemos centrarnos en enfatizar esa complejidad y ver cómo se comporta bajo carga.

© JMA 2020. All rights reserved

LoadTest

- En SoapUI, los LoadTests se basan en los TestCases funcionales existentes. Un LoadTest ejecuta el TestCase repetidamente durante el tiempo especificado con un número deseado de subprocesos (o usuarios virtuales). Los LoadTests se muestran en el navegador como elementos secundarios del TestCase que usan. Se puede crear cualquier número de LoadTests para un TestCase.
- Las diferentes estrategias de carga disponibles en SoapUI permiten simular varios tipos de carga a lo largo del tiempo, lo que permite probar fácilmente el rendimiento de sus servicios de destino en una serie de condiciones. Dado que SoapUI también le permite ejecutar múltiples LoadTests simultáneamente, se puede usar una combinación de LoadTests para afirmar aún más el comportamiento de los servicios.
- Si queremos simular varios clientes consumiendo el servicio, vamos a la opción threads, por defecto aparecen 5 pero se puedan variar ese número a discreción, aunque mientras más hilos más consumo del CPU. Si queremos especificar una demora entre una petición y otra vamos a la opción delay (1000 milisegundos por defecto) y la cantidad aleatoria de la demora que se desea aleatorizar (es decir, establecerlo en 0.5 dará como resultado retrasos entre 0,5 y 1 segundo)
- Si queremos determinar el tiempo de duración del servicio vamos a la opción Limit y aquí se muestra que solo durará 60 segundos pero pueden cambiarla para especificar la cantidad de peticiones por hilo por ejemplo.

© JMA 2020. All rights reserved

Estrategias

- Estrategia simple: pruebas de línea de base, carga y remojo
  - La estrategia simple ejecuta la cantidad especificada de subprocesos con el retraso especificado entre cada ejecución para simular un respiro para el servidor.
- Estrategia de tasa fija (Fixed Rate): simple con un toque especial
  - La estrategia de tasa fija ejecuta una cantidad de subprocesos dentro de un tiempo determinado, la estrategia iniciará automáticamente el número de subprocesos requeridos intentando mantener el valor configurado.
- Estrategias de carga variable (Variance)
  - La estrategia de variación cambia el número de subprocesos a lo largo del tiempo en "dientes de sierra" según lo configurado; se establece el Intervalo en el valor deseado y la Varianza en cuánto debe disminuir y aumentar el número de subprocesos.
- Estrategias de carga variable (Burst)
  - La estrategia Burst está diseñada específicamente para pruebas de recuperación y lleva la variación al extremo. No hace nada durante el retardo de ráfaga configurado, luego ejecuta la cantidad configurada de subprocesos para la "Duración de ráfaga" y vuelve al modo de suspensión.

© JMA 2020. All rights reserved

Ejecución de LoadTest

- SoapUI permite ejecutar el LoadTest con tantos subprocesos (o usuarios virtuales) como el hardware pueda administrar, dependiendo principalmente de la memoria, la CPU, el tiempo de respuesta del servicio objetivo y otros factores. Establezca el valor deseado e inicie LoadTest con el botón Ejecutar en la barra de herramientas del editor LoadTest.
- El TestCase subyacente se clona internamente para cada subproceso configurado y se inicia en su propio contexto; scripts, transferencias de propiedades, etc. El TestCase accederá a una "copia" única de TestCase y TestSteps que evita problemas de subprocesos en el nivel TestStep y TestCase (pero no más arriba en la jerarquía).
- A medida que se ejecuta LoadTest, la tabla de estadísticas de LoadTest se actualiza continuamente con los datos recopilados cada vez que finaliza un TestCase ejecutado, lo que le permite controlar de forma interactiva el rendimiento de sus servicios de destino mientras se ejecuta LoadTest. Si el TestCase contiene un bucle o TestSteps de ejecución prolongada, las estadísticas pueden tardar un tiempo en actualizarse (ya que se recopilan cuando finaliza TestCase), en este caso, seleccione la opción "Estadísticas TestStep" en el cuadro de diálogo Opciones de LoadTest para actualizarse estadísticas en el nivel TestStep en lugar del nivel TestCase (esto requiere un poco más de procesamiento interno, por eso está desactivado de forma predeterminada).

© JMA 2020. All rights reserved

Estadísticas

- Tiempos mínimos (min), máximos (max), medios (avg) y de la última petición (last), todos en milisegundos.
- Número de ejecuciones (cnt), total de bytes (bytes), número errores (err) y el ratio de errores (rat).
- TPS (transacciones por segundo) y BPS (bytes por segundo) se pueden calcular de dos maneras diferentes (Opciones de LoadTest);
  - Según el tiempo real transcurrido (predeterminado):
    - TPS: cnt/segundos pasados, es decir, un TestCase que se ejecutó durante 10 segundos y manejó 100 solicitudes obtendrá un TPS de 10
    - BPS: bytes/tiempo transcurrido, es decir, un TestCase que se ejecutó durante 10 segundos y manejó 100000 bytes obtendrá un BPS de 10000.
  - Basado en el tiempo promedio de ejecución:
    - TPS: (1000/avg) * número de hilos, por ejemplo avg = 100 ms con diez hilos dará un TPS de 100
    - BPS: (bytes/cnt) * TPS, es decir, el número promedio de bytes por solicitud * TPS.
- Se pueden exportar a ficheros .csv

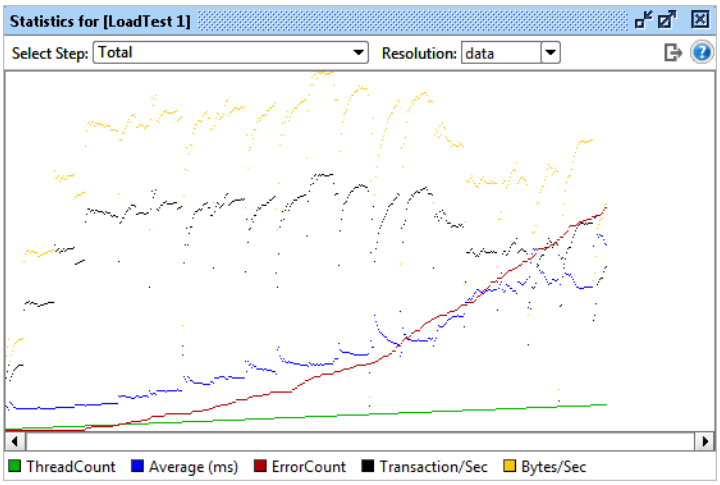
© JMA 2020. All rights reserved

Gráficos estadísticos

- Hay dos tipos de gráficos disponibles en la barra de herramientas de LoadTest durante la ejecución; estadísticas e historial de estadísticas.
- El objetivo principal de estos es visualizar estadísticas seleccionadas a lo largo del tiempo para poder detectar cambios repentinos e inesperados. La visualización de estadísticas es relativa (no absoluta) y, por lo tanto, los gráficos no son muy útiles para analizar datos exactos.
- Ambos gráficos tienen una configuración de Resolución que controla con qué frecuencia se actualiza el gráfico; establecer esto en "datos" actualizará el gráfico con el mismo intervalo que la Tabla de estadísticas (que son los datos subyacentes para el gráfico, de ahí el nombre). Alternativamente, si desea una de las resoluciones fijas, seleccione estas.
- El gráfico de estadísticas muestra todas las estadísticas relevantes para un paso seleccionado o el caso de prueba completo a lo largo del tiempo, lo que le permite ver cómo cambian los valores cuando, por ejemplo, aumenta el número de subprocesos.
- El gráfico de historial de estadísticas muestra un valor estadístico seleccionado para todos los pasos que le permite compararlos y ver si la distribución de cualquier valor entre los pasos de prueba cambia con el tiempo.
- Sus datos se pueden exportar a ficheros .csv

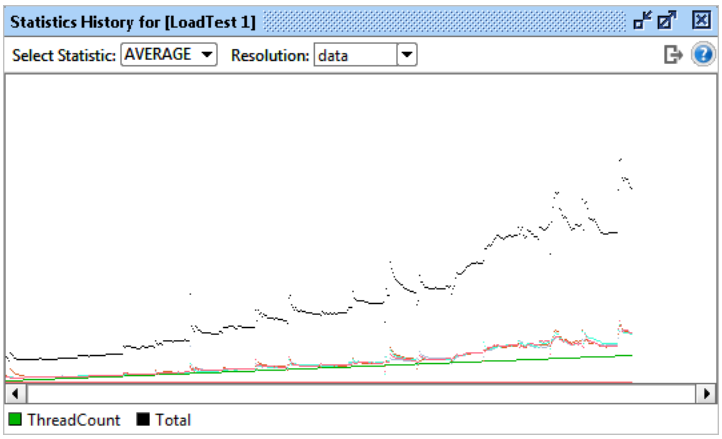
© JMA 2020. All rights reserved

Gráficos estadísticos



© JMA 2020. All rights reserved

Gráficos estadísticos



© JMA 2020. All rights reserved

Validando el rendimiento

- Al igual que puede agregar aserciones funcionales a ciertos TestSteps en un TestCase (para afirmar sus resultados), se pueden agregar aserciones a un LoadTest para validar tanto el rendimiento como la funcionalidad durante la ejecución de LoadTest. Las aserciones configuradas se aplican continuamente a los resultados de LoadTest y puede fallar el LoadTest al igual que su contraparte funcional.
- Las aserciones disponibles son:
  - Step Average: valida que el valor promedio de un TestStep o todo el TestCase no excede el límite especificado.
  - Step TPS: valida el valor de TPS (transacción por segundo) para el TestStep o TestCase correspondiente.
  - Step Maximum: valida el valor tiempo máximo para el TestStep o TestCase correspondiente.
  - Step Status: comprueba que el estado de ejecución subyacente del TestStep o TestCase sea exitoso; de lo contrario, la aserción falla.
  - Max Errors: compruebe que el número de fallos para el TestStep o TestCase correspondiente no exceda el valor configurado.

© JMA 2020. All rights reserved

Pruebas de seguridad

- Las funciones de prueba de seguridad introducidas en SoapUI 4.0 facilitan enormemente la validación de la seguridad funcional de los servicios de destino, lo que permite evaluar la vulnerabilidad del sistema para los ataques de seguridad comunes. Esto es especialmente crítico si el sistema está disponible públicamente, pero incluso si ese no es el caso, garantizar un entorno completamente seguro es igualmente importante.
- Los siguientes análisis de seguridad están disponibles actualmente:
  - Inyección SQL: intenta aprovechar la mala codificación de integración de bases de datos
  - Inyección XPath: intenta explotar el procesamiento XML incorrecto dentro de su servicio de destino
  - Escaneo de límites: intenta aprovechar el mal manejo de valores que están fuera de los rangos definidos
  - Tipos no válidos: intenta aprovechar el manejo de datos de entrada no válidos
  - XML con formato incorrecto: intenta aprovechar el mal manejo de XML no válido en su servidor o en su servicio
  - XML Bomb: intenta aprovechar el mal manejo de solicitudes XML maliciosas (tenga cuidado)
  - Adjunto malicioso: intenta aprovechar el mal manejo de los archivos adjuntos
  - Cross Site Scripting: intenta encontrar vulnerabilidades de cross-site scripting
  - Secuencia de comandos personalizada: le permite utilizar una secuencia de comandos para generar valores de fuzzing de parámetros personalizados

© JMA 2020. All rights reserved

Data Driven Testing

- Las pruebas basadas en datos son aquellas que almacenan los datos de la prueba (entrada, salida esperada, etc.) en algún almacenamiento externo (base de datos, hoja de cálculo, archivos xml, etc.) y luego usa esos datos de forma iterativa en las pruebas cuando se ejecutan.
- Las pruebas basadas en datos solo están disponibles en ReadyAPI y crearlas es realmente fácil.
- El DataSource TestStep permite la lectura de los datos de prueba en propiedades SoapUI estándar a partir de una serie de fuentes externas (archivos de Excel, propiedades XML, fuentes JDBC, archivos/directorios, etc.), estas propiedades se pueden utilizar en los siguientes TestSTEPS (solicitudes, aserciones, consultas XPath, scripts, etc.), ya sea a través de Transferencias de propiedad o Expansiones de propiedad. Finalmente un DataSource Loop TestStep permite recorrer la fuente de datos para aplicar los TestSteps anteriores para los datos de cada fila/registro del DataSource.
- Esto se reduce a la siguiente configuración:
  1. DataSource: lee datos de prueba en propiedades de una fuente externa
  2. TestSteps (cualquier número): prueba la funcionalidad del servicio utilizando las propiedades DataSource disponibles en (1)
  3. DataSource Loop: realiza un bucle de regreso a (2) para cada fila en (1)

© JMA 2020. All rights reserved

Ejecución

- Ejecución de prueba
  - Todas las vistas anteriores tienen una barra de herramientas en la parte superior con botones que ejecutan los elementos de prueba contenidos.
- Salida de la Prueba
  - Todas las vistas anteriores también contienen un registro de ejecución en la parte inferior que muestra información continua sobre los pasos de prueba ejecutados y su estado.
- Informes
  - ReadyAPI también agrega un botón "Crear informe" a la barra de herramientas superior, lo que le permite exportar los resultados de la ejecución actual a un documento, por ejemplo, un PDF.

© JMA 2020. All rights reserved

TestRunner Command-Line

- Una vez creado, es posible que se desee ejecutar los Tests desde la línea de comandos, tal vez como parte de una compilación de integración continua o para monitorear el rendimiento diario de sus servicios.
- SoapUI proporciona utilidades de línea de comandos y complementos para que maven haga esto.
- Las utilidades están disponible en la carpeta SoapUI\bin (.bat o .sh):
  - testrunner.bat
  - loadtestrunner.bat
  - securitytestrunner.bat
  - mockservicerunner.bat
- Se necesitan muchos argumentos relacionados con informes, propiedades, etc., que pueden hacer que la creación de la lista de argumentos inicial sea bastante tediosa. Afortunadamente, la interfaz de usuario incluye un asistente para iniciar LoadTestRunner, que ayudará a evaluar la salida, así como a crear una cadena de invocación de línea de comandos.

https://www.soapui.org/test-automation/running-from-command-line/functional-tests.html

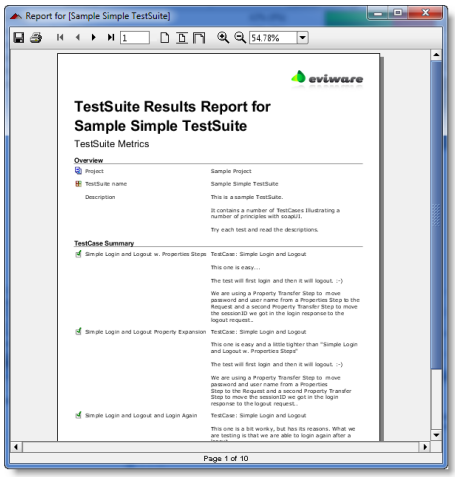
© JMA 2020. All rights reserved

Informes

- ReadyAPI proporciona tres tipos de informes que se pueden generar desde el interior de la interfaz de usuario a nivel de Proyecto, TestSuite, TestCase y LoadTest:
  - Informes imprimibles: se pueden imprimir o guardar como PDF, HTML, RTF, Excel, etc. y son totalmente personalizables tanto a nivel global como de proyecto, lo que permite crear y personalizar cualquier tipo de informe que se pueda necesitar.
  - Exportación de datos: permite la exportación de datos de informes subyacentes en formato XML y CSV. Esto es útil si se desea importar datos de informes en otras herramientas para informes personalizados o integraciones.
  - Informes HTML: brinda una descripción general simplificada de los resultados de las pruebas funcionales en formato HTML (no disponible en el nivel de LoadTest).
- Se puede incluir fácilmente cualquier dato deseado en los informes generados, ya que puede exportar datos personalizados tanto en informes imprimibles como de exportación de datos a través del SubReport DataSink.

© JMA 2020. All rights reserved

Informes



© JMA 2020. All rights reserved

SOAP Service Mocking

- La funcionalidad de SOAP Service Mocking en SoapUI le permite crear una simulación compatible con los estándares de un servicio basado en WSDL solo a partir de su contrato WSDL, que en SoapUI se denomina "MockService". Se puede ejecutar directamente desde dentro de SoapUI, con la utilidad de línea de comandos o con un contenedor de servlet estándar.
- Los escenarios de uso son muchos:
  - Prototipos rápidos de servicios web: generar una implementación simulada completa estática de un WSDL en segundos y agregar funcionalidad dinámica usando Groovy. Esto permite implementar y probar clientes mucho más rápido que si se hubiera tenido que esperar a que se desarrollara la solución real.
  - Prueba o desarrollo del cliente: crear implementaciones simuladas de las operaciones deseadas y configurar una serie de respuestas alternativas (incluidos scripts, archivos adjuntos y encabezados http personalizados). Los clientes pueden desarrollarse contra el MockService y probarse sin acceso a los servicios en vivo. Las respuestas pueden ciclarse, aleatorizarse o seleccionarse con la expresión XPath de la solicitud entrante
  - Desarrollo dirigido por pruebas: crear pruebas funcionales y de carga en SoapUI contra un MockService antes o durante la implementacíon los servicios reales
- Un MockService cumple con los estándares aceptados WSDL, SOAP o HTTP y un cliente debe poder usarlo como si fuera un servicio real.

© JMA 2020. All rights reserved

SOAP Service Mocking

- Un MockService puede simular cualquier número de contratos WSDL y la funcionalidad de scripting incorporada le permite simular básicamente cualquier tipo de comportamiento deseado; respuestas fijas, errores aleatorios, resultados dinámicos, etc. Incluso es posible implementar un servicio completo en SoapUI y desplegarlo en un contenedor de servlet estándar utilizando la funcionalidad DeployAsWar (aunque no se recomienda para uso en producción).
- MockServices proporciona la simulación del servicio al exponer un número arbitrario de MockOperations que, a su vez, pueden contener un MockRequest Dispatcher y cualquier número de mensajes MockResponse.
- Cuando el MockService recibe una solicitud SOAP y se envía a una MockOperation específica y se selecciona la MockResponse correspondiente en función del MockRequest Dispatcher configurado.
- Cada MockOperation se corresponde a una Operación WSDL de un Servicio WSDL del proyecto de prueba, y los mensajes MockResponse contenidos se crean a partir del esquema asociado con la operación. Esto no significa que un MockResponse deba cumplir con el esquema asociado, puede devolver cualquier mensaje de texto o XML arbitrario y, por ejemplo, se puede configurar para que devuelva un mensaje de respuesta para una operación completamente diferente, lo que permite probar en los clientes la 'capacidad para manejar mensajes de respuesta no válidos e inesperados.

© JMA 2020. All rights reserved

SOAP Service Mocking

- El MockResponse es el mensaje que MockService debe devolver al cliente que realiza la llamada. MockResponse puede contener contenido personalizado, encabezados y archivos adjuntos, lo que le permite simular cualquier tipo de respuesta HTTP válida (o no válida), y las posibilidades de secuencias de comandos adicionales permiten agregar fácilmente contenido dinámico a la respuesta saliente. Los MockResponse se pueden generar desde los servicios reales.
- Una vez que MockService ha recibido una solicitud y se ha enviado a una MockOperation, SoapUI debe seleccionar el método de respuesta correcto y devolverlo al cliente. Hay varios métodos de envío diferentes:
  - SECUENCIA: este es el método de envío más simple; las respuestas se seleccionan y devuelven en el orden en que se agregaron al MockOperation.
  - ALEATORIO: casi tan simple, este despachador selecciona qué respuesta usar al azar, con el tiempo todas las respuestas se habrán devuelto la misma cantidad de veces.
  - QUERY-MATCH: es bastante versátil, puede devolver diferentes respuestas en función del contenido de la solicitud.
  - XPATH: similar a QUERY_MATCH pero no tan potente, solo permite una condición.
  - SCRIPT: la opción más versátil y más difícil de dominar, se crea un script que debería devolver el nombre de MockResponse para usar.

© JMA 2020. All rights reserved

SOAP Service Mocking

- Al hacer clic en el botón Ejecutar , MockService se inicia inmediatamente dentro de SoapUI, se puede consultar jetty log.
- El MockService ahora está listo para manejar las solicitudes SOAP entrantes en la ruta y el puerto configurados. Estas se enviarán a la correspondiente MockOperation en función de su contenido, si no hay disponible ninguna MockOperation coincidente, se devolverá un fallo SOAP estándar.
- El registro de mensajes en la parte inferior muestra todos los mensajes que ha recibido el MockService desde que se inició por última vez y permite consultar, a través del visor de mensajes, la petición recibida así como la respuesta emitida.
- Además, también expone el WSDL de MockServic en la URL estándar de WSDL; por ejemplo, introduciendo http://localhost:8088/mockSampleServiceBinding?WSDL en un navegador se mostrará el WSDL.

© JMA 2020. All rights reserved

REST Service Mocking

- De forma similar a los SOAP Service Mocking, la función de simulación del servicio REST permite simular un servicio REST creando un servicio simulado. Luego se puede ejecutar directamente desde SoapUI o usar la aplicación de línea de comandos mockservicerunner.bat
- Un servicio simulado simula un servicio en vivo al exponer un cierto número de acciones simuladas (método y URL). Las acciones simuladas (Mock Action), a su vez, contienen una serie de respuestas simuladas.
- Las respuestas simuladas permiten establecer el HTTP Status Code, los valores de las cabeceras, el contenido (response body) y su media-type (content-type).
- El MockRequest Dispatcher esta limitado a SECUENCIA y SCRIPT.
- Los REST Service Mocking se ejecutan y supervisan de la misma forma que los SOAP Service Mocking.

© JMA 2020. All rights reserved

ANEXOS

© JMA 2020. All rights reserved

GenerateData

- GenerateData es una herramienta para la generación automatizada de juegos de datos.
- Ofrece ya una serie de datos precargados en BBDD y un conjunto de tipos de datos bastante amplio, así como la posibilidad de generar tipos genéricos.
- Podemos elegir en que formato se desea la salida de entre los siguientes:
  - CSV
  - Excel
  - HTML
  - JSON
  - LDIF
  - Lenguajes de programación (JavaScript, Perl, PHP, Ruby, C#)
  - SQL (MySQL, Postgres, SQLite, Oracle, SQL Server)
  - XML
- Online: http://www.generatedata.com/?lang=es
- Instalación (PHP): http://benkeen.github.io/generatedata/

© JMA 2020. All rights reserved

Mockaroo

https://www.mockaroo.com/

- Mockaroo permite descargar rápida y fácilmente grandes cantidades de datos realistas de prueba generados aleatoriamente en función de sus propias especificaciones que luego puede cargar directamente en su entorno de prueba utilizando formatos CSV, JSON, XML, Excel, SQL, … No se requiere programación y es gratuita (para generar datos de 1.000 en 1.000).
- Los datos realistas son variados y contendrán caracteres que pueden no funcionar bien con nuestro código, como apóstrofes o caracteres unicode de otros idiomas. Las pruebas con datos realistas harán que la aplicación sea más robusta porque detectará errores en escenarios de producción.
- El proceso es sencillo ya que solo hay que ir añadiendo nombres de campos y escoger su tipo. Por defecto nos ofrece mas de 140 tipos de datos diferentes que van desde nombre y apellidos (pudiendo escoger estilo, género, etc...) hasta ISBNs, ubicaciones geográficas o datos encriptados simulados.
- Además es posible hacer que los datos sigan una distribución Normal o de Poisson, secuencias, que cumplan una expresión regular o incluso que fuercen cadenas complicadas con caracteres extraños y cosas así. Tenemos la posibilidad de crear fórmulas propias para generarlos, teniendo en cuenta otros campos, condicionales, etc... Es altamente flexible.

© JMA 2020. All rights reserved